

前言

类别	RTOS开发
文档名	FreeRTOS完全开发手册之上册: 快速使用
当前版本	V1.0
日期	2021.08.10
编辑	深圳百问网科技有限公司
审核	韦东山

百问网提供嵌入式软件开发的全栈式培训，涉及：

- 单片机：HAL库开发、寄存器开发+ARM架构、RTOS快速入门、RTOS内部实现
- Linux：嵌入式Linux应用开发基础、嵌入式Linux驱动开发基础、嵌入式Linux驱动大全、Linux项目开发
- Android

本教程属于"FreeRTOS完全开发手册之上册: 快速使用"，正在编写"FreeRTOS完全开发手册之下册: 内部实现"。

本文档参考FreeRTOS官方文档：

161204_Mastering_the_Freertos_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf

更多信息请关注：

- 官网：www.100ask.net
- 淘宝：100ask.taobao.com
- 微信公众号



第一章 FreeRTOS概述与体验

1.1 FreeRTOS目录结构

以Keil工具下STM32F103芯片为例，它的FreeRTOS的目录如下：

```
|-- FreeRTOS
|   |-- Demo                                // 预先制作好的示例工程
|   |   |-- CORTEX_STM32F103_Keil // STM32F103在Keil环境下的工程文件
|   |   |   |-- FreeRTOSConfig.h
|   |   |   |-- .....
|   |   |-- Common                        // 独立于demo的通用代码，大部分已经废弃
|   |-- Source                            // FreeRTOS源码
|   |   |-- croutine.c
|   |   |-- event_groups.c
|   |   |-- list.c
|   |   |-- queue.c
|   |   |-- stream_buffer.c
|   |   |-- tasks.c
|   |   |-- timers.c                    // 核心文件
|   |   |-- include
|   |   |   |-- portable                // 移植时需要实现的文件
|   |   |   |   |-- RVDS                // IDE为RVD或Keil
|   |   |   |   |   |-- ARM_CM3        // CortexM3架构
|   |   |   |   |   |   |-- port.c
|   |   |   |   |   |   |-- portmacro.h
|   |   |   |-- MemMang                  // 内存管理
|   |   |   |   |-- heap_1.c
|   |   |   |   |-- heap_2.c
|   |   |   |   |-- heap_3.c
|   |   |   |   |-- heap_4.c
|   |   |   |   |-- heap_5.c
|-- FreeRTOS-Plus                        // FreeRTOS生态的文件，非必需
|   |-- Demo
|   |-- Source
```

主要涉及2个目录：

- Demo
 - Demo目录下是工程文件，以"芯片和编译器"组合成一个名字
 - 比如：CORTEX_STM32F103_Keil
- Source
 - 根目录下是核心文件，这些文件是通用的
 - portable目录下是移植时需要实现的文件
 - 目录名为：[compiler]/[architecture]
 - 比如：RVDS/ARM_CM3，这表示cortexM3架构在RVDS工具上的移植文件

1.2 核心文件

FreeRTOS的最核心文件只有2个：

- FreeRTOS/Source/tasks.c
- FreeRTOS/Source/list.c

其他文件的作用也一起列表如下：

FreeRTOS/Source/下的文件	作用
tasks.c	必需，任务操作
list.c	必需，列表
queue.c	基本必需，提供队列操作、信号量(semaphore)操作
timer.c	可选，software timer
event_groups.c	可选，提供event group功能
croutine.c	可选，过时了

1.3 移植时涉及的文件

移植FreeRTOS时涉及的文件放在 `FreeRTOS/Source/portable/[compiler]/[architecture]` 目录下,

比如: `RVDS/ARM_CM3`, 这表示cortexM3架构在RVDS或Keil工具上的移植文件。

里面有2个文件:

- `port.c`
- `portmacro.h`

1.4 头文件相关

1.4.1 头文件目录

FreeRTOS需要3个头文件目录：

- FreeRTOS本身的头文件：FreeRTOS/Source/include
- 移植时用到的头文件：FreeRTOS/Source/portable/[compiler]/[architecture]
- 含有配置文件FreeRTOSConfig.h的目录

1.4.2 头文件

列表如下：

头文件	作用
FreeRTOSConfig.h	FreeRTOS的配置文件，比如选择调度算法：configUSE_PREEMPTION 每个demo都必定含有FreeRTOSConfig.h 建议去修改demo中的FreeRTOSConfig.h，而不是从头写一个
FreeRTOS.h	使用FreeRTOS API函数时，必须包含此文件。 在FreeRTOS.h之后，再去包含其他头文件，比如： task.h、queue.h、semphr.h、event_group.h

1.5 内存管理

文件在 FreeRTOS/Source/portable/MemMang 下，它也是放在 portable 目录下，表示你可以提供自己的函数。

源码中默认提供了5个文件，对应内存管理的5种方法。

参考文章：[FreeRTOS说明书吐血整理【适合新手+入门】](#)

后续章节会详细讲解。

文件	优点	缺点
heap_1.c	分配简单，时间确定	只分配、不回收
heap_2.c	动态分配、最佳匹配	碎片、时间不定
heap_3.c	调用标准库函数	速度慢、时间不定
heap_4.c	相邻空闲内存可合并	可解决碎片问题、时间不定
heap_5.c	在heap_4基础上支持分隔的内存块	可解决碎片问题、时间不定

1.6 Demo

Demo目录下是预先配置好的、没有编译错误的工程。目的是让你可以基于它进行修改，以适配你的单板。

这些Demo还可以继续精简：

- Demo/Common 中的文件可以完全删除
- main函数中只需要保留2个函数：
 - prvSetupHardware()
 - vTaskStartScheduler()
 - 如下图所示

```
int main( void )
{
    /* Perform any hardware setup necessary. */
    prvSetupHardware();

    /* --- APPLICATION TASKS CAN BE CREATED HERE --- */

    /* Start the created tasks running. */
    vTaskStartScheduler();

    /* Execution will only reach here if there was insufficient heap to
    start the scheduler. */
    for( ;; );
    return 0;
}
```

1.7 数据类型和编程规范

1.7.1 数据类型

每个移植的版本都含有自己的 `portmacro.h` 头文件，里面定义了2个数据类型：

- `TickType_t`:
 - FreeRTOS配置了一个周期性的时钟中断：Tick Interrupt
 - 每发生一次中断，中断次数累加，这被称为tick count
 - tick count这个变量的类型就是`TickType_t`
 - `TickType_t`可以是16位的，也可以是32位的
 - FreeRTOSConfig.h中定义`configUSE_16_BIT_TICKS`时，`TickType_t`就是`uint16_t`
 - 否则`TickType_t`就是`uint32_t`
 - 对于32位架构，建议把`TickType_t`配置为`uint32_t`
- `BaseType_t`:
 - 这是该架构最高效的数据类型
 - 32位架构中，它就是`uint32_t`
 - 16位架构中，它就是`uint16_t`
 - 8位架构中，它就是`uint8_t`
 - `BaseType_t`通常用作简单的返回值的类型，还有逻辑值，比如 `pdTRUE`/`pdFALSE`

1.7.2 变量名

变量名有前缀：

变量名前缀	含义
c	char
s	int16_t, short
l	int32_t, long
x	BaseType_t, 其他非标准的类型：结构体、task handle、queue handle等
u	unsigned
p	指针
uc	uint8_t, unsigned char
pc	char指针

1.7.3 函数名

函数名的前缀有2部分：返回值类型、在哪个文件定义。

函数名前缀	含义
vTaskPrioritySet	返回值类型: void 在task.c中定义
xQueueReceive	返回值类型: BaseType_t 在queue.c中定义
pvTimerGetTimerID	返回值类型: pointer to void 在tmer.c中定义

1.7.4 宏的名

宏的名字是大小，可以添加小写的前缀。前缀是用来表示：宏在哪个文件中定义。

宏的前缀	含义：在哪个文件里定义
port (比如portMAX_DELAY)	portable.h或portmacro.h
task (比如taskENTER_CRITICAL())	task.h
pd (比如pdTRUE)	projdefs.h
config (比如configUSE_PREEMPTION)	FreeRTOSConfig.h
err (比如errQUEUE_FULL)	projdefs.h

通用的宏定义如下：

宏	值
pdTRUE	1
pdFALSE	0
pdPASS	1
pdFAIL	0

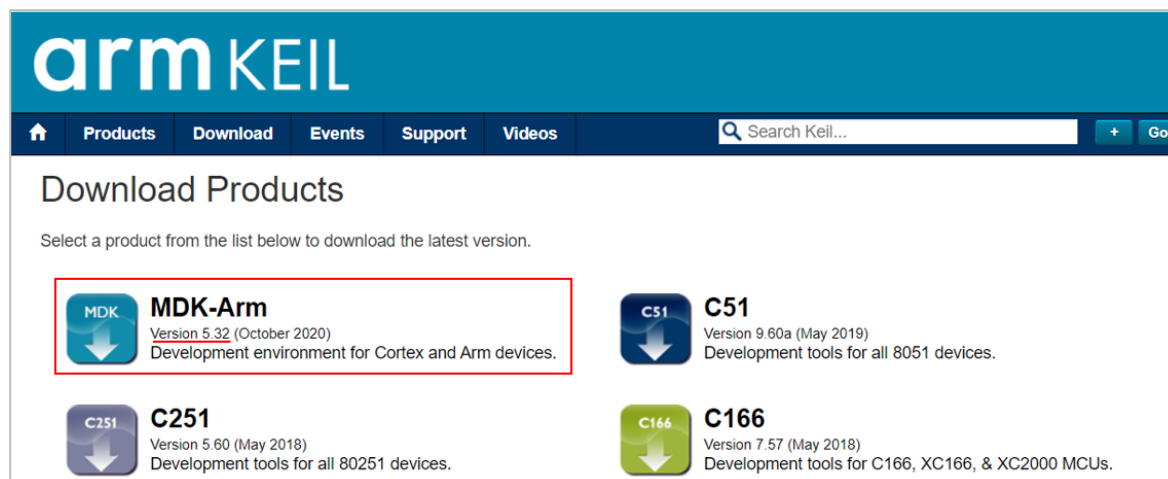
1.8 安装Keil

本教程的所有程序，都是使用Keil开发，运行在Keil的模拟器上。

1.8.1 下载Keil

Keil-MDK (Keil ARM Microcontroller Development Kit) 前生是德国Keil公司，后被ARM收购，是ARM官方的集成开发环境。

打开Keil官网 (<https://www.keil.com/download/product/>)，点击“MDK-Arm”进行下载。



随后进入个人信息完善页面，按提示填写所有的信息，如下图所示，填写完后，点击“Submit”提交。

MDK-ARM

MDK-ARM Version 5.32

Version 5.32

Complete the following form to download the Keil software development tools.

Enter Your Contact Information Below

First Name:	bai
Last Name:	wen
E-mail:	learn@100ask.net
Company:	100ask
Job Title:	software engineer
Country/Region:	China
Phone:	1234567890
☐ Send me e-mail when there is a new update.	
NOTICE: If you select this check box, you will receive an e-mail message from Keil whenever a new update is available. If you don't wish to receive an e-mail notification, don't check this box.	
Which device are you using? (eg, STM32)	STM32

Arm will process your information in accordance with the Evaluation section of our [Privacy Policy](#).

☐ Please keep me updated on products, services and other relevant offerings from Arm. You can change your mind and unsubscribe at any time.

Submit

Reset

随后进入下载页面，点击“MDK532.EXE”即可下载。

[Home](#) / [Product Downloads](#)

MDK-ARM

MDK-ARM Version 5.32

Version 5.32

- Review the [hardware requirements](#) before installing this software.
- Note the [limitations of the evaluation tools](#).
- [Further installation instructions for MDK5](#)

(MD5:66ee47746652ca12d57ec11c34561ece)

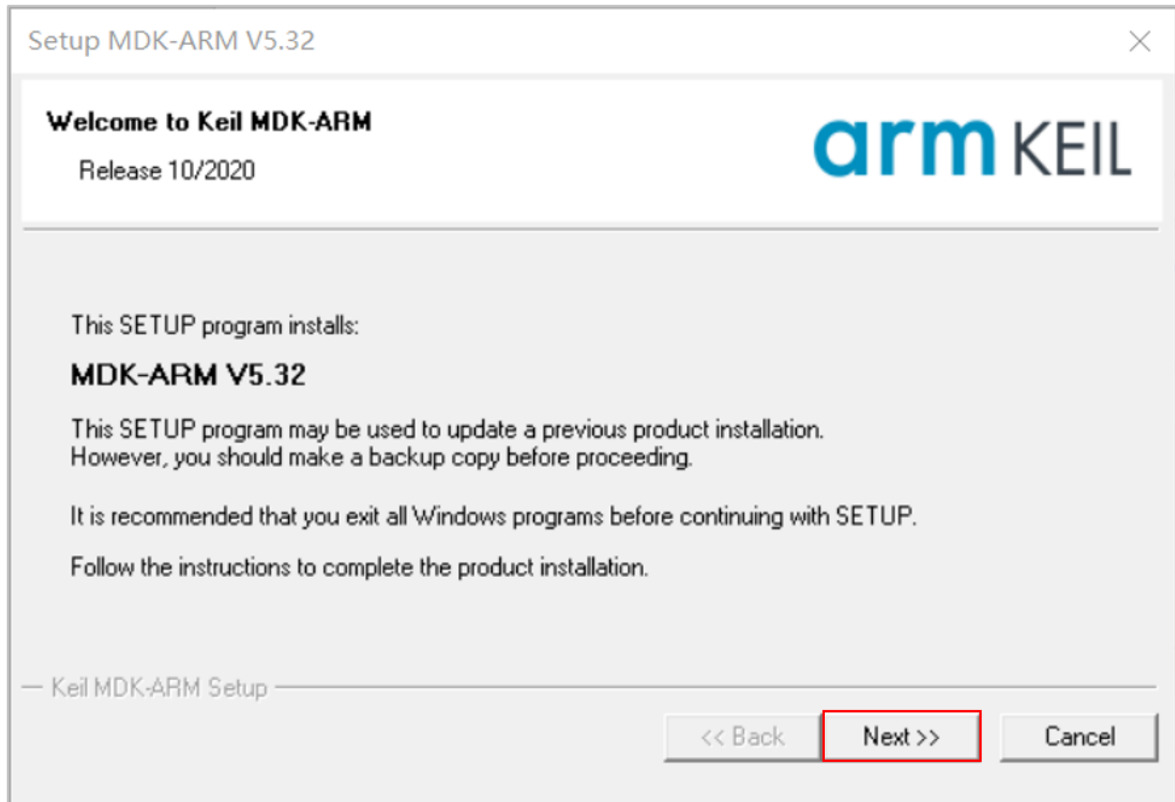
To install the MDK-ARM Software...

- Right-click on **MDK532.EXE** and save it to your computer.
- PDF files may be opened with Acrobat Reader.
- ZIP files may be opened with PKZIP or WINZIP.

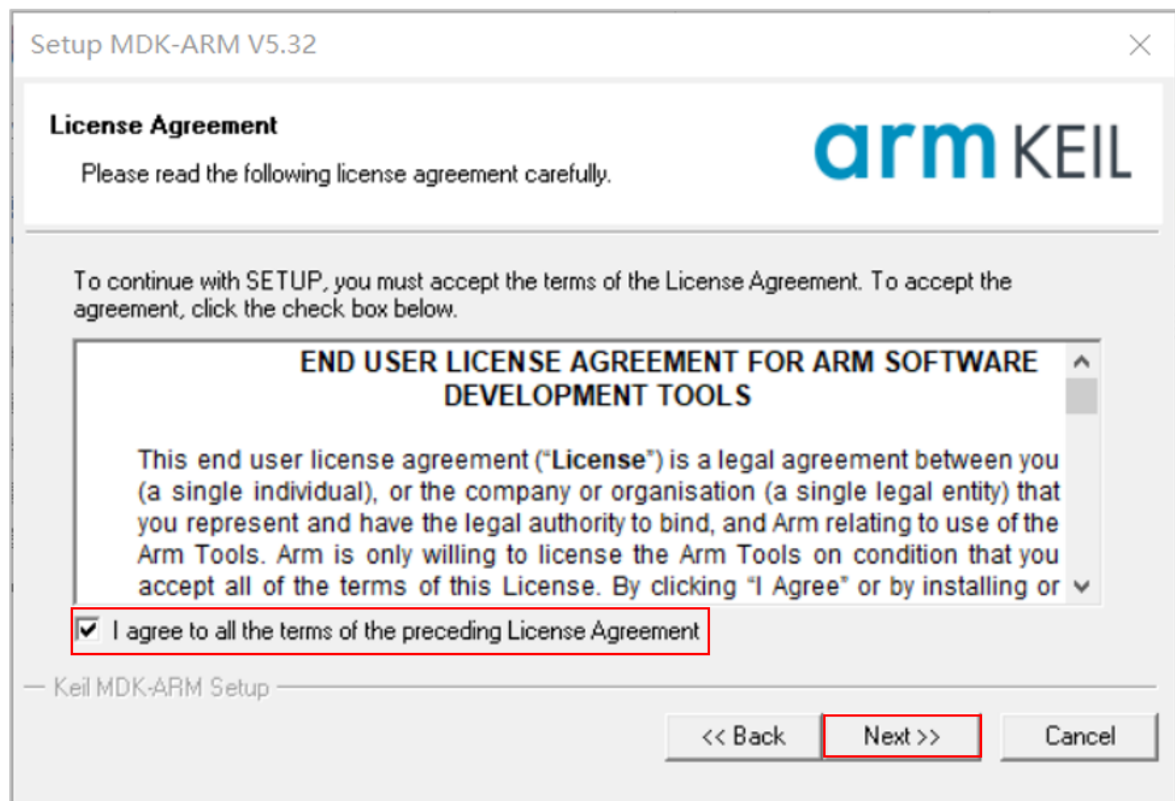
MDK532.EXE (914,178K)
Monday, October 5, 2020

1.8.2 安装Keil

下载完后，点击运行该文件，进入安装界面，选择“Next >>”。



接着进入用户协议界面，勾选同意协议，点击“Next >>”。



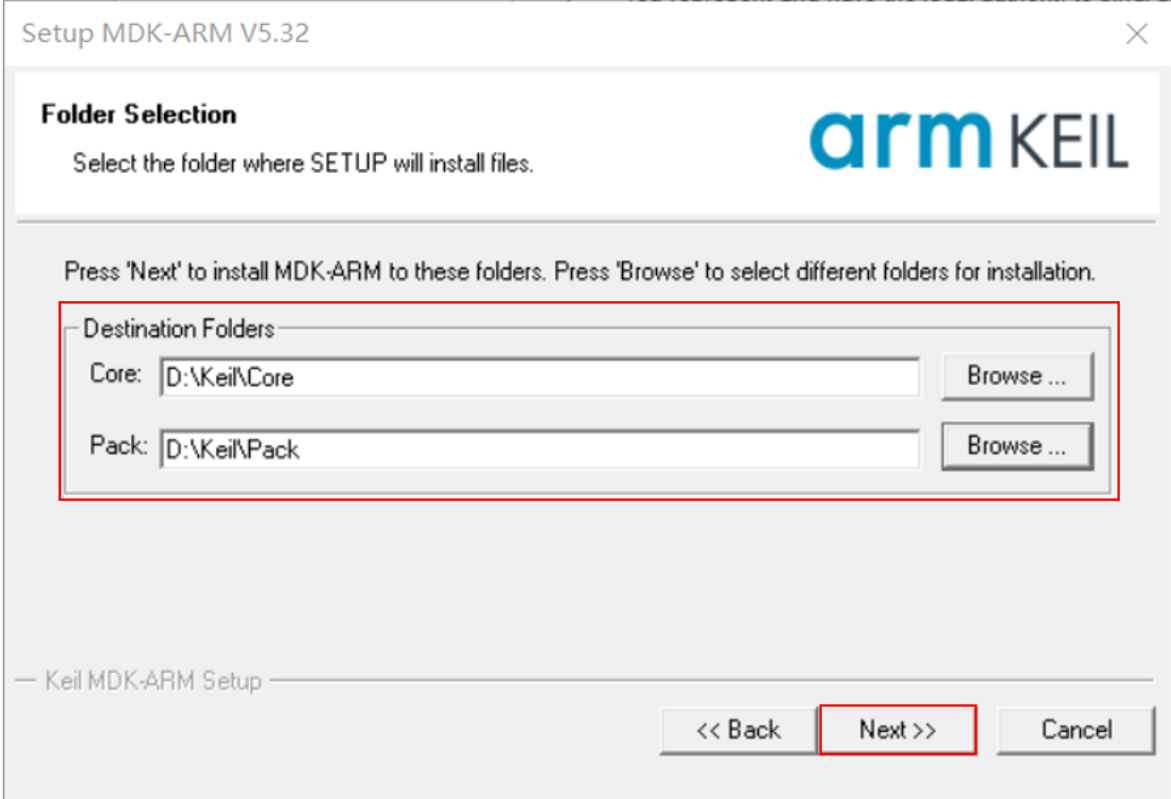
然后设置安装路径，如下图所示：

- 第一个“Core”是软件的安装路径，
- 第二个“Pack”是芯片的硬件支持包的安装路径，

保持默认路径或者设置为如下图所示一样的即可。

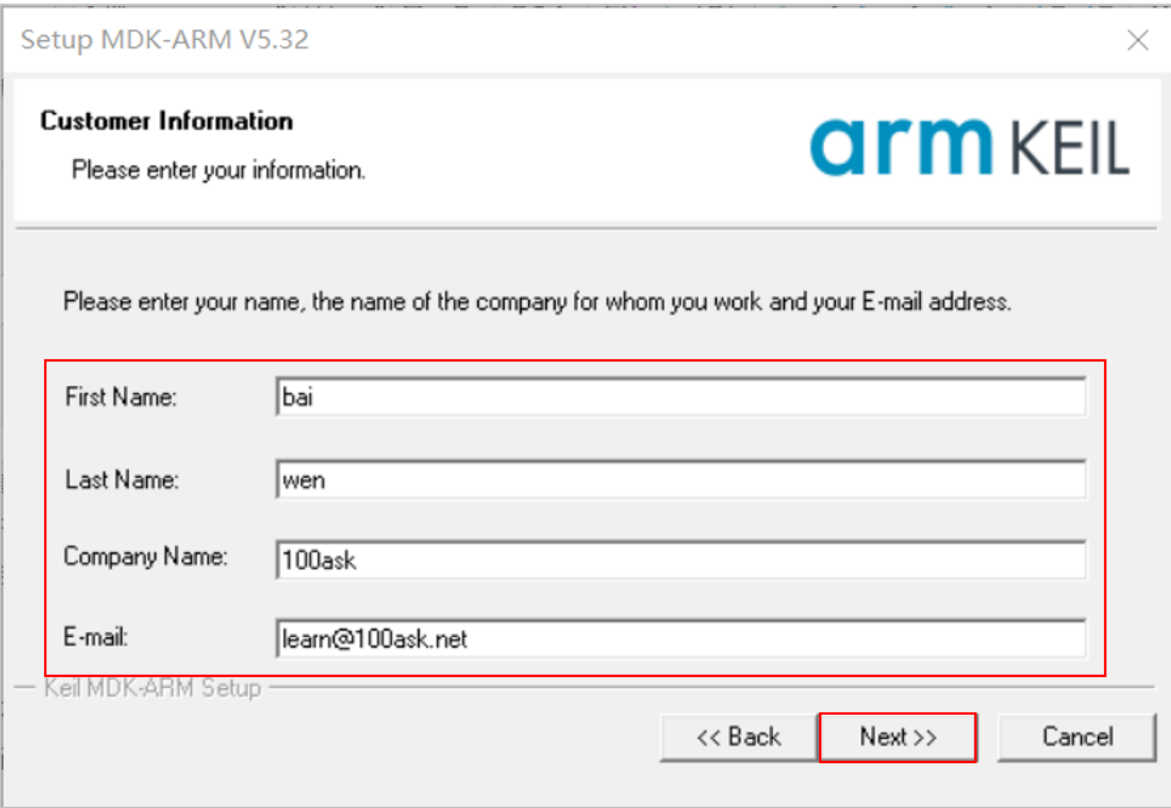
如果是自定义设置，建议为全英文路径，**不建议为包含有中文的路径。**

选择好之后点击“Next >>”。



The screenshot shows the 'Folder Selection' window of the 'Setup MDK-ARM V5.32' installer. The window has a title bar with the text 'Setup MDK-ARM V5.32' and a close button. Below the title bar is the 'arm KEIL' logo. The main heading is 'Folder Selection' with the instruction 'Select the folder where SETUP will install files.' Below this, a message says 'Press 'Next' to install MDK-ARM to these folders. Press 'Browse' to select different folders for installation.' The 'Destination Folders' section contains two text boxes: 'Core:' with the value 'D:\Keil\Core' and 'Pack:' with the value 'D:\Keil\Pack'. Each text box has a 'Browse ...' button to its right. At the bottom of the window, there are three buttons: '<< Back', 'Next >>', and 'Cancel'. The 'Next >>' button is highlighted with a red border.

随后需要设置个人信息，随便填写即可，如下图所示。



The screenshot shows the 'Customer Information' window of the 'Setup MDK-ARM V5.32' installer. The window has a title bar with the text 'Setup MDK-ARM V5.32' and a close button. Below the title bar is the 'arm KEIL' logo. The main heading is 'Customer Information' with the instruction 'Please enter your information.' Below this, a message says 'Please enter your name, the name of the company for whom you work and your E-mail address.' The 'Customer Information' section contains four text boxes: 'First Name:' with the value 'bai', 'Last Name:' with the value 'wen', 'Company Name:' with the value '100ask', and 'E-mail:' with the value 'learn@100ask.net'. At the bottom of the window, there are three buttons: '<< Back', 'Next >>', and 'Cancel'. The 'Next >>' button is highlighted with a red border.

之后便进入安装进度界面，等待安装完成。

安装过程中，回弹出驱动安装界面，勾选“始终信任来自‘ARM Ltd’的软件”，然后点击“安装”，如下图。



之后会自动进入“Pack Installer”界面，这里会检查安装的编译器、CMSIS等是否是最新的，由于我们安装的是官网提供的最新的MDK，所以这里一般情况下都是不需要更新的。

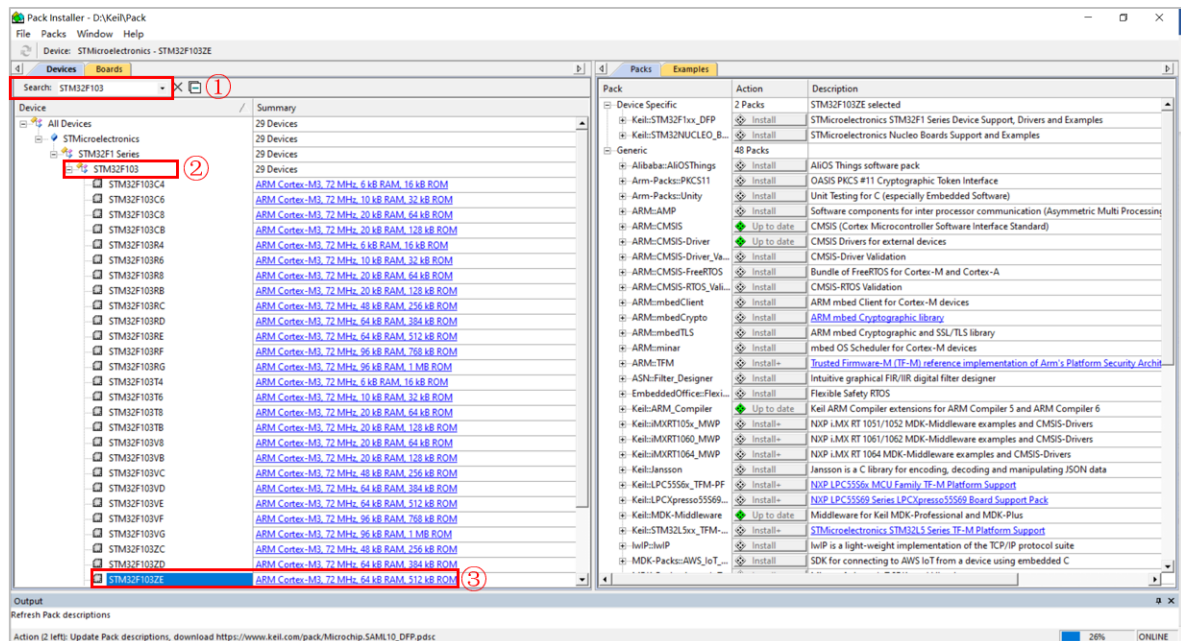
1.8.3 安装Pack

一个Keil的开发环境，除了Keil软件，还需要安装对应的Pack。

比如这里目标机的MCU是STM32F103ZET6，就需要下载该系列的的Pack，如果是STM32F4系列，就需要下其它系列Pack。

使用“Pack Installer”可以方便的对Pack安装和管理。

在左上角搜索框输入“STM32F103”，展开搜索结果，可以看到STM32F103ZE，点击右边的简介链接即可跳转到Pack下载页面。



Product Information

Product Overview
Supported Microcontrollers
Shows and Seminars

Technical Support

Support Knowledgebase
Product Manuals
Application Notes
Discussion Forum

Software Downloads

Product Downloads
File Downloads

Other Information

Books
Links

Contact Information

Corporate
Sales Channels
Distributors

Home / MDK5 Device List / STM32F103ZE

STMicroelectronics STM32F103ZE

ARM Cortex-M3, 72 MHz, 512 kB ROM, 64 kB RAM

STMicroelectronics' STM32F1 series of mainstream MCUs covers the needs of a large variety of applications in the industrial, medical and consumer markets. High performance with first-class peripherals and low-power, low-voltage operation is paired with a high level of integration at accessible prices with a simple architecture and easy-to-use tools. Typical applications include motor drives and application control, medical and handheld equipment, industrial applications, PLCs, inverters, printers, and scanners, alarm systems, video intercom, HVAC and home audio equipment. - LCD parallel interface, 8080/6800 modes - 5 V-tolerant I/Os - Timer with quadrature (incremental) encoder input - 96-bit unique ID

- **Core** ARM Cortex-M3, 72 MHz
- **Memory** 64 kB RAM, 512 kB ROM
- **Clock & Power** 2.00 V .. 3.60 V, 72 MHz
- **Communication** SPI, I2C, UART, I2S, CAN, USART, USB, Device

Quick Links

- Board List
- Software Packs
- MDK Version 5
- Legacy Support
- Feedback

Silicon Supplier

- STMicroelectronics



Device Family Pack

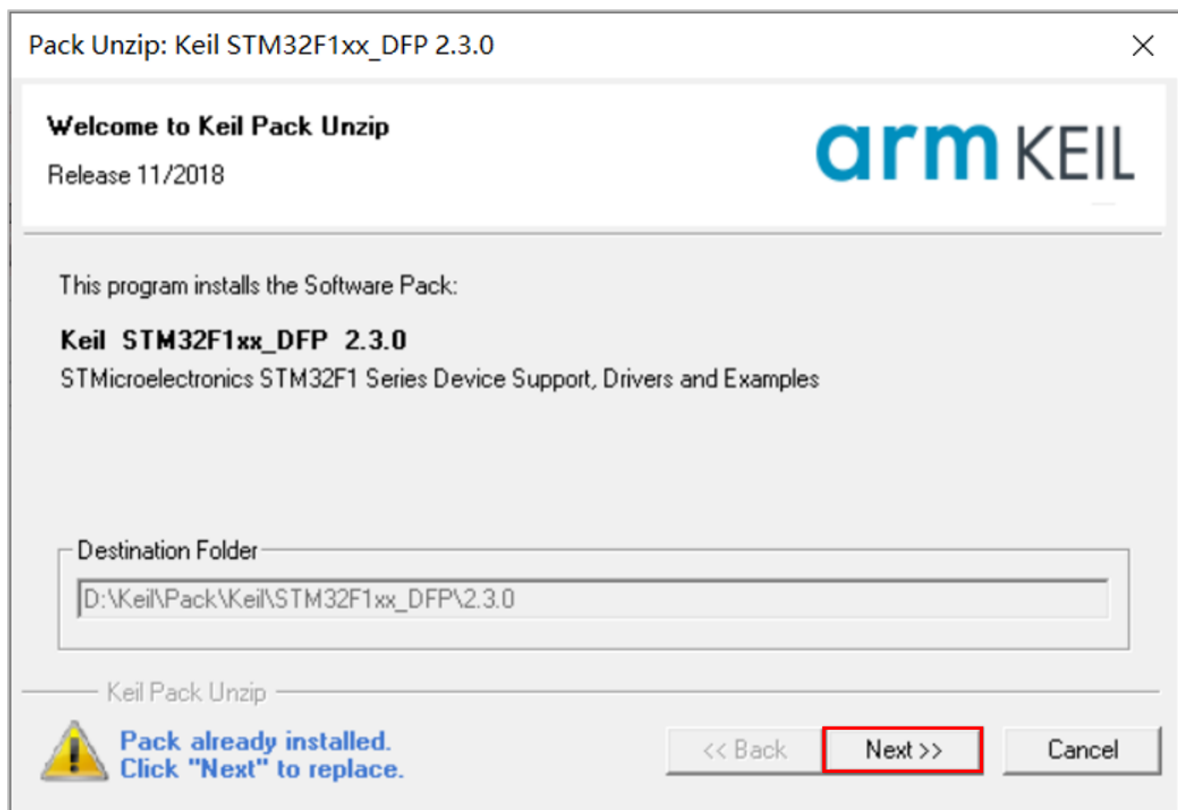
DFP

Support for this device is contained in:
STMicroelectronics STM32F1 Series
Device Support, Drivers and
Examples

Download

下载完成得到“Keil.STM32F1xx_DFP.2.3.0.pack”。

直接双击该文件，随后弹出如下图所示界面，点击“Next”进行安装。



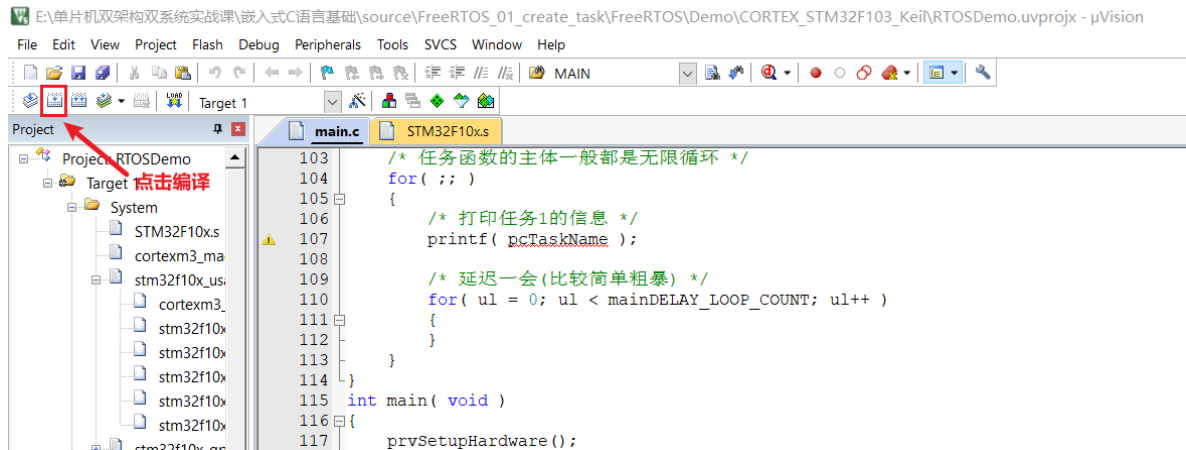
至此，Keil和Pack就安装完成了。

1.9 使用模拟器运行第1个程序

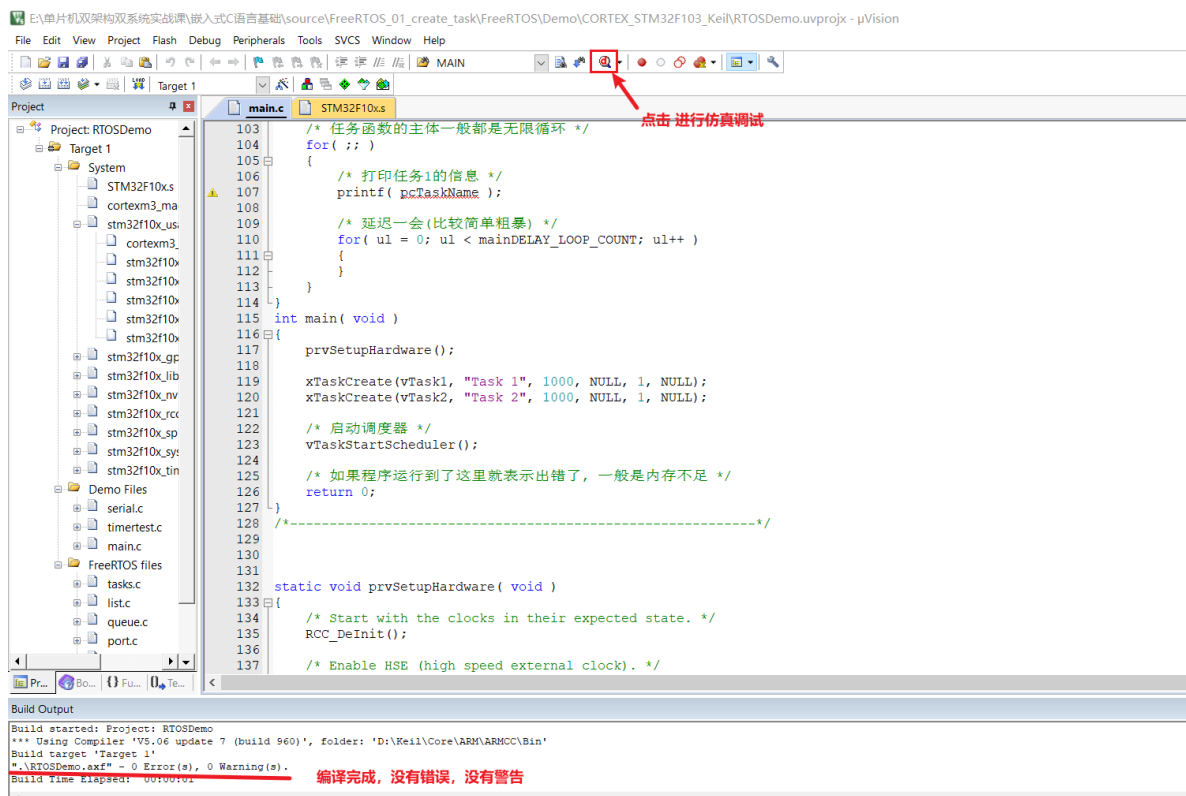
先获取配套示例代码。

双击"FreeRTOS_01_create_task\FreeRTOS\Demo\CORTEX_STM32F103_Keil\RTOSDemo.uvprojx"打开第一个示例。

打开之后，首先要**编译工程**，才能使用模拟器运行，点击"Build"图标进行编译，如下图所示：



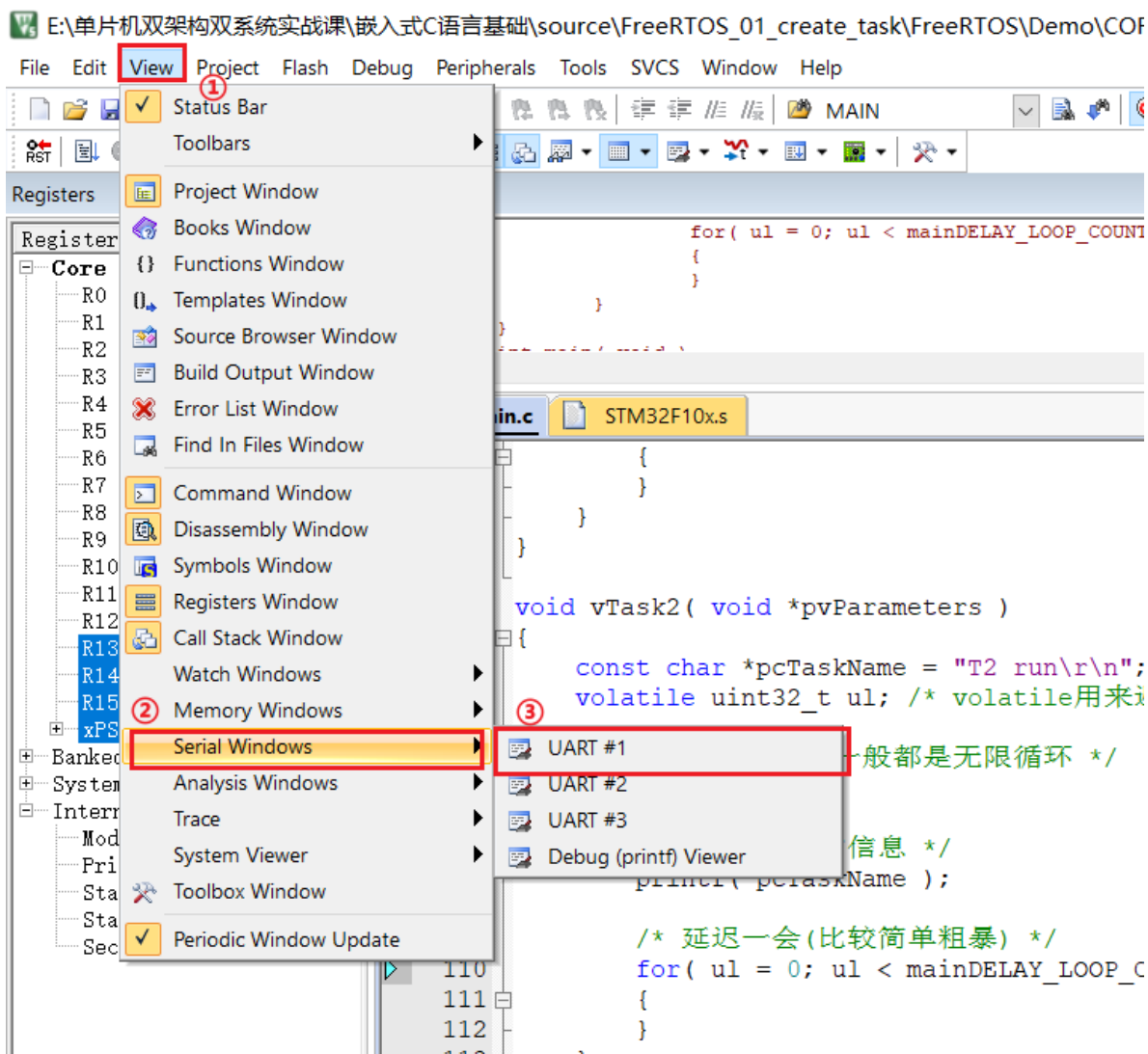
编译完成后，点击"Debug"按钮进行仿真，如下图所示：



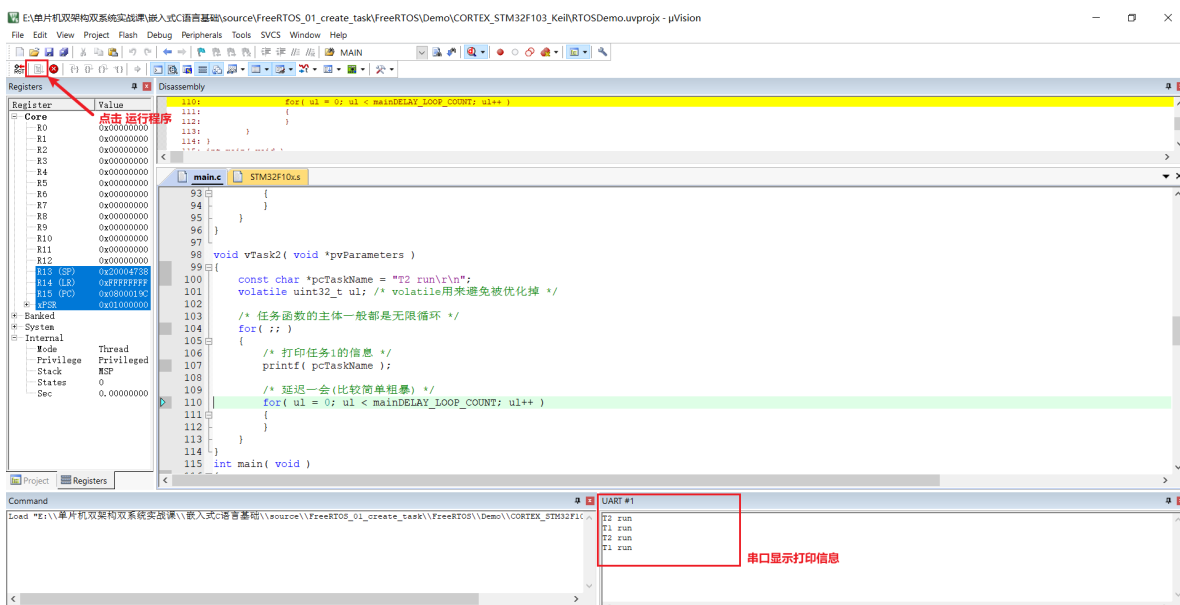
第一个程序里面创建了两个任务，两个任务一直打印各自的信息。

这里需要打开串口显示模拟窗口，显示任务的打印内容。

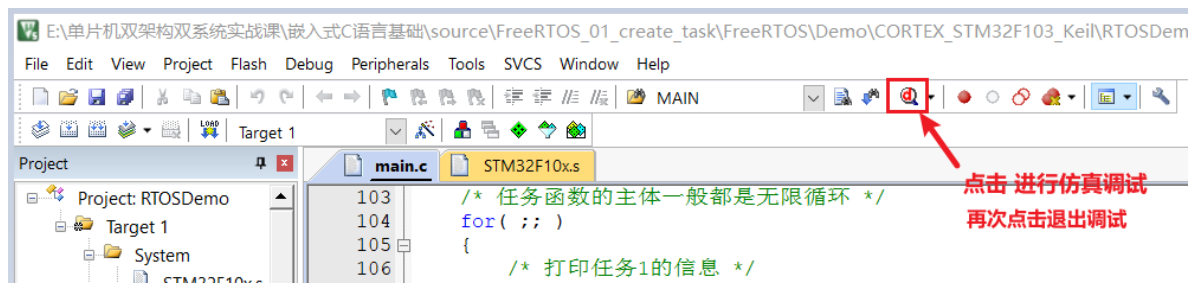
点击左上角菜单的"View"，然后选择"Serial Windows"，点击"UART #1"，如下图所示：



最后，点击“Run”运行程序，右下角串口显示窗口将打印两个任务的信息。



如果想退出模拟器仿真，再次“Debug”按钮退出，如下图所示：



1.10 使用逻辑分析仪

本课程的程序有两种输出方式：

- 串口：查看打印信息
- 逻辑分析仪：观察全局变量的波形，根据波形解析任务调度情况

下面举例说明逻辑分析仪的用法。

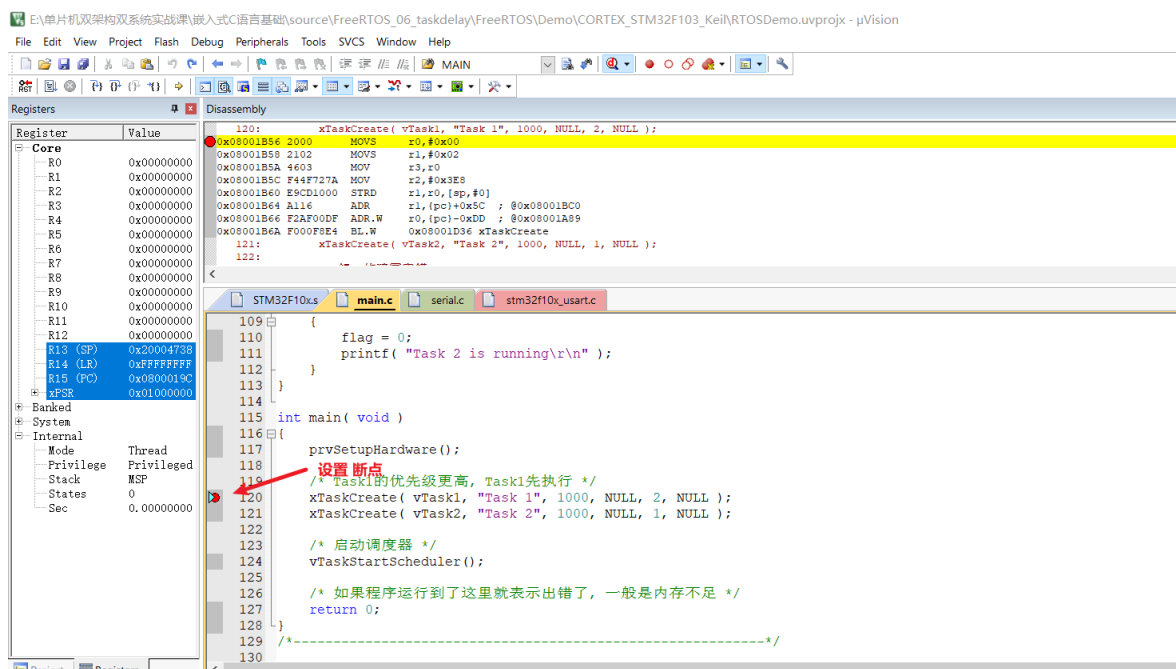
双击"FreeRTOS_06_taskdelay\FreeRTOS\Demo\CORTEX_STM32F103_Keil\RTOSDemo.uvprojx"打开该示例。

打开之后，首先要**编译工程**，点击"Build"图标进行编译。

编译完成后，点击"Debug"按钮进行仿真。

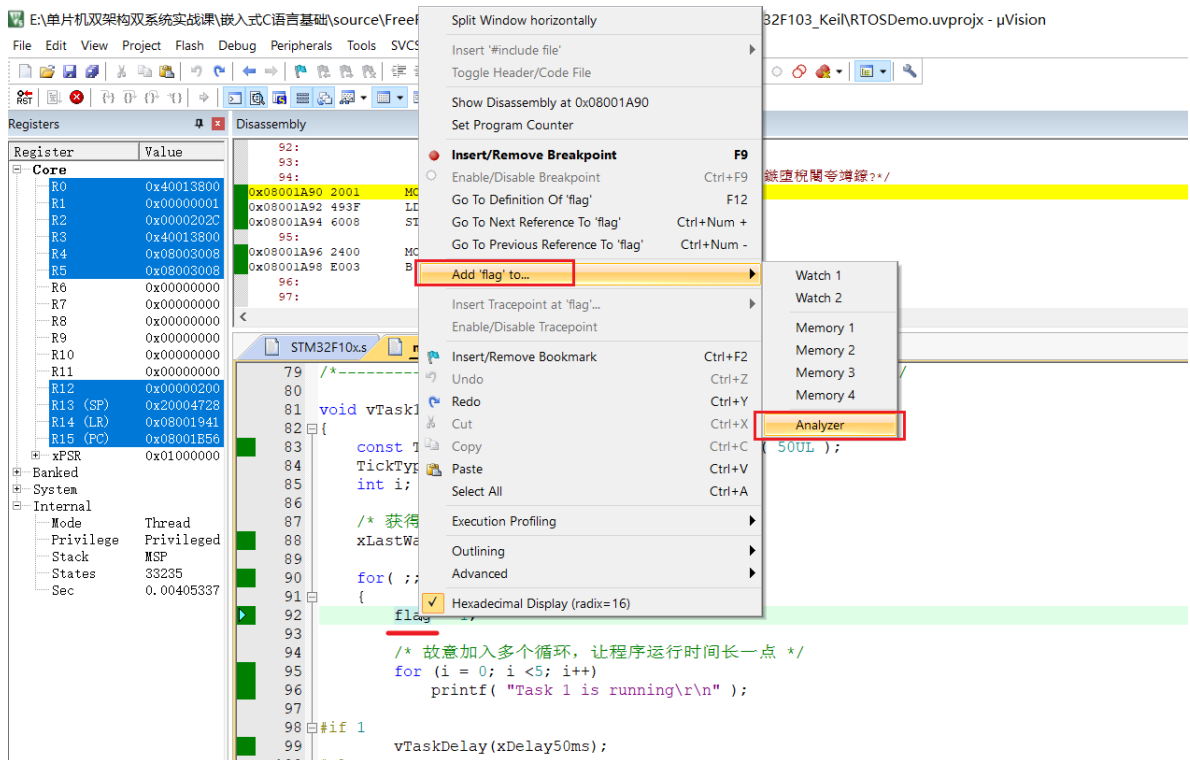
本实例使用模拟器的逻辑分析仪观察现象。

首先在“main.c”的主函数加入断点，在代码行前的灰色处，点击一下就会有一个红色小点，就是设置的“断点”。



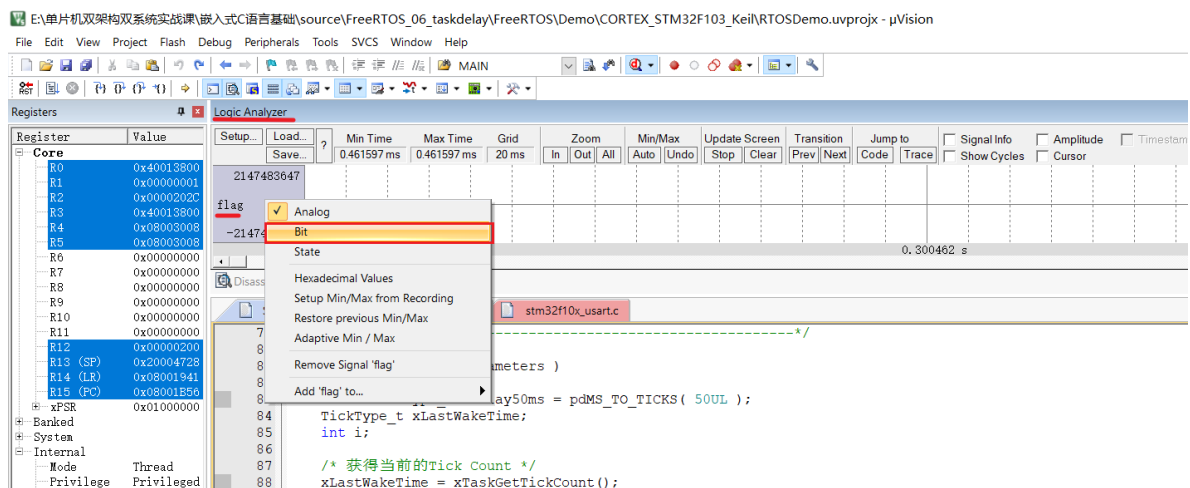
然后点击"Run"运行，程序运行到断点位置，就会停下来等待下一步操作：

- 在代码中找到全局变量flag
- 鼠标选中flag，然后点击鼠标右键，在弹出的菜单里选择"Add 'flag' to...", 选择"Analyzer"，如下图所示：

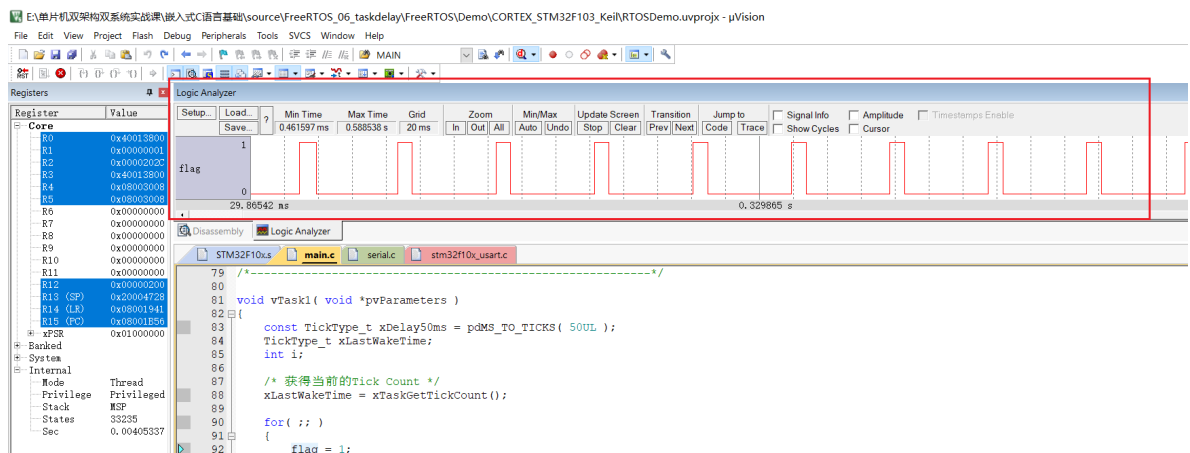


此时在代码框上面，就会出现逻辑分析仪“Logic Analyzer”显示窗口，里面分析的就是变量flag。

点击这个flag，然后右键，选择“Bit”，以便观察，如下图所示：



再点击一下“Run”，继续运行，此时逻辑分析仪窗口显示变量flag的bit值变化，如下图所示：



在逻辑分析仪窗口，可以使用鼠标滚轮放大、缩小波形。

第二章 内存管理

2.1 为什么要自己实现内存管理

后续的章节涉及这些内核对象：task、queue、semaphores和event group等。为了让FreeRTOS更容易使用，这些内核对象一般都是动态分配：用到时分配，不使用时释放。使用内存的动态管理功能，简化了程序设计：不再需要小心翼翼地提前规划各类对象，简化API函数的涉及，甚至可以减少内存的使用。

内存的动态管理是C程序的知识范畴，并不属于FreeRTOS的知识范畴，但是它跟FreeRTOS关系是如此紧密，所以我们先讲解它。

在C语言的库函数中，有malloc、free等函数，但是在FreeRTOS中，它们不适用：

- 不适合用在资源紧缺的嵌入式系统中
- 这些函数的实现过于复杂、占据的代码空间太大
- 并非线程安全的(thread-safe)
- 运行有不确定性：每次调用这些函数时花费的时间可能都不相同
- 内存碎片化
- 使用不同的编译器时，需要进行复杂的配置
- 有时候难以调试

注意：我们经常"堆栈"混合着说，其实它们不是同一个东西：

- 堆，heap，就是一块空闲的内存，需要提供管理函数
 - malloc：从堆里划出一块空间给程序使用
 - free：用完后，再把它标记为"空闲"的，可以再次使用
- 栈，stack，函数调用时局部变量保存在栈中，当前程序的环境也是保存在栈中
 - 可以从堆中分配一块空间用作栈



被管理的话
就叫做堆(heap)



栈寄存器指向
就叫做栈(stack)

2.2 FreeRTOS的5中内存管理方法

FreeRTOS中内存管理的接口函数为：pvPortMalloc 、vPortFree，对应于C库的malloc、free。

文件在 FreeRTOS/Source/portable/MemMang 下，它也是放在 portable 目录下，表示你可以提供自己的函数。

源码中默认提供了5个文件，对应内存管理的5种方法。

参考文章：[FreeRTOS说明书吐血整理【适合新手+入门】](#)

文件	优点	缺点
heap_1.c	分配简单，时间确定	只分配、不回收
heap_2.c	动态分配、最佳匹配	碎片、时间不定
heap_3.c	调用标准库函数	速度慢、时间不定
heap_4.c	相邻空闲内存可合并	可解决碎片问题、时间不定
heap_5.c	在heap_4基础上支持分隔的内存块	可解决碎片问题、时间不定

2.2.1 Heap_1

它只实现了pvPortMalloc，没有实现vPortFree。

如果你的程序不需要删除内核对象，那么可以使用heap_1：

- 实现最简单
- 没有碎片问题
- 一些要求非常严格的系统里，不允许使用动态内存，就可以使用heap_1

它的实现原理很简单，首先定义一个大数组：

```
/* Allocate the memory for the heap. */
#if ( configAPPLICATION_ALLOCATED_HEAP == 1 )

/* The application writer has already defined the array used for the RTOS
 * heap - probably so it can be placed in a special segment or address. */
extern uint8_t ucHeap[ configTOTAL_HEAP_SIZE ];
#else
static uint8_t ucHeap[ configTOTAL_HEAP_SIZE ];
#endif /* configAPPLICATION_ALLOCATED_HEAP */
```

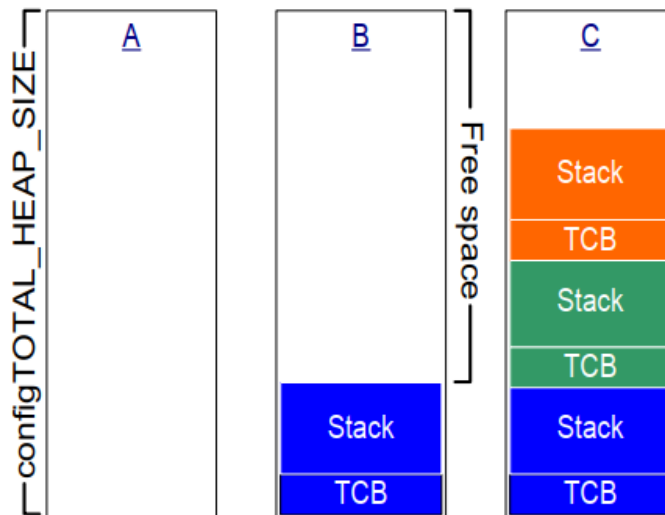
然后，对于pvPortMalloc调用时，从这个数组中分配空间。

FreeRTOS在创建任务时，需要2个内核对象：task control block(TCB)、stack。

使用heap_1时，内存分配过程如下图所示：

- A：创建任务之前整个数组都是空闲的

- B: 创建第1个任务之后，蓝色区域被分配出去了
- C: 创建3个任务之后的数组使用情况



RAM being allocated from the heap_1 array each time a task is created

2.2.2 Heap_2

Heap_2之所以还保留，只是为了兼容以前的代码。新设计中不再推荐使用Heap_2。建议使用Heap_4来替代Heap_2，更加高效。

Heap_2也是在数组上分配内存，跟Heap_1不一样的地方在于：

- Heap_2使用**最佳匹配算法**(best fit)来分配内存
- 它支持vPortFree

最佳匹配算法：

- 假设heap有3块空闲内存：5字节、25字节、100字节
- pvPortMalloc想申请20字节
- 找出最小的、能满足pvPortMalloc的内存：25字节
- 把它划分为20字节、5字节
 - 返回这20字节的地址
 - 剩下的5字节仍然是空闲状态，留给后续的pvPortMalloc使用

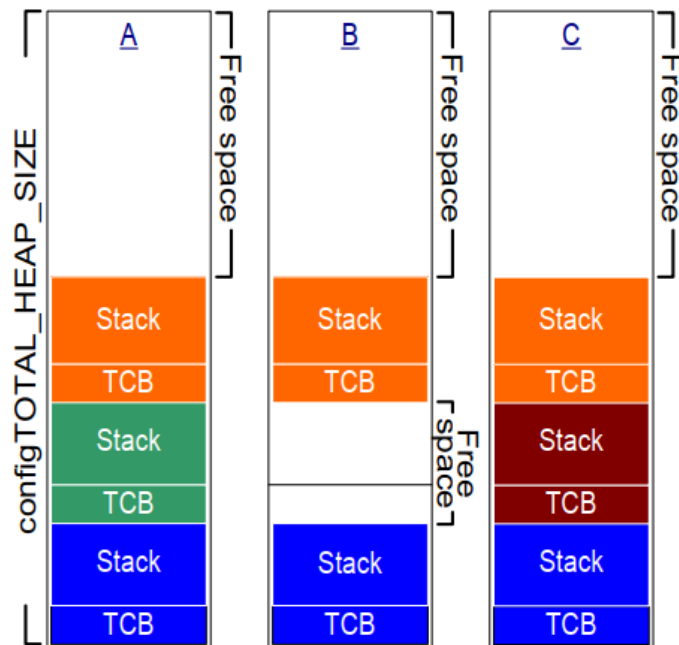
与Heap_4相比，Heap_2不会合并相邻的空闲内存，所以Heap_2会导致严重的"碎片化"问题。

但是，如果申请、分配内存时大小总是相同的，这类场景下Heap_2没有碎片化的问题。所以它适合这种场景：频繁地创建、删除任务，但是任务的栈大小都是相同的(创建任务时，需要分配TCB和栈，TCB总是一样的)。

虽然不再推荐使用heap_2，但是它的效率还是远高于malloc、free。

使用heap_2时，内存分配过程如下图所示：

- A: 创建了3个任务
- B: 删除了一个任务，空闲内存有3部分：顶层的、被删除任务的TCB空间、被删除任务的Stack空间
- C: 创建了一个新任务，因为TCB、栈大小跟前面被删除任务的TCB、栈大小一致，所以刚好分配到原来的内存



RAM being allocated and freed from the heap_2 array as tasks are created and deleted

2.2.3 Heap_3

Heap_3使用标准C库里的malloc、free函数，所以堆大小由链接器的配置决定，配置项configTOTAL_HEAP_SIZE不再起作用。

C库里的malloc、free函数并非线程安全的，Heap_3中先暂停FreeRTOS的调度器，再去调用这些函数，使用这种方法实现了线程安全。

2.2.4 Heap_4

跟Heap_1、Heap_2一样，Heap_4也是使用大数组来分配内存。

Heap_4使用**首次适应算法**(first fit)来分配内存。它还会把相邻的空闲内存合并为一个更大的空闲内存，这有助于较少内存的碎片问题。

首次适应算法：

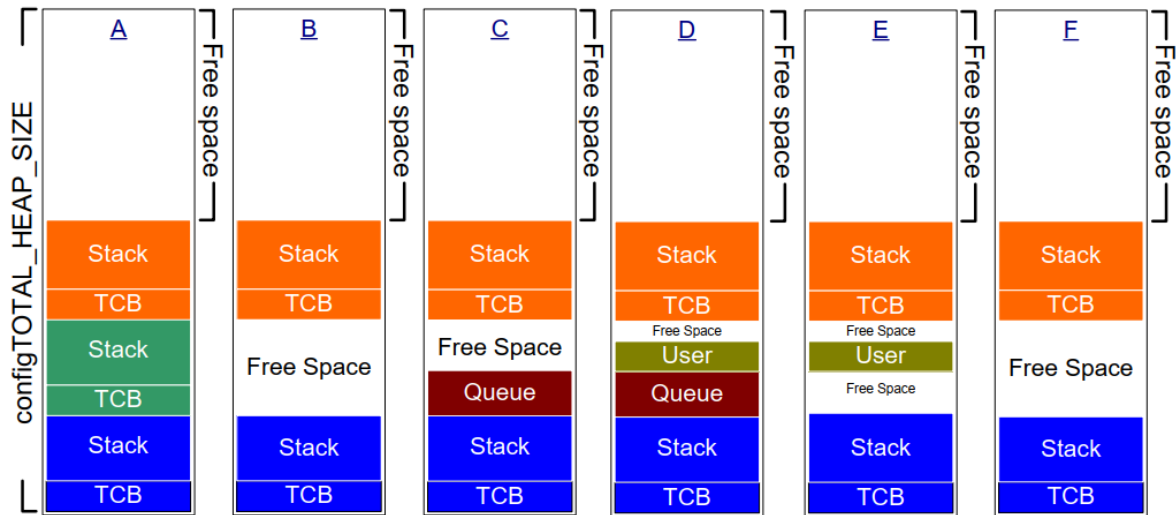
- 假设堆中有3块空闲内存：5字节、200字节、100字节
- pvPortMalloc想申请20字节
- 找出第1个能满足pvPortMalloc的内存：200字节
- 把它划分为20字节、180字节
 - 返回这20字节的地址
 - 剩下的180字节仍然是空闲状态，留给后续的pvPortMalloc使用

Heap_4会把相邻空闲内存合并为一个大的空闲内存，可以较少内存的碎片化问题。适用于这种场景：频繁地分配、释放不同大小的内存。

Heap_4的使用过程举例如下：

- A：创建了3个任务
- B：删除了一个任务，空闲内存有2部分：

- 顶层的
- 被删除任务的TCB空间、被删除任务的Stack空间合并起来的
- C: 分配了一个Queue, 从第1个空闲块中分配空间
- D: 分配了一个User数据, 从Queue之后的空闲块中分配
- E: 释放的Queue, User前后都有一块空闲内存
- F: 释放了User数据, User前后的内存、User本身占据的内存, 合并为一个大的空闲内存



RAM being allocated and freed from the heap_4 array

Heap_4执行的时间是不确定的, 但是它的效率高于标准库的malloc、free。

2.2.5 Heap_5

Heap_5分配内存、释放内存的算法跟Heap_4是一样的。

相比于Heap_4, Heap_5并不局限于管理一个大数组: 它可以管理多块、分隔开的内存。

在嵌入式系统中, 内存的地址可能并不连续, 这种场景下可以使用Heap_5。

既然内存时分隔开的, 那么就需要进行初始化: 确定这些内存块在哪、多大:

- 在使用pvPortMalloc之前, 必须先指定内存块的信息
- 使用vPortDefineHeapRegions来指定这些信息

怎么指定一块内存? 使用如下结构体:

```
typedef struct HeapRegion
{
    uint8_t * pucStartAddress; // 起始地址
    size_t xSizeInBytes;       // 大小
} HeapRegion_t;
```

怎么指定多块内存? 使用一个HeapRegion_t数组, 在这个数组中, 低地址在前、高地址在后。

比如:

```
HeapRegion_t xHeapRegions[] =
{
    { ( uint8_t * ) 0x80000000UL, 0x10000 }, // 起始地址0x80000000, 大小0x10000
    { ( uint8_t * ) 0x90000000UL, 0xa0000 }, // 起始地址0x90000000, 大小0xa0000
    { NULL, 0 } // 表示数组结束
};
```

vPortDefineHeapRegions函数原型如下:

```
void vPortDefineHeapRegions( const HeapRegion_t * const pxHeapRegions );
```

把xHeapRegions数组传给vPortDefineHeapRegions函数, 即可初始化Heap_5。

2.3 Heap相关的函数

2.3.1 pvPortMalloc/vPortFree

函数原型:

```
void * pvPortMalloc( size_t xwantedSize );  
void vPortFree( void * pv );
```

作用: 分配内存、释放内存。

如果分配内存不成功, 则返回值为NULL。

2.3.2 xPortGetFreeHeapSize

函数原型:

```
size_t xPortGetFreeHeapSize( void );
```

当前还有多少空闲内存, 这函数可以用来优化内存的使用情况。比如当所有内核对象都分配好后, 执行此函数返回2000, 那么configTOTAL_HEAP_SIZE就可减小2000。

注意: 在heap_3中无法使用。

2.3.3 xPortGetMinimumEverFreeHeapSize

函数原型:

```
size_t xPortGetMinimumEverFreeHeapSize( void );
```

返回: 程序运行过程中, 空闲内存容量的最小值。

注意: 只有heap_4、heap_5支持此函数。

2.3.4 malloc失败的钩子函数

在pvPortMalloc函数内部:

```
void * pvPortMalloc( size_t xwantedSize ) vPortDefineHeapRegions  
{  
    .....  
    #if ( configUSE_MALLOC_FAILED_HOOK == 1 )  
    {  
        if( pvReturn == NULL )  
        {  
            extern void vApplicationMallocFailedHook( void );  
            vApplicationMallocFailedHook();  
        }  
    }  
}
```

```
#endif  
  
    return pvReturn;  
}
```

所以，如果想使用这个钩子函数：

- 在FreeRTOSConfig.h中，把configUSE_MALLOC_FAILED_HOOK定义为1
- 提供vApplicationMallocFailedHook函数
- pvPortMalloc失败时，才会调用此函数

第三章 任务管理

在本章中，会涉及如下内容：

- FreeRTOS如何给每个任务分配CPU时间
- 如何选择某个任务来运行
- 任务优先级如何起作用
- 任务有哪些状态
- 如何实现任务
- 如何使用任务参数
- 怎么修改任务优先级
- 怎么删除任务
- 怎么实现周期性的任务
- 如何使用空闲任务

3.1 基本概念

对于整个单片机程序，我们称之为application，应用程序。

使用FreeRTOS时，我们可以在application中创建多个任务(task)，有些文档把任务也称为线程(thread)。



以日常生活为例，比如这个母亲要同时做两件事：

- 喂饭：这是一个任务
- 回信息：这是另一个任务

这可以引入很多概念：

- 任务状态(State):
 - 当前正在喂饭，它是running状态；另一个"回信息"的任务就是"not running"状态
 - "not running"状态还可以细分：
 - ready：就绪，随时可以运行
 - blocked：阻塞，卡住了，母亲在等待同事回信息
 - suspended：挂起，同事废话太多，不管他了
- 优先级(Priority)
 - 我工作生活兼顾：喂饭、回信息优先级一样，轮流做
 - 我忙里偷闲：还有空闲任务，休息一下
 - 厨房着火了，什么都别说了，先灭火：优先级更高
- 栈(Stack)
 - 喂小孩时，我要记得上一口喂了米饭，这口要喂青菜了
 - 回信息时，我要记得刚才聊的是啥
 - 做不同的任务，这些细节不一样
 - 对于人来说，当然是记在脑子里
 - 对于程序，是记在栈里
 - 每个任务有自己的栈
- 事件驱动
 - 孩子吃饭太慢：先休息一会，等他咽下去了、等他提醒我了，再喂下一口
- 协助式调度(Co-operative Scheduling)
 - 你在给同事回信息
 - 同事说：好了，你先去给小孩喂一口饭吧，你才能离开
 - 同事不放你走，即使孩子哭了你也不能走

- 你好不容易可以给孩子喂饭了
 - 孩子说：好了，妈妈你去处理一下工作吧，你才能离开
 - 孩子不放你走，即使同事连发信息你也不能走

这涉及很多概念，后续章节详细分析。

3.2 任务创建与删除

3.2.1 什么是任务

在FreeRTOS中，任务就是一个函数，原型如下：

```
void ATaskFunction( void *pvParameters );
```

要注意的是：

- 这个函数不能返回
- 同一个函数，可以用来创建多个任务；换句话说，多个任务可以运行同一个函数
- 函数内部，尽量使用局部变量：
 - 每个任务都有自己的栈
 - 每个任务运行这个函数时
 - 任务A的局部变量放在任务A的栈里、任务B的局部变量放在任务B的栈里
 - 不同任务的局部变量，有自己的副本
 - 函数使用全局变量、静态变量的话
 - 只有一个副本：多个任务使用的是同一个副本
 - 要防止冲突(后续会讲)

下面是一个示例：

```
void ATaskFunction( void *pvParameters )
{
    /* 对于不同的任务，局部变量放在任务的栈里，有各自的副本 */
    int32_t lvariableExample = 0;

    /* 任务函数通常实现为一个无限循环 */
    for( ;; )
    {
        /* 任务的代码 */
    }

    /* 如果程序从循环中退出，一定要使用vTaskDelete删除自己
     * NULL表示删除的是自己
     */
    vTaskDelete( NULL );

    /* 程序不会执行到这里，如果执行到这里就出错了 */
}
```

3.2.2 创建任务

创建任务时使用的函数如下：

```

 BaseType_t xTaskCreate( TaskFunction_t pxTaskCode, // 函数指针，任务函数
                        const char * const pcName, // 任务的名字
                        const configSTACK_DEPTH_TYPE usStackDepth, // 栈大小,单位为word,10表示40字节
                        void * const pvParameters, // 调用任务函数时传入的参数
                        UBaseType_t uxPriority, // 优先级
                        TaskHandle_t * const pxCreatedTask ); // 任务句柄，以后使用它来操作这个任务

```

参数说明：

参数	描述
pvTaskCode	函数指针，可以简单地认为任务就是一个C函数。 它稍微特殊一点：永远不退出，或者退出时要调用"vTaskDelete(NULL)"
pcName	任务的名字，FreeRTOS内部不使用它，仅仅起调试作用。 长度为：configMAX_TASK_NAME_LEN
usStackDepth	每个任务都有自己的栈，这里指定栈大小。 单位是word，比如传入100，表示栈大小为100 word，也就是400字节。 最大值为uint16_t的最大值。 怎么确定栈的大小，并不容易，很多时候是估计。 精确的办法是看反汇编码。
pvParameters	调用pvTaskCode函数指针时用到：pvTaskCode(pvParameters)
uxPriority	优先级范围：0~(configMAX_PRIORITIES - 1) 数值越小优先级越低， 如果传入过大的值，xTaskCreate会把它调整为(configMAX_PRIORITIES - 1)
pxCreatedTask	用来保存xTaskCreate的输出结果：task handle。 以后如果想操作这个任务，比如修改它的优先级，就需要这个handle。 如果不想使用该handle，可以传入NULL。
返回值	成功：pdPASS； 失败：errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY(失败原因只有内存不足) 注意：文档里都说失败时返回值是pdFAIL，这不对。 pdFAIL是0，errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY是-1。

3.2.3 示例1: 创建任务

代码为：FreeRTOS_01_create_task

使用2个函数分别创建2个任务。

任务1的代码：

```

void vTask1( void *pvParameters )
{
    const char *pcTaskName = "T1 run\r\n";

```

```

volatile uint32_t u1; /* volatile用来避免被优化掉 */

/* 任务函数的主体一般都是无限循环 */
for( ;; )
{
    /* 打印任务1的信息 */
    printf( pcTaskName );

    /* 延迟一会(比较简单粗暴) */
    for( u1 = 0; u1 < mainDELAY_LOOP_COUNT; u1++ )
    {
    }
}
}

```

任务2的代码：

```

void vTask2( void *pvParameters )
{
    const char *pcTaskName = "T2 run\r\n";
    volatile uint32_t u1; /* volatile用来避免被优化掉 */

    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        /* 打印任务1的信息 */
        printf( pcTaskName );

        /* 延迟一会(比较简单粗暴) */
        for( u1 = 0; u1 < mainDELAY_LOOP_COUNT; u1++ )
        {
        }
    }
}

```

main函数：

```

int main( void )
{
    prvSetupHardware();

    xTaskCreate(vTask1, "Task 1", 1000, NULL, 1, NULL);
    xTaskCreate(vTask2, "Task 2", 1000, NULL, 1, NULL);

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

运行结果如下：

UART #1

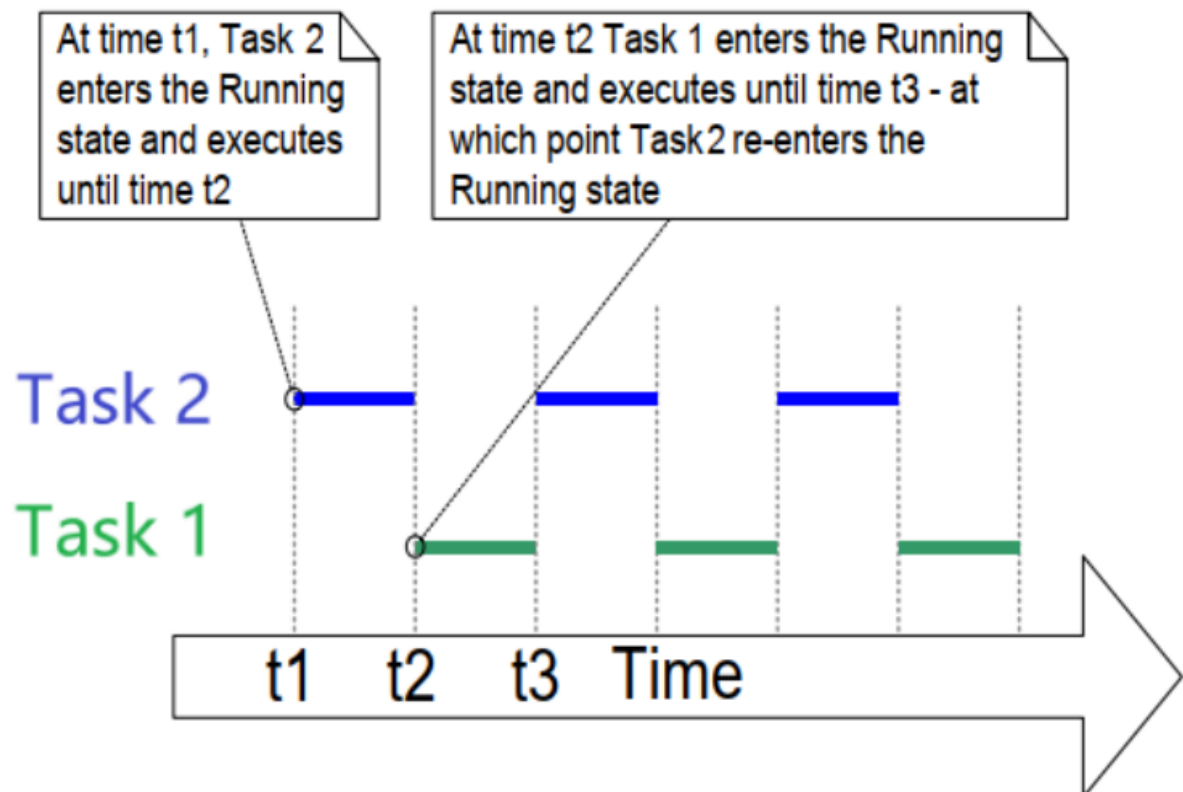
```
T2 run
T1 run
T2 run
T1 run
T2 run
T1 run
T2 run
T1 run
T1 run
```

注意：

- task 2先运行！
- 要分析xTaskCreate的代码才能知道原因：更高优先级的、或者后面创建的任务先运行。

任务运行图：

- 在t1：Task2进入运行态，一直运行直到t2
- 在t2：Task1进入运行态，一直运行直到t3；在t3，Task2重新进入运行态



3.2.4 示例2: 使用任务参数

代码为：FreeRTOS_02_create_task_use_params

我们说过，多个任务可以使用同一个函数，怎么体现它们的差别？

- 栈不同
- 创建任务时可以传入不同的参数

我们创建2个任务，使用同一个函数，代码如下：

```
void vTaskFunction( void *pvParameters )
{
```

```

const char *pcTaskText = pvParameters;
volatile uint32_t u1; /* volatile用来避免被优化掉 */

/* 任务函数的主体一般都是无限循环 */
for( ;; )
{
    /* 打印任务的信息 */
    printf(pcTaskText);

    /* 延迟一会(比较简单粗暴) */
    for( u1 = 0; u1 < mainDELAY_LOOP_COUNT; u1++ )
    {
    }
}
}

```

上述代码中的 `pcTaskText` 来自参数 `pvParameters`，`pvParameters` 来自哪里？创建任务时传入的。

代码如下：

- 使用 `xTaskCreate` 创建2个任务时，第4个参数就是 `pvParameters`
- 不同的任务，`pvParameters` 不一样

```

static const char *pcTextForTask1 = "T1 run\r\n";
static const char *pcTextForTask2 = "T2 run\r\n";

int main( void )
{
    prvSetupHardware();

    xTaskCreate(vTaskFunction, "Task 1", 1000, (void *)pcTextForTask1, 1, NULL);
    xTaskCreate(vTaskFunction, "Task 2", 1000, (void *)pcTextForTask2, 1, NULL);

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

3.2.5 任务的删除

删除任务时使用的函数如下：

```
void vTaskDelete( TaskHandle_t xTaskToDelete );
```

参数说明：

参数	描述
<code>pvTaskCode</code>	任务句柄，使用 <code>xTaskCreate</code> 创建任务时可以得到一个句柄。也可传入 <code>NULL</code> ，这表示删除自己。

怎么删除任务？举个不好的例子：

- 自杀： `vTaskDelete(NULL)`
- 被杀：别的任务执行 `vTaskDelete(pvTaskCode)`，`pvTaskCode`是自己的句柄
- 杀人：执行 `vTaskDelete(pvTaskCode)`，`pvTaskCode`是别的任务的句柄

3.2.6 示例3: 删除任务

代码为： `FreeRTOS_03_delete_task`

本节代码会涉及优先级的知识，可以只看`vTaskDelete`的用法，忽略优先级的讲解。

我们要做这些事情：

- 创建任务1：任务1的大循环里，创建任务2，然后休眠一段时间
- 任务2：打印一句话，然后就删除自己

任务1的代码如下：

```
void vTask1( void *pvParameters )
{
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100UL );
    BaseType_t ret;

    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        /* 打印任务的信息 */
        printf("Task1 is running\r\n");

        ret = xTaskCreate( vTask2, "Task 2", 1000, NULL, 2, &xTask2Handle );
        if (ret != pdPASS)
            printf("Create Task2 Failed\r\n");

        // 如果不休眠的话，Idle任务无法得到执行
        // Idle任务会清理任务2使用的内存
        // 如果不休眠则Idle任务无法执行，最后内存耗尽
        vTaskDelay( xDelay100ms );
    }
}
```

任务2的代码如下：

```
void vTask2( void *pvParameters )
{
    /* 打印任务的信息 */
    printf("Task2 is running and about to delete itself\r\n");

    // 可以直接传入参数NULL，这里只是为了演示函数用法
    vTaskDelete(xTask2Handle);
}
```

main函数代码如下：

```

int main( void )
{
    prvSetupHardware();

    xTaskCreate(vTask1, "Task 1", 1000, NULL, 1, NULL);

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

运行结果如下：

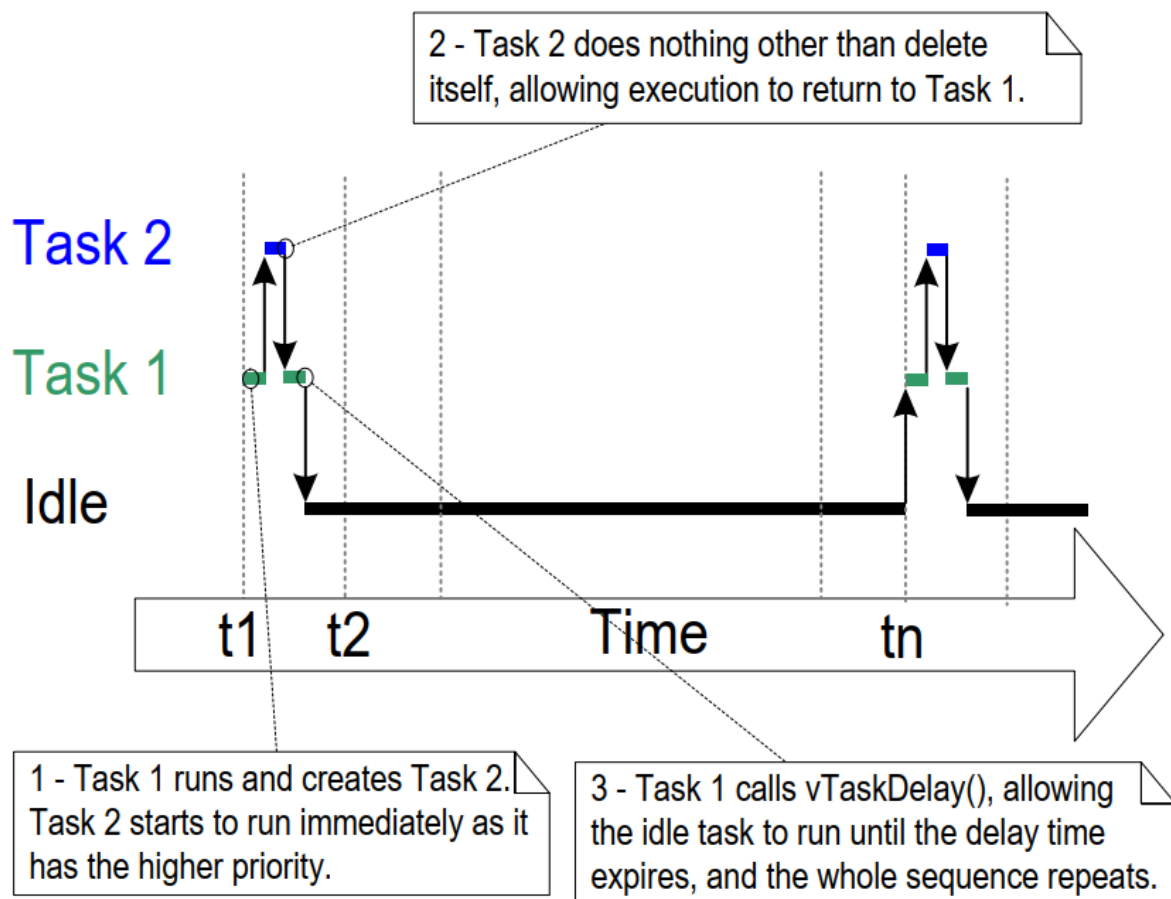
```

Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself

```

任务运行图：

- main函数中创建任务1，优先级为1。任务1运行时，它创建任务2，任务2的优先级是2。
- 任务2的优先级最高，它马上执行。
- 任务2打印一句话后，就删除了自己。
- 任务2被删除后，任务1的优先级最高，轮到任务1继续运行，它调用 `vTaskDelay()` 进入Block状态
- 任务1 Block期间，轮到Idle任务执行：它释放任务2的内存(TCB、栈)
- 时间到后，任务1变为最高优先级的任务继续执行。
- 如此循环。



在任务1的函数中，如果不调用vTaskDelay，则Idle任务用于没有机会执行，它就无法释放创建任务2是分配的内存。

而任务1在不断地创建任务，不断地消耗内存，最终内存耗尽再也无法创建新的任务。

现象如下：

```
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Create Task2 Failed
Task1 is running
Create Task2 Failed
Task1 is running
Create Task2 Failed
Task1 is running
Create Task2 Failed
Task1 is running
```

任务1的代码中，需要注意的是：xTaskCreate的返回值。

- 很多手册里说它失败时返回值是pdFAIL，这个宏是0
- 其实失败时返回值是errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY，这个宏是-1
- 为了避免混淆，我们使用返回值跟pdPASS来比较，这个宏是1

3.3 任务优先级和Tick

3.3.1 任务优先级

在上个示例中我们体验过优先级的使用：高优先级的任务先运行。

优先级的取值范围是：0~(configMAX_PRIORITIES - 1)，数值越大优先级越高。

FreeRTOS的调度器可以使用2种方法来快速找出优先级最高的、可以运行的任务。使用不同的方法时，configMAX_PRIORITIES 的取值有所不同。

- 通用方法
使用C函数实现，对所有的架构都是同样的代码。对configMAX_PRIORITIES的取值没有限制。但是configMAX_PRIORITIES的取值还是尽量小，因为取值越大越浪费内存，也浪费时间。
configUSE_PORT_OPTIMISED_TASK_SELECTION被定义为0、或者未定义时，使用此方法。
- 架构相关的优化的方法
架构相关的汇编指令，可以从一个32位的数里快速地找出为1的最高位。使用这些指令，可以快速找出优先级最高的、可以运行的任务。
使用这种方法时，configMAX_PRIORITIES的取值不能超过32。
configUSE_PORT_OPTIMISED_TASK_SELECTION被定义为1时，使用此方法。

在学习调度方法之前，你只要初略地知道：

- FreeRTOS会确保最高优先级的、可运行的任务，马上就能执行
- 对于相同优先级的、可运行的任务，轮流执行

这无需记忆，就像我们举的例子：

- 厨房着火了，当然优先灭火
- 喂饭、回复信息同样重要，轮流做

3.3.2 Tick

对于同优先级的任务，它们“轮流”执行。怎么轮流？你执行一会，我执行一会。

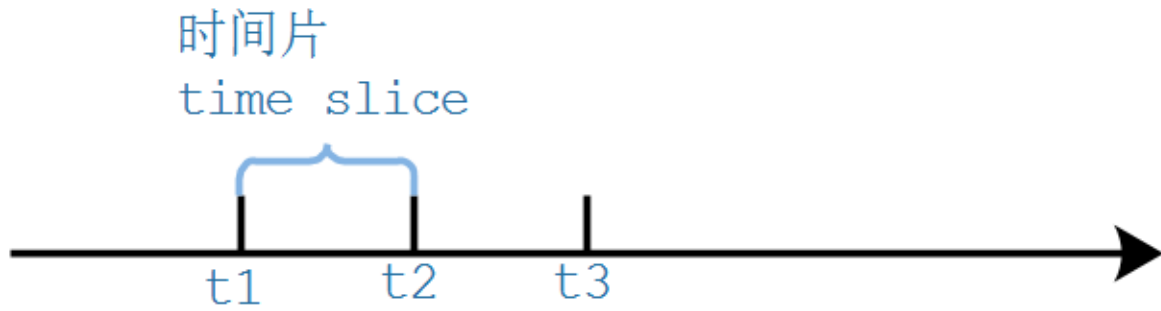
"一会"怎么定义？

人有心跳，心跳间隔基本恒定。

FreeRTOS中也有心跳，它使用定时器产生固定间隔的中断。这叫Tick、滴答，比如每10ms发生一次时钟中断。

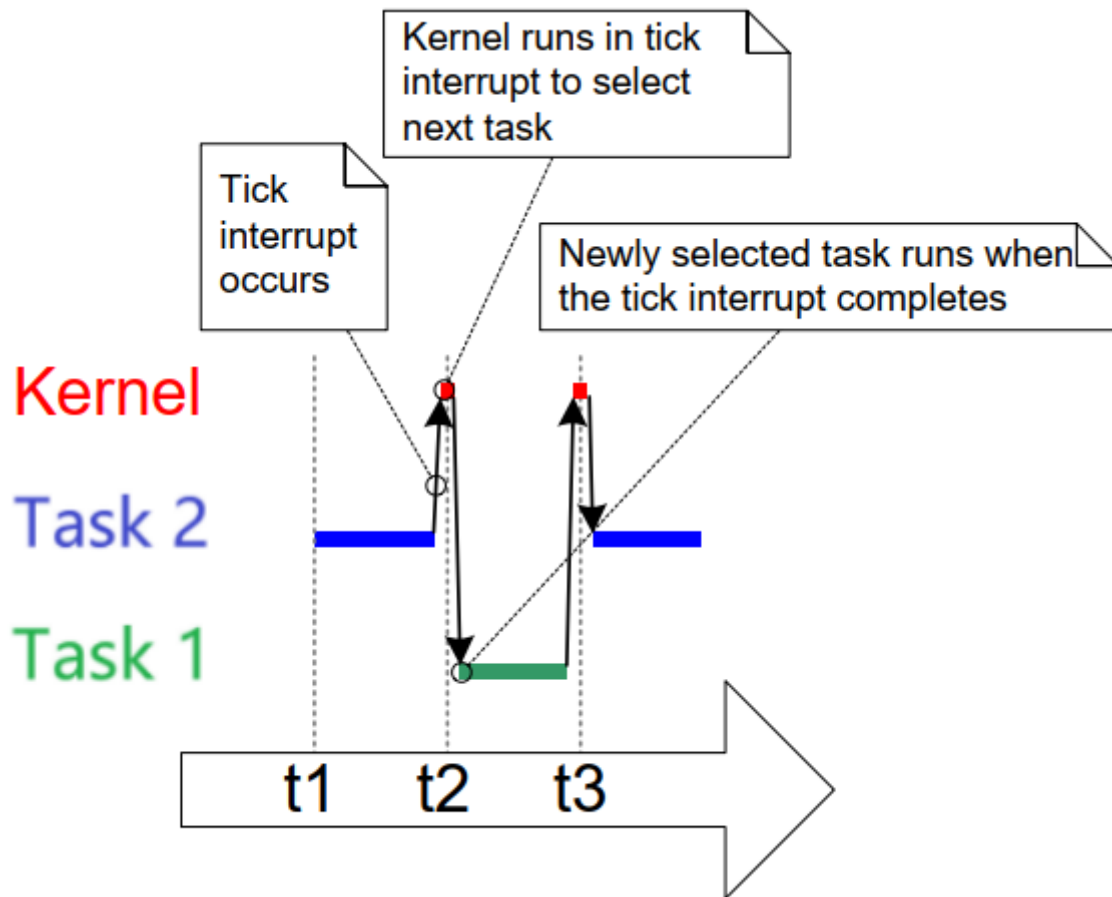
如下图：

- 假设t1、t2、t3发生时钟中断
- 两次中断之间的时间被称为时间片(time slice、tick period)
- 时间片的长度由configTICK_RATE_HZ 决定，假设configTICK_RATE_HZ为100，那么时间片长度就是10ms



相同优先级的任务怎么切换呢？请看下图：

- 任务2从t1执行到t2
- 在t2发生tick中断，进入tick中断处理函数：
 - 选择下一个要运行的任务
 - 执行完中断处理函数后，切换到新的任务：任务1
- 任务1从t2执行到t3
- 从图中可以看出，任务运行的时间并不是严格从t1,t2,t3哪里开始



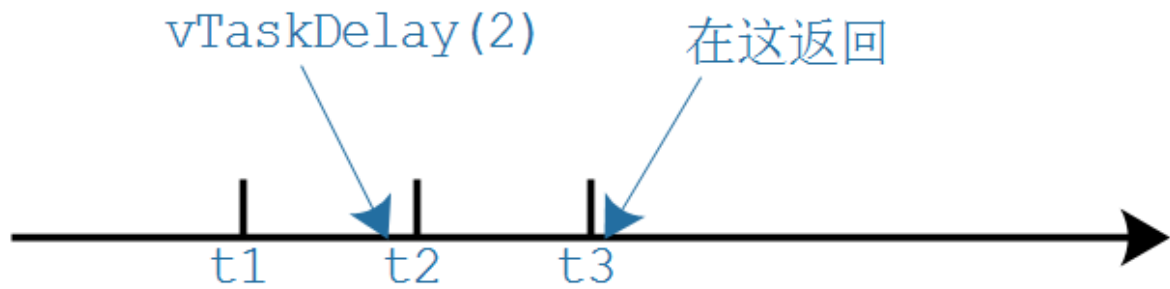
有了Tick的概念后，我们就可以使用Tick来衡量时间了，比如：

```
vTaskDelay(2); // 等待2个Tick，假设configTICK_RATE_HZ=100，Tick周期为10ms，等待20ms

// 还可以使用pdMS_TO_TICKS宏把ms转换为tick
vTaskDelay(pdMS_TO_TICKS(100)); // 等待100ms
```

注意，基于Tick实现的延时并不精确，比如 `vTaskDelay(2)` 的本意是延迟2个Tick周期，有可能经过1个Tick多一点就返回了。

如下图：



使用 `vTaskDelay` 函数时，建议以ms为单位，使用 `pdMS_TO_TICKS` 把时间转换为Tick。

这样的代码就与 `configTICK_RATE_HZ` 无关，即使配置项 `configTICK_RATE_HZ` 改变了，我们也不用去修改代码。

3.3.3 示例4: 优先级实验

代码为： `FreeRTOS_04_task_priority`

本程序会创建3个任务：

- 任务1、任务2：优先级相同，都是1
- 任务3：优先级最高，是2

任务1、2代码如下：

```
void vTask1( void *pvParameters )
{
    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        /* 打印任务的信息 */
        printf("T1\r\n");
    }
}

void vTask2( void *pvParameters )
{
    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        /* 打印任务的信息 */
        printf("T2\r\n");
    }
}
```

任务3代码如下：

```

void vTask3( void *pvParameters )
{
    const TickType_t xDelay3000ms = pdMS_TO_TICKS( 3000UL );

    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        /* 打印任务的信息 */
        printf("T3\r\n");

        // 如果不休眠的话，其他任务无法得到执行
        vTaskDelay( xDelay3000ms );
    }
}

```

main函数代码如下：

```

{
    prvSetupHardware();

    xTaskCreate(vTask1, "Task 1", 1000, NULL, 1, NULL);
    xTaskCreate(vTask2, "Task 2", 1000, NULL, 1, NULL);
    xTaskCreate(vTask3, "Task 3", 1000, NULL, 2, NULL);

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

运行情况如下图所示：

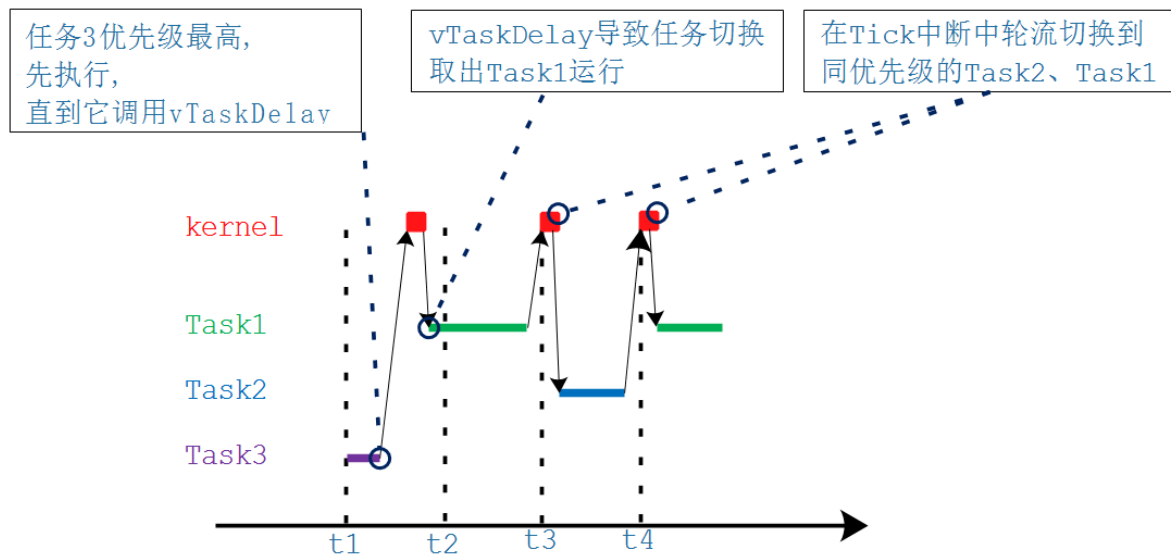
- 任务3优先执行，直到它调用vTaskDelay主动放弃运行
- 任务1、任务2：轮流执行

```

T3
T1
T1
T2
T2
T1
T1
T1

```

调度情况如下图所示：



3.3.4 示例5: 修改优先级

本节代码为：FreeRTOS_05_change_priority。

使用uxTaskPriorityGet来获得任务的优先级：

```
UBaseType_t uxTaskPriorityGet( const TaskHandle_t xTask );
```

使用参数xTask来指定任务，设置为NULL表示获取自己的优先级。

使用vTaskPrioritySet 来设置任务的优先级：

```
void vTaskPrioritySet( TaskHandle_t xTask,
                      UBaseType_t uxNewPriority );
```

使用参数xTask来指定任务，设置为NULL表示设置自己的优先级；

参数uxNewPriority表示新的优先级，取值范围是0~(configMAX_PRIORITIES - 1)。

main函数的代码如下，它创建了2个任务：任务1的优先级更高，它先执行：

```
int main( void )
{
    prvSetupHardware();

    /* Task1的优先级更高，Task1先执行 */
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, &xTask2Handle );

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

任务1的代码如下：

```
void vTask1( void *pvParameters )
{
    UBaseType_t uxPriority;

    /* Task1,Task2都不会进入阻塞或者暂停状态
     * 根据优先级决定谁能运行
     */

    /* 得到Task1自己的优先级 */
    uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        printf( "Task 1 is running\r\n" );

        printf("About to raise the Task 2 priority\r\n" );

        /* 提升Task2的优先级高于Task1
         * Task2会即刻执行
         */
        vTaskPrioritySet( xTask2Handle, ( uxPriority + 1 ) );

        /* 如果Task1能运行到这里，表示它的优先级比Task2高
         * 那就表示Task2肯定把自己的优先级降低了
         */
    }
}
```

任务2的代码如下：

```
void vTask2( void *pvParameters )
{
    UBaseType_t uxPriority;

    /* Task1,Task2都不会进入阻塞或者暂停状态
     * 根据优先级决定谁能运行
     */

    /* 得到Task2自己的优先级 */
    uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        /* 能运行到这里表示Task2的优先级高于Task1
         * Task1提高了Task2的优先级
         */
        printf( "Task 2 is running\r\n" );

        printf( "About to lower the Task 2 priority\r\n" );

        /* 降低Task2自己的优先级，让它小于Task1
         * Task1得以运行
         */
    }
}
```

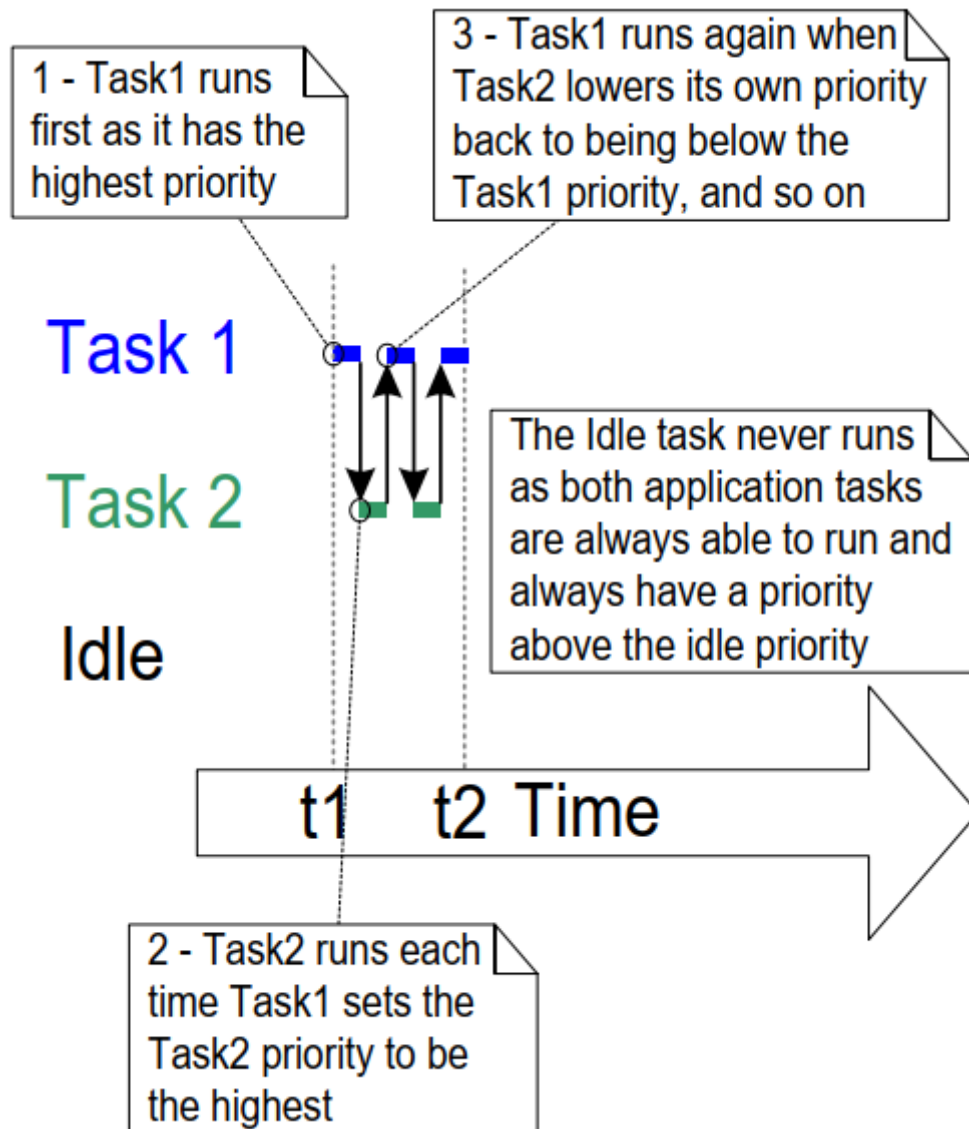
```

vTaskPrioritySet( NULL, ( uxPriority - 2 ) );
    }
}

```

调度情况如下图所示：

- 1：一开始Task1优先级最高，它先执行。它提升了Task2的优先级。
- 2：Task2的优先级最高，它执行。它把自己的优先级降低了。
- 3：Task1的优先级最高，再次执行。它提升了Task2的优先级。
- 如此循环。
- 注意：Task1的优先级一直是2，Task2的优先级是3或1，都大于0。所以Idle任务没有机会执行。



3.4 任务状态

以前我们很简单地把任务的状态分为2中：运行(Runing)、非运行(Not Running)。

对于非运行的状态，还可以继续细分，比如前面的 `FreeRTOS_04_task_priority` 中：

- Task3执行vTaskDelay后：处于非运行状态，要过3秒种才能再次运行
- Task3运行期间，Task1、Task2也处于非运行状态，但是它们**随时可以运行**
- 这两种"非运行"状态就不一样，可以细分为：
 - 阻塞状态(Blocked)
 - 暂停状态(Suspended)
 - 就绪状态(Ready)

3.4.1 阻塞状态(Blocked)

在日常生活的例子中，母亲在电脑前跟同事沟通时，如果同事一直没回复，那么母亲的工作就被卡住了、被堵住了、处于阻塞状态(Blocked)。重点在于：母亲在**等待**。

在 `FreeRTOS_04_task_priority` 实验中，如果把任务3中的vTaskDelay调用注释掉，那么任务1、任务2根本没有执行的机会，任务1、任务2被"饿死"了(starve)。

在实际产品中，我们不会让一个任务一直运行，而是使用"事件驱动"的方法让它运行：

- 任务要等待某个事件，事件发生后它才能运行
- 在等待事件过程中，它不消耗CPU资源
- 在等待事件的过程中，这个任务就处于阻塞状态(Blocked)

在阻塞状态的任务，它可以等待两种类型的事件：

- 时间相关的事件
 - 可以等待一段时间：我等2分钟
 - 也可以一直等待，直到某个绝对时间：我等到下午3点
- 同步事件：这事件由别的任务，或者是中断程序产生
 - 例子1：任务A等待任务B给它发送数据
 - 例子2：任务A等待用户按下按键
 - 同步事件的来源有很多(这些概念在后面会细讲)：
 - 队列(queue)
 - 二进制信号量(binary semaphores)
 - 计数信号量(counting semaphores)
 - 互斥量(mutexes)
 - 递归互斥量、递归锁(recursive mutexes)
 - 事件组(event groups)
 - 任务通知(task notifications)

在等待一个同步事件时，可以加上超时时间。比如等待队里数据，超时时间设为10ms：

- 10ms之内有数据到来：成功返回
- 10ms到了，还是没有数据：超时返回

3.4.2 暂停状态(Suspended)

在日常生活的例子中，母亲正在电脑前跟同事沟通，母亲可以暂停：

- 好烦啊，我暂停一会
- 领导说：你暂停一下

FreeRTOS中的任务也可以进入暂停状态，唯一的方法是通过vTaskSuspend函数。函数原型如下：

```
void vTaskSuspend( TaskHandle_t xTaskToSuspend );
```

参数xTaskToSuspend表示要暂停的任务，如果为NULL，表示暂停自己。

要退出暂停状态，只能由**别人**来操作：

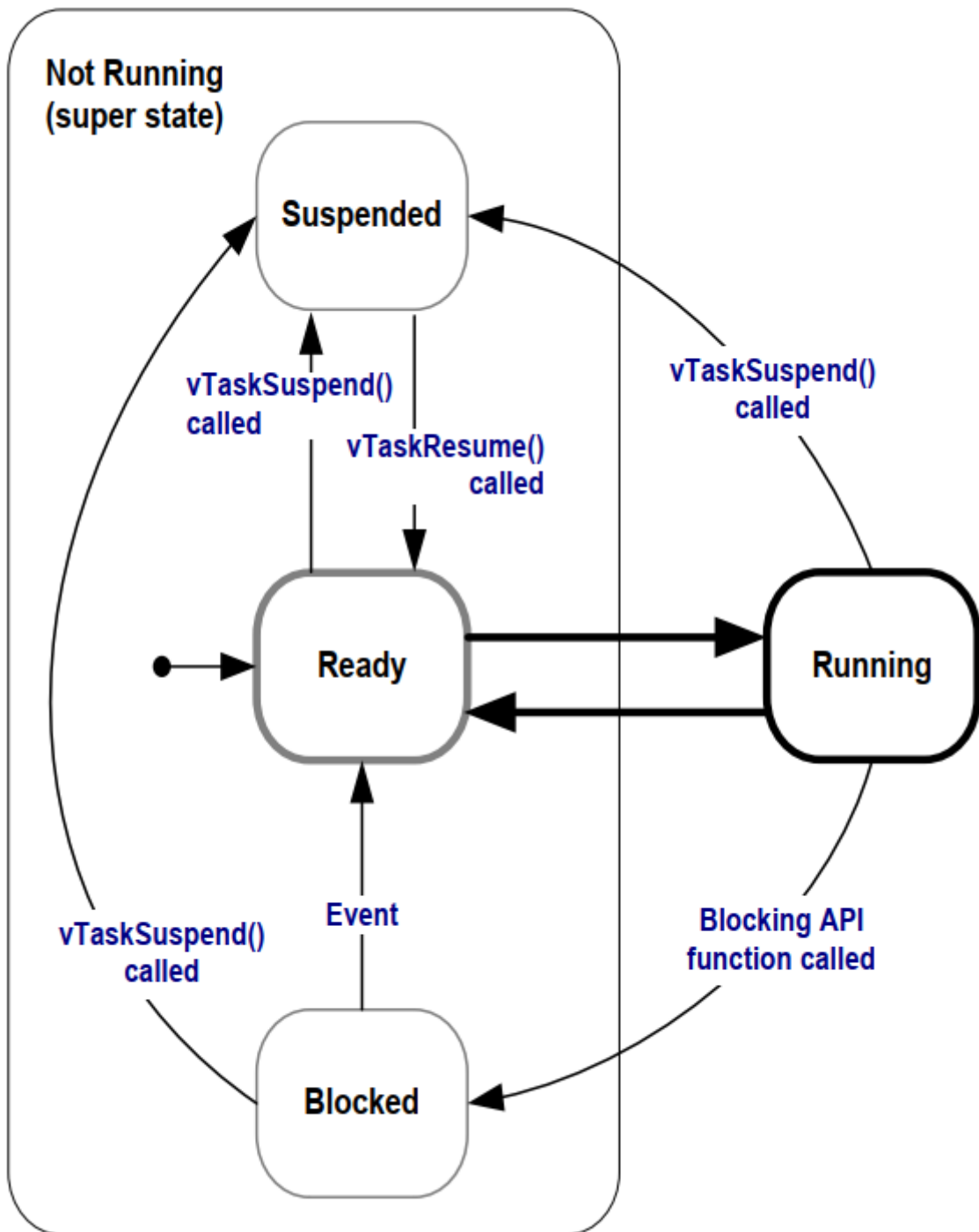
- 别的任务调用：vTaskResume
- 中断程序调用：xTaskResumeFromISR

实际开发中，暂停状态用得不多。

3.4.3 就绪状态(Ready)

这个任务完全准备好了，随时可以运行：只是还轮不到它。这时，它就处于就绪态(Ready)。

3.4.4 完整的状态转换图



3.5 Delay函数

3.5.1 两个Delay函数

有两个Delay函数：

- vTaskDelay：至少等待指定个数的Tick Interrupt才能变为就绪状态
- vTaskDelayUntil：等待到指定的绝对时刻，才能变为就绪态。

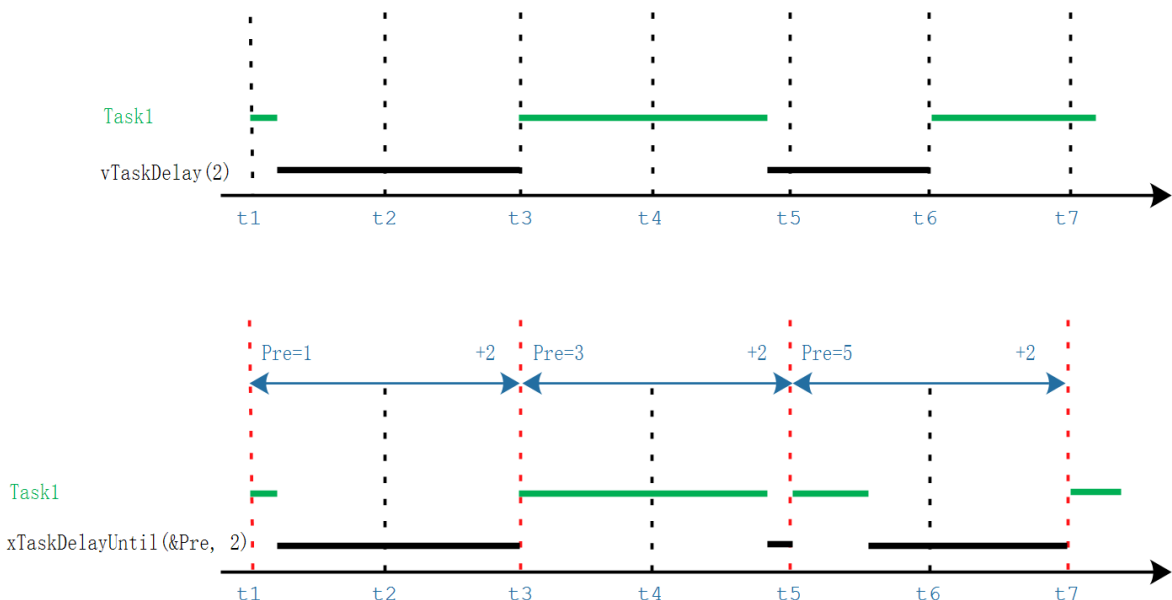
这2个函数原型如下：

```
void vTaskDelay( const TickType_t xTicksToDelay ); /* xTicksToDelay: 等待多少给
Tick */

/* pxPreviousWakeTime: 上一次被唤醒的时间
 * xTimeIncrement: 要阻塞到(pxPreviousWakeTime + xTimeIncrement)
 * 单位都是Tick Count
 */
BaseType_t xTaskDelayUntil( TickType_t * const pxPreviousWakeTime,
                             const TickType_t xTimeIncrement );
```

下面画图说明：

- 使用vTaskDelay(n)时，进入、退出vTaskDelay的时间间隔至少是n个Tick中断
- 使用xTaskDelayUntil(&Pre, n)时，前后两次退出xTaskDelayUntil的时间至少是n个Tick中断
 - 退出xTaskDelayUntil时任务就进入的就绪状态，一般都能得到执行机会
 - 所以可以使用xTaskDelayUntil来让任务周期性地运行



3.5.2 示例6: Delay

本节代码为：`FreeRTOS_06_taskdelay`。

本程序会创建2个任务：

- Task1：
 - 高优先级

- 设置变量flag为1, 然后调用 `vTaskDelay(xDelay50ms);` 或 `vTaskDelayUntil(&xLastWakeTime, xDelay50ms);`
- Task2:
 - 低优先级
 - 设置变量flag为0

main函数代码如下:

```
int main( void )
{
    prvSetupHardware();

    /* Task1的优先级更高, Task1先执行 */
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了, 一般是内存不足 */
    return 0;
}
```

Task1的代码中使用条件开关来选择Delay函数, 把 `#if 1` 改为 `#if 0` 就可以使用 `vTaskDelayUntil`, 代码如下:

```
void vTask1( void *pvParameters )
{
    const TickType_t xDelay50ms = pdMS_TO_TICKS( 50UL );
    TickType_t xLastWakeTime;
    int i;

    /* 获得当前的Tick Count */
    xLastWakeTime = xTaskGetTickCount();

    for( ;; )
    {
        flag = 1;

        /* 故意加入多个循环, 让程序运行时间长一点 */
        for ( i = 0; i < 5; i++)
            printf( "Task 1 is running\r\n" );

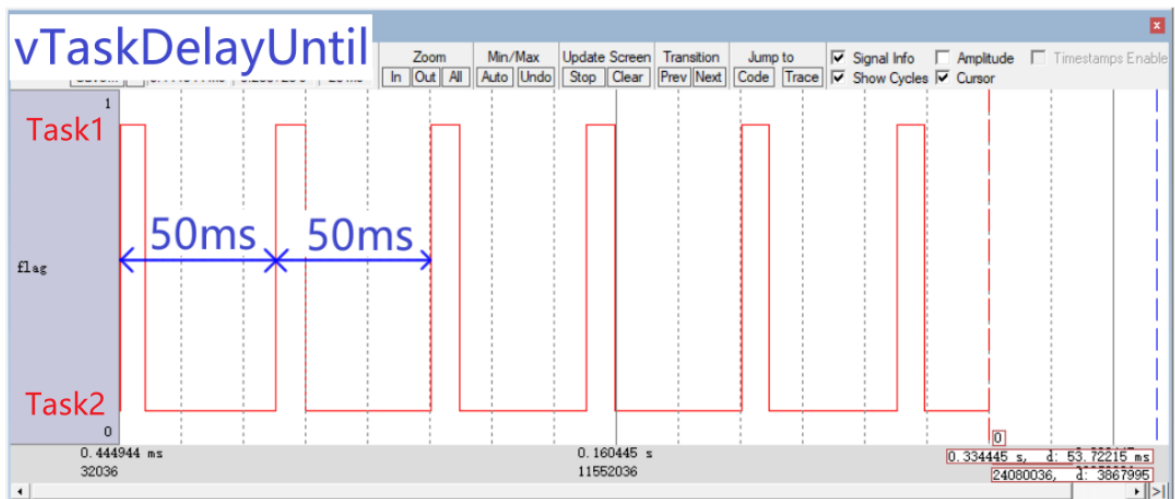
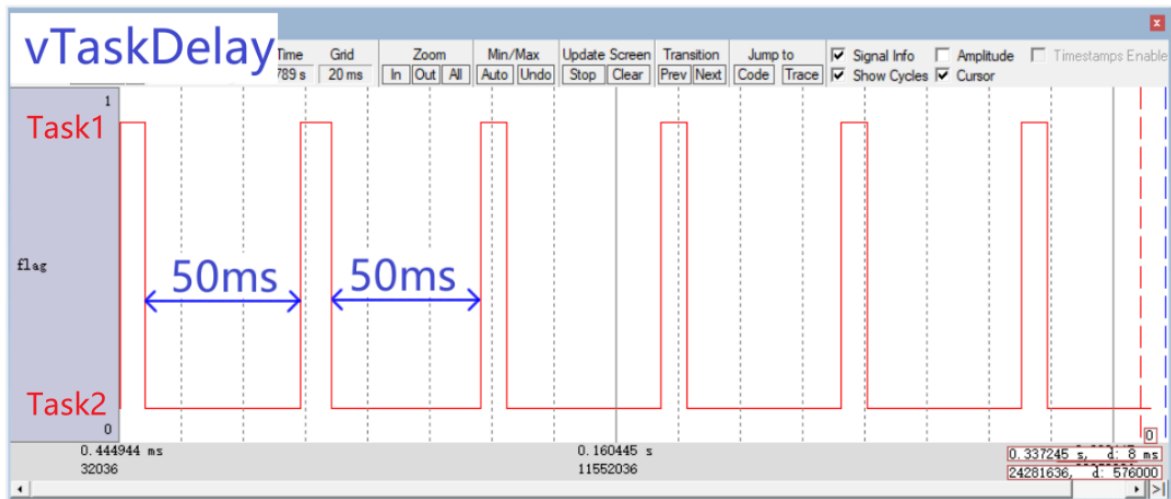
        #if 1
            vTaskDelay(xDelay50ms);
        #else
            vTaskDelayUntil(&xLastWakeTime, xDelay50ms);
        #endif
    }
}
```

Task2的代码如下：

```
void vTask2( void *pvParameters )
{
    for( ;; )
    {
        flag = 0;
        printf( "Task 2 is running\r\n" );
    }
}
```

使用Keil的逻辑分析观察flag变量的bit波形，如下：

- flag为1时表示Task1在运行，flag为0时表示Task2在运行，也就是Task1处于阻塞状态
- vTaskDelay：指定的是阻塞的时间
- vTaskDelayUntil：指定的是任务执行的间隔、周期



3.6 空闲任务及其钩子函数

3.6.1 介绍

在 FreeRTOS_03_delete_task 的实验里，我们体验过空闲任务(Idle任务)的作用：释放被删除的任务的内存。

除了上述目的之外，为什么必须要有空闲任务？一个好的程序，它的任务都是事件驱动的：平时大部分时间处于阻塞状态。有可能我们自己创建的所有任务都无法执行，但是调度器必须能找到一个可以运行的任务：所以，我们要提供空闲任务。在使用 `vTaskStartScheduler()` 函数来创建、启动调度器时，这个函数内部会创建空闲任务：

- 空闲任务优先级为0：它不能阻碍用户任务运行
- 空闲任务要么处于就绪态，要么处于运行态，永远不会阻塞

空闲任务的优先级为0，这以为着一旦某个用户的任务变为就绪态，那么空闲任务马上被切换出去，让这个用户任务运行。在这种情况下，我们说用户任务“抢占”(pre-empt)了空闲任务，这是由调度器实现的。

要注意的是：如果使用 `vTaskDelete()` 来删除任务，那么你就要确保空闲任务有机会执行，否则就无法释放被删除任务的内存。

我们可以添加一个空闲任务的钩子函数(Idle Task Hook Functions)，空闲任务的循环没执行一次，就会调用一次钩子函数。钩子函数的作用有这些：

- 执行一些低优先级的、后台的、需要连续执行的函数
- 测量系统的空闲时间：空闲任务能被执行就意味着所有的高优先级任务都停止了，所以测量空闲任务占据的时间，就可以算出处理器占用率。
- 让系统进入省电模式：空闲任务能被执行就意味着没有重要的事情要做，当然可以进入省电模式了。

空闲任务的钩子函数的限制：

- 不能导致空闲任务进入阻塞状态、暂停状态
- 如果你会使用 `vTaskDelete()` 来删除任务，那么钩子函数要非常高效地执行。如果空闲任务移植卡在钩子函数里的话，它就无法释放内存。

3.6.2 使用钩子函数的前提

在 `FreeRTOS\Source\tasks.c` 中，可以看到如下代码，所以前提就是：

- 把这个宏定义为1: `configUSE_IDLE_HOOK`
- 实现 `vApplicationIdleHook` 函数

```
#if ( configUSE_IDLE_HOOK == 1 )  
{  
    extern void vApplicationIdleHook( void );  
  
    /* Call the user defined function from within the idle task. This  
     * allows the application designer to add background functionality  
     * without the overhead of a separate task.  
     * NOTE: vApplicationIdleHook() MUST NOT, UNDER ANY CIRCUMSTANCES,  
     * CALL A FUNCTION THAT MIGHT BLOCK. */  
    vApplicationIdleHook();  
}  
#endif /* configUSE_IDLE_HOOK */
```

1. 定义这个宏

2. 实现这个函数

3.7 调度算法

3.7.1 重要概念

这些知识在前面都提到过了，这里总结一下。

正在运行的任务，被称为"正在使用处理器"，它处于运行状态。在单处理系统中，任何时间里只能有一个任务处于运行状态。

非运行状态的任务，它处于这3中状态之一：阻塞(Blocked)、暂停(Suspended)、就绪(Ready)。就绪态的任务，可以被调度器挑选出来切换为运行状态，调度器永远都是挑选最高优先级的就绪态任务并让它进入运行状态。

阻塞状态的任务，它在等待"事件"，当事件发生时任务就会进入就绪状态。事件分为两类：时间相关的事件、同步事件。所谓时间相关的事件，就是设置超时时间：在指定时间内阻塞，时间到了就进入就绪状态。使用时间相关的事件，可以实现周期性的功能、可以实现超时功能。同步事件就是：某个任务在等待某些信息，别的任务或者中断服务程序会给它发送信息。怎么"发送信息"？方法很多，有：任务通知(task notification)、队列(queue)、事件组(event group)、信号量(semaphoe)、互斥量(mutex)等。这些方法用来发送同步信息，比如表示某个外设得到了数据。

3.7.2 配置调度算法

所谓调度算法，就是怎么确定哪个就绪态的任务可以切换为运行状态。

通过配置文件FreeRTOSConfig.h的两个配置项来配置调度算法：configUSE_PREEMPTION、configUSE_TIME_SLICING。

还有第三个配置项：configUSE_TICKLESS_IDLE，它是一个高级选项，用于关闭Tick中断来实现省电，后续单独讲解。现在我们假设configUSE_TICKLESS_IDLE被设为0，先不使用这个功能。

调度算法的行为主要体现在两方面：高优先级的任务先运行、同优先级的就绪态任务如何被选中。调度算法要确保同优先级的就绪态任务，能"轮流"运行，策略是"轮转调度"(Round Robin Scheduling)。轮转调度并不保证任务的运行时间是公平分配的，我们还可以细化时间的分配方法。

从3个角度统一理解多种调度算法：

- 可否抢占？高优先级的任务能否优先执行(配置项: configUSE_PREEMPTION)
 - 可以：被称作"可抢占调度"(Pre-emptive)，高优先级的就绪任务马上执行，下面再细化。
 - 不可以：不能抢就只能协商了，被称作"合作调度模式"(Co-operative Scheduling)
 - 当前任务执行时，更高优先级的任务就绪了也不能马上运行，只能等待当前任务主动让出CPU资源。
 - 其他同优先级的任务也只能等待：更高优先级的任务都不能抢占，平级的更应该老实点
- 可抢占的前提下，同优先级的任务是否轮流执行(配置项: configUSE_TIME_SLICING)
 - 轮流执行：被称为"时间片轮转"(Time Slicing)，同优先级的任务轮流执行，你执行一个时间片、我再执行一个时间片
 - 不轮流执行：英文为"without Time Slicing"，当前任务会一直执行，直到主动放弃、或者被高优先级任务抢占
- 在"可抢占"+"时间片轮转"的前提下，进一步细化：空闲任务是否让步于用户任务(配置项: configIDLE_SHOULD_YIELD)
 - 空闲任务低人一等，每执行一次循环，就看看是否主动让位给用户任务
 - 空闲任务跟用户任务一样，大家轮流执行，没有谁更特殊

列表如下：

配置项	A	B	C	D	E
configUSE_PREEMPTION	1	1	1	1	0
configUSE_TIME_SLICING	1	1	0	0	x
configIDLE_SHOULD_YIELD	1	0	1	0	x
说明	常用	很少用	很少用	很少用	几乎不用

注：

- A：可抢占+时间片轮转+空闲任务让步
- B：可抢占+时间片轮转+空闲任务不让步
- C：可抢占+非时间片轮转+空闲任务让步
- D：可抢占+非时间片轮转+空闲任务不让步
- E：合作调度

3.7.3 示例7: 调度

本节代码为：`FreeRTOS_07_scheduler`。后续的实验都是基于这个程序，通过修改配置项来观察效果。

代码里创建了3个任务：Task1、Task2的优先级都是0，跟空闲任务一样，Task3优先级最高为2。程序里定义了4个全局变量，当某个的任务执行时，对应的变量就被设为1，可以通过Keil的逻辑分析仪查看任务切换情况：

```
static volatile int flagIdleTaskrun = 0; // 空闲任务运行时flagIdleTaskrun=1
static volatile int flagTask1run = 0; // 任务1运行时flagTask1run=1
static volatile int flagTask2run = 0; // 任务2运行时flagTask2run=1
static volatile int flagTask3run = 0; // 任务3运行时flagTask3run=1
```

main函数代码如下：

```
int main( void )
{
    prvSetupHardware();

    xTaskCreate(vTask1, "Task 1", 1000, NULL, 0, NULL);
    xTaskCreate(vTask2, "Task 2", 1000, NULL, 0, NULL);
    xTaskCreate(vTask3, "Task 3", 1000, NULL, 2, NULL);

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

任务1、任务2代码如下，它们是"连续任务"(continuous task):

```
void vTask1( void *pvParameters )
{
    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        flagIdleTaskrun = 0;
        flagTask1run = 1;
        flagTask2run = 0;
        flagTask3run = 0;

        /* 打印任务的信息 */
        printf("T1\r\n");
    }
}

void vTask2( void *pvParameters )
{
    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        flagIdleTaskrun = 0;
        flagTask1run = 0;
        flagTask2run = 1;
        flagTask3run = 0;

        /* 打印任务的信息 */
        printf("T2\r\n");
    }
}
```

任务3代码如下，它会调用 `vTaskDelay`，这样别的任务才可以运行：

```
void vTask3( void *pvParameters )
{
    const TickType_t xDelay5ms = pdMS_TO_TICKS( 5UL );

    /* 任务函数的主体一般都是无限循环 */
    for( ;; )
    {
        flagIdleTaskrun = 0;
        flagTask1run = 0;
        flagTask2run = 0;
        flagTask3run = 1;

        /* 打印任务的信息 */
        printf("T3\r\n");

        // 如果不休眠的话，其他任务无法得到执行
        vTaskDelay( xDelay5ms );
    }
}
```

提供了一个空闲任务的钩子函数：

```
void vApplicationIdleHook(void)
{
    flagIdleTaskrun = 1;
    flagTask1run = 0;
    flagTask2run = 0;
    flagTask3run = 0;

    /* 故意加入打印让flagIdleTaskrun变为1的时间维持长一点 */
    printf("Id\r\n");
}
```

3.7.4 对比效果: 抢占与否

在 `FreeRTOSConfig.h` 中，定义这样的宏，对比逻辑分析仪的效果：

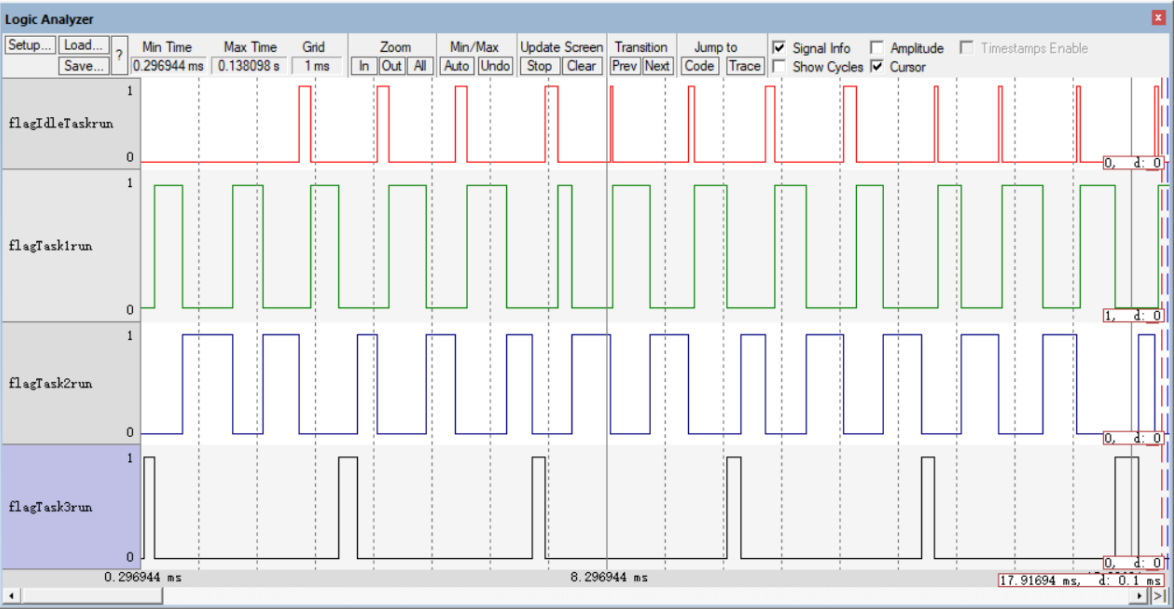
```
// 实验1: 抢占
#define configUSE_PREEMPTION      1
#define configUSE_TIME_SLICING   1
#define configIDLE_SHOULD_YIELD  1

// 实验2: 不抢占
#define configUSE_PREEMPTION      0
#define configUSE_TIME_SLICING   1
#define configIDLE_SHOULD_YIELD  1
```

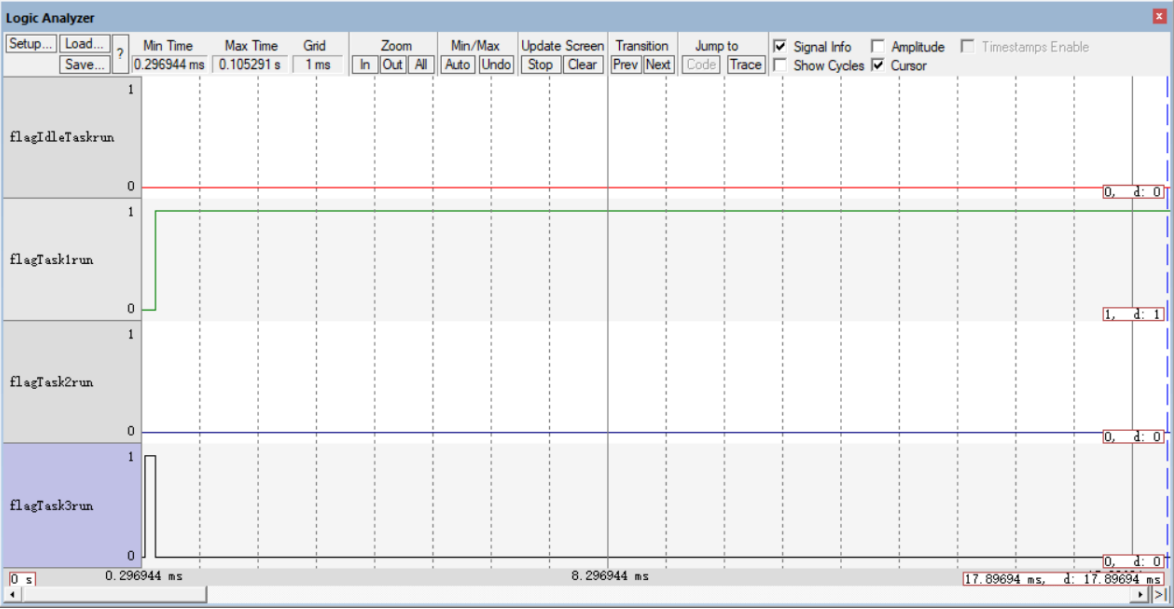
从下面的对比图可以知道：

- 抢占时：高优先级任务就绪时，就可以马上执行
- 不抢占时：优先级失去意义了，既然不能抢占就只能协商了，图中任务1一直在运行(一点都没有协商精神)，其他任务都无法执行。即使任务3的 `vTaskDelay` 已经超时、即使它的优先级更高，都没办法执行。

抢占



不抢占



3.7.5 对比效果: 时间片轮转与否

在 FreeRTOSConfig.h 中, 定义这样的宏, 对比逻辑分析仪的效果:

```
// 实验1: 时间片轮转
#define configUSE_PREEMPTION 1
#define configUSE_TIME_SLICING 1
#define configIDLE_SHOULD_YIELD 1

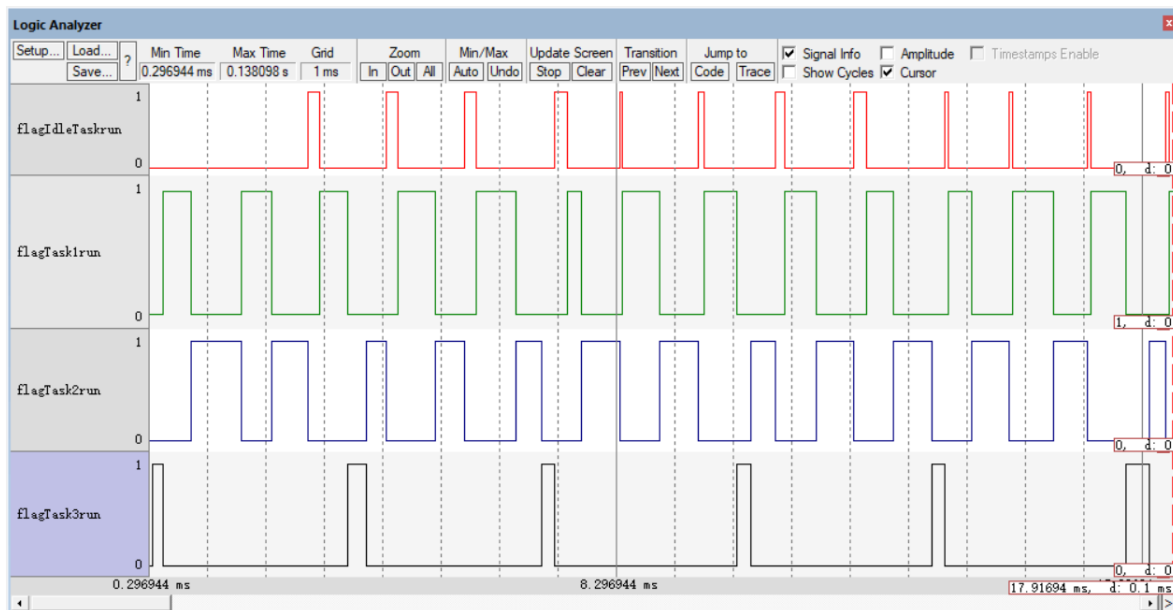
// 实验2: 时间片不轮转
#define configUSE_PREEMPTION 1
#define configUSE_TIME_SLICING 0
#define configIDLE_SHOULD_YIELD 1
```

从下面的对比图可以知道:

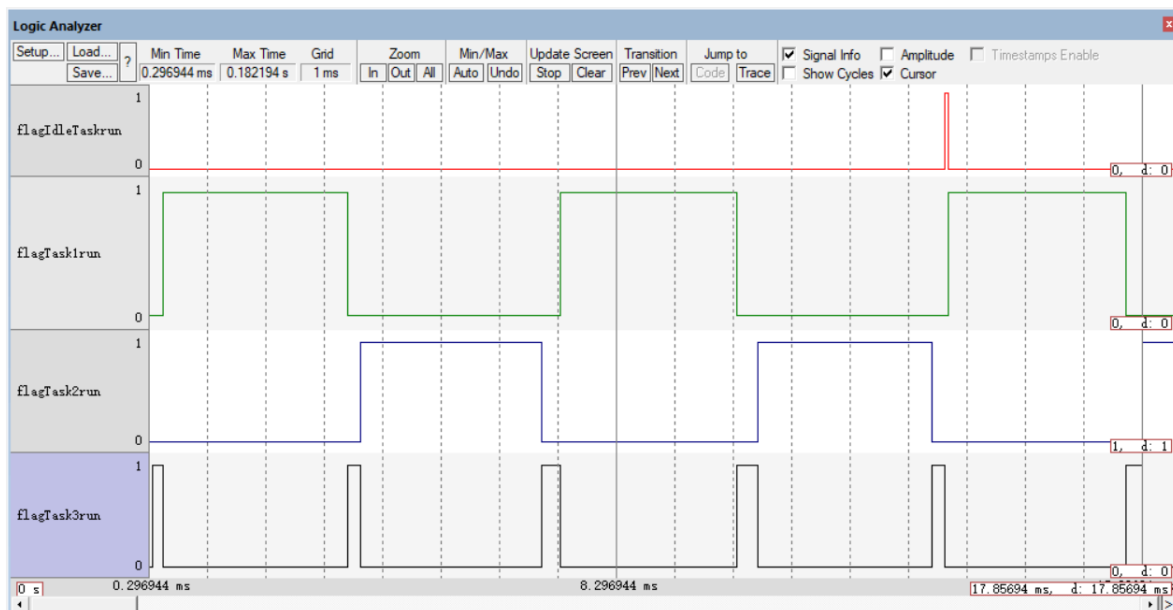
- 时间片轮转: 在Tick中断中会引起任务切换

- 时间片不轮转：高优先级任务就绪时会引起任务切换，高优先级任务不再运行时也会引起任务切换。可以看到任务3就绪后可以马上执行，它运行完毕后导致任务切换。其他时间没有任务切换，可以看到任务1、任务2都运行了很长时间。

时间片轮转



时间片不轮转



3.7.6 对比效果: 空闲任务让步

在 FreeRTOSConfig.h 中，定义这样的宏，对比逻辑分析仪的效果：

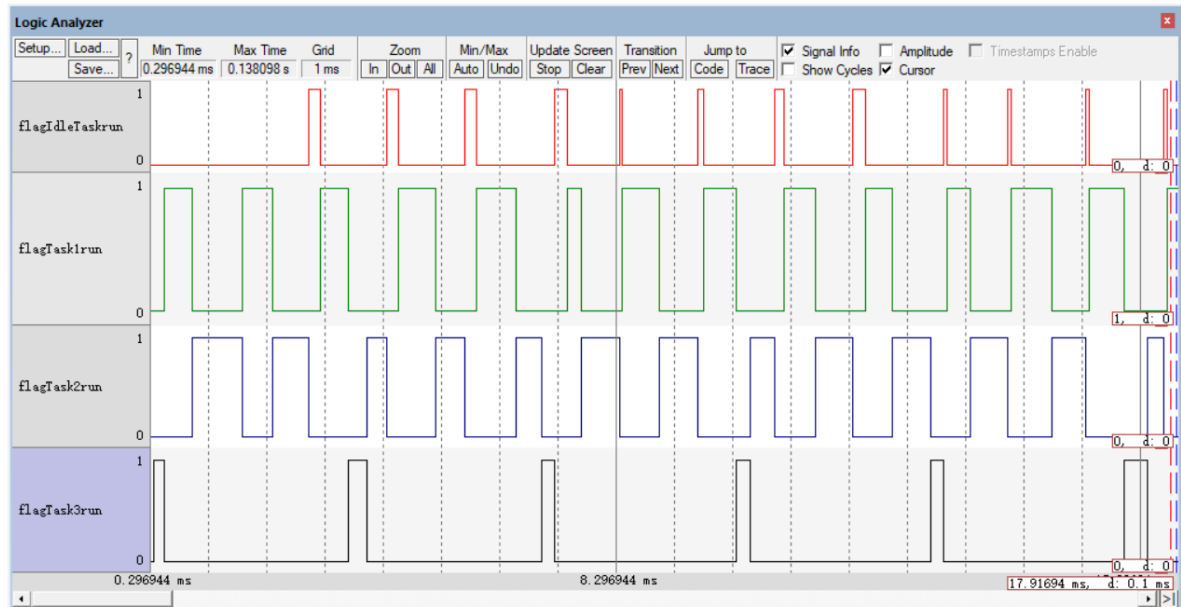
```
// 实验1: 空闲任务让步
#define configUSE_PREEMPTION 1
#define configUSE_TIME_SLICING 1
#define configIDLE_SHOULD_YIELD 1

// 实验2: 空闲任务不让步
#define configUSE_PREEMPTION 1
#define configUSE_TIME_SLICING 1
#define configIDLE_SHOULD_YIELD 0
```

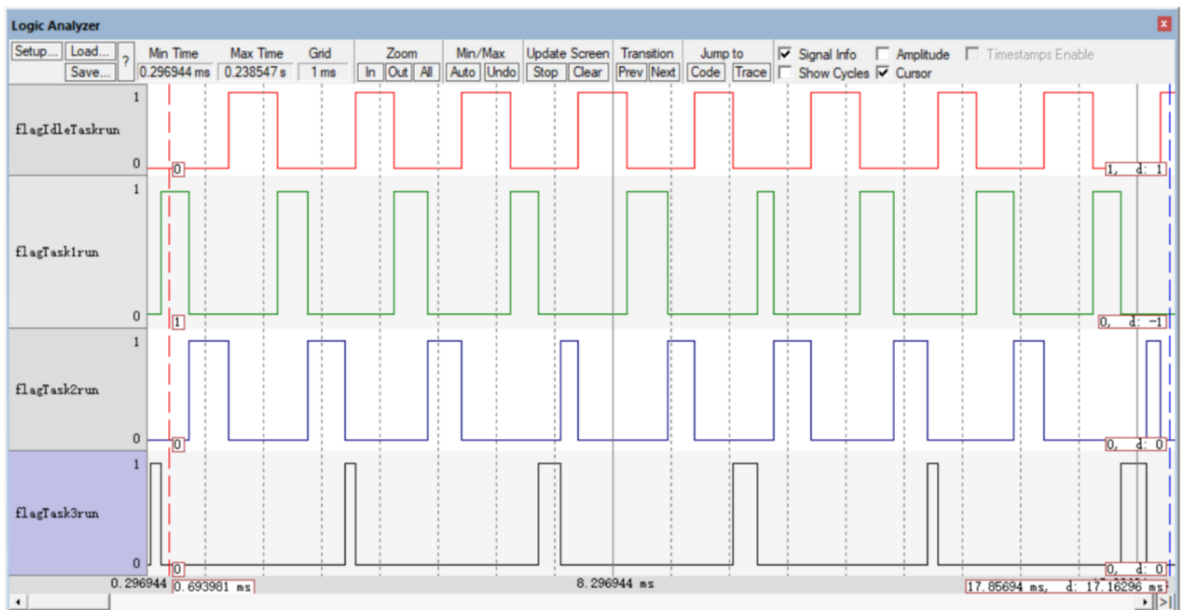
从下面的对比图可以知道：

- 让步时：在空闲任务的每个循环中，会主动让出处理器，从图中可以看到flagIdleTaskrun的波形很小
- 不让步时：空闲任务跟任务1、任务2同等待遇，它们的波形宽度是差不多的

空闲任务让步



空闲任务不让步



第四章 同步互斥与通信

本章是概述性的内容。可以把多任务系统当做一个团队，里面的每一个任务就相当于团队里的一个人。团队成员之间要协调工作进度(同步)、争用会议室(互斥)、沟通(通信)。多任务系统中所涉及的概念，都可以在现实生活中找到例子。

各类RTOS都会涉及这些概念：任务通知(task notification)、队列(queue)、事件组(event group)、信号量(semaphoe)、互斥量(mutex)等。我们先站在更高角度来讲解这些概念。

4.1 同步与互斥的概念

一句话理解同步与互斥：我等你用完厕所，我再用厕所。

什么叫同步？就是：哎哎哎，我正在用厕所，你等会。

什么叫互斥？就是：哎哎哎，我正在用厕所，你不能进来。

同步与互斥经常放在一起讲，是因为它们之间的关系很大，“互斥”操作可以使用“同步”来实现。我“等”你用完厕所，我再用厕所。这不就是用“同步”来实现“互斥”吗？

再举一个例子。在团队活动里，同事A先写完报表，经理B才能拿去向领导汇报。经理B必须等同事A完成报表，AB之间有依赖，B必须放慢脚步，被称为同步。在团队活动中，同事A已经使用会议室了，经理B也想使用，即使经理B是领导，他也得等着，这就叫互斥。经理B跟同事A说：你用完会议室就提醒我。这就是使用“同步”来实现“互斥”。

有时候看代码更容易理解，伪代码如下：

```
01 void 抢厕所(void)
02 {
03     if (有人在用) 我眯一会；
04     用厕所；
05     喂，醒醒，有人要用厕所吗；
06 }
```

假设有A、B两人早起抢厕所，A先行一步占用了；B慢了一步，于是就眯一会；当A用完后叫醒B，B也就愉快地上厕所了。

在这个过程中，A、B是互斥地访问“厕所”，“厕所”被称之为临界资源。我们使用了“休眠-唤醒”的同步机制实现了“临界资源”的“互斥访问”。

同一时间只能有一个人使用的资源，被称为临界资源。比如任务A、B都要使用串口来打印，串口就是临界资源。如果A、B同时使用串口，那么打印出来的信息就是A、B混杂，无法分辨。所以使用串口时，应该是这样：A用完，B再用；B用完，A再用。

4.2 同步与互斥并不简单

4.3 各类方法的对比

能实现同步、互斥的内核方法有：任务通知(task notification)、队列(queue)、事件组(event group)、信号量(semaphore)、互斥量(mutex)。

它们都有类似的操作方法：获取/释放、阻塞/唤醒、超时。比如：

- A获取资源，用完后A释放资源
- A获取不到资源则阻塞，B释放资源并把A唤醒
- A获取不到资源则阻塞，并定个闹钟；A要么超时返回，要么在这段时间内因为B释放资源而被唤醒。

这些内核对象五花八门，记不住怎么办？我也记不住，通过对比的方法来区分它们。

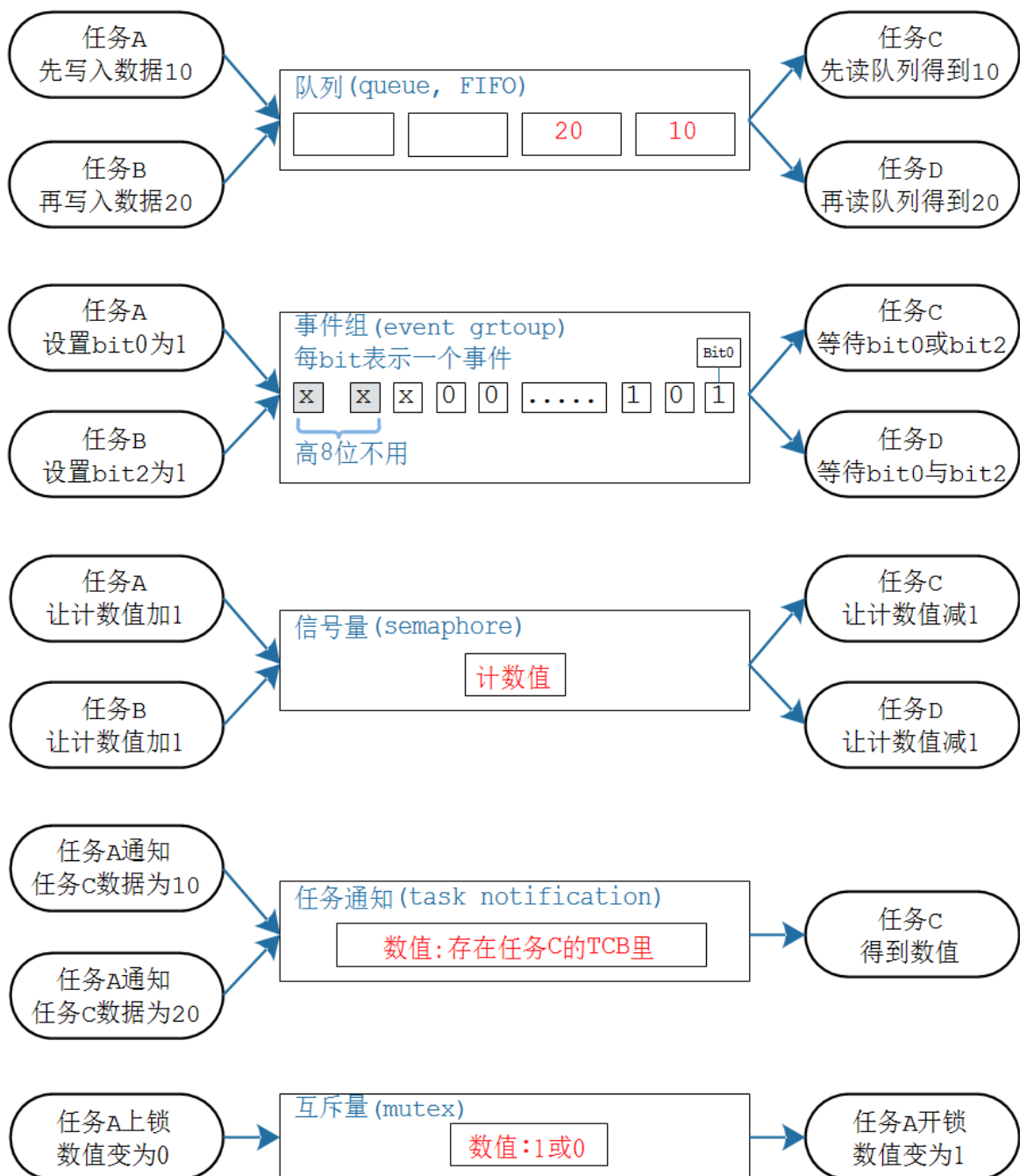
- 能否传信息？只能传递状态？
- 为众生？只为你？
- 我生产，你们消费？
- 我上锁，只能由我开锁

内核对象	生产者	消费者	数据/状态	说明
队列	ALL	ALL	数据：若干个数据 谁都可以往队列里扔数据， 谁都可以从队列里读数据	用来传递数据， 发送者、接收者无限制， 一个数据只能唤醒一个接收者
事件组	ALL	ALL	多个位：或、与 谁都可以设置(生产)多个位， 谁都可以等待某个位、若干个位	用来传递事件， 可以是N个事件， 发送者、接受者无限制， 可以唤醒多个接收者：像广播
信号量	ALL	ALL	数量：0~n 谁都可以增加一个数量， 谁都可消耗一个数量	用来维持资源的个数， 生产者、消费者无限制， 1个资源只能唤醒1个接收者
任务通知	ALL	只有我	数据、状态都可以传输， 使用任务通知时， 必须指定接受者	N对1的关系： 发送者无限制， 接收者只能是这个任务
互斥量	只能A开锁	A上锁	位：0、1 我上锁：1变为0， 只能由我开锁：0变为1	就像一个空厕所， 谁使用谁上锁， 也只能由他开锁

使用图形对比如下：

- 队列：
 - 里面可以放任意数据，可以放多个数据
 - 任务、ISR都可以放入数据；任务、ISR都可以从中读出数据
- 事件组：

- 一个事件用一bit表示, 1表示事件发生了, 0表示事件没发生
 - 可以用来表示事件、事件的组合发生了, 不能传递数据
 - 有广播效果: 事件或事件的组合发生了, 等待它的多个任务都会被唤醒
- 信号量:
 - 核心是"计数值"
 - 任务、ISR释放信号量时让计数值加1
 - 任务、ISR获得信号量时, 让计数值减1
- 任务通知:
 - 核心是任务的TCB里的数值
 - 会被覆盖
 - 发通知给谁? 必须指定接收任务
 - 只能由接收任务本身获取该通知
- 互斥量:
 - 数值只有0或1
 - 谁获得互斥量, 就必须由谁释放同一个互斥量



第五章 队列(queue)

队列(queue)可以用于"任务到任务"、"任务到中断"、"中断到任务"直接传输信息。

本章涉及如下内容：

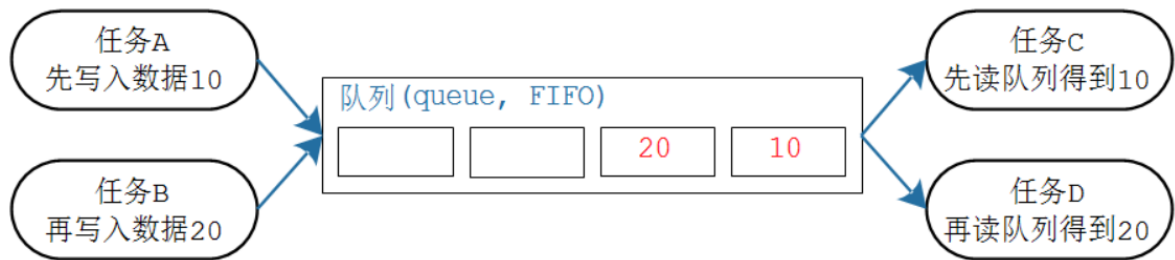
- 怎么创建、清除、删除队列
- 队列中消息如何保存
- 怎么向队列发送数据、怎么从队列读取数据、怎么覆盖队列的数据
- 在队列上阻塞是什么意思
- 怎么在多个队列上阻塞
- 读写队列时如何影响任务的优先级

5.1 队列的特性

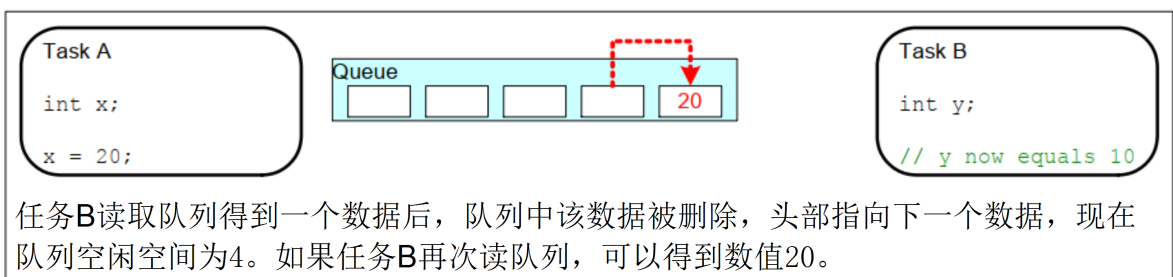
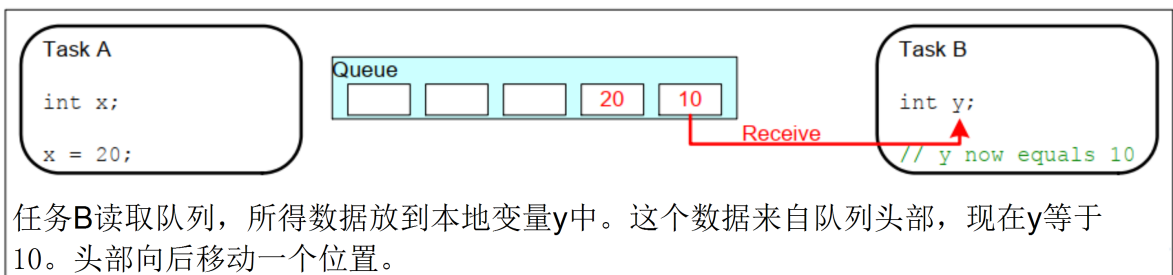
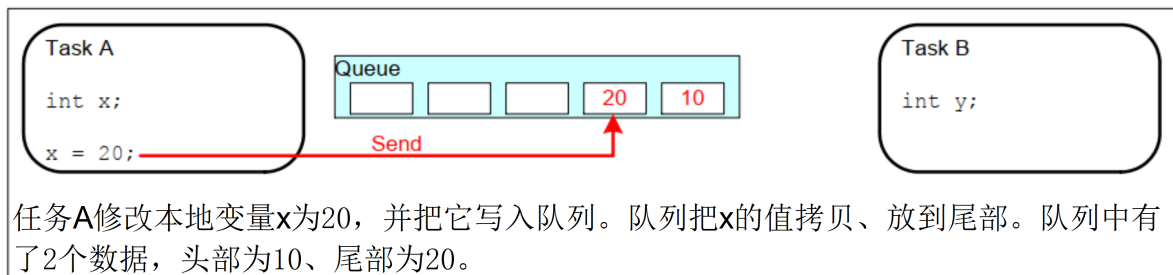
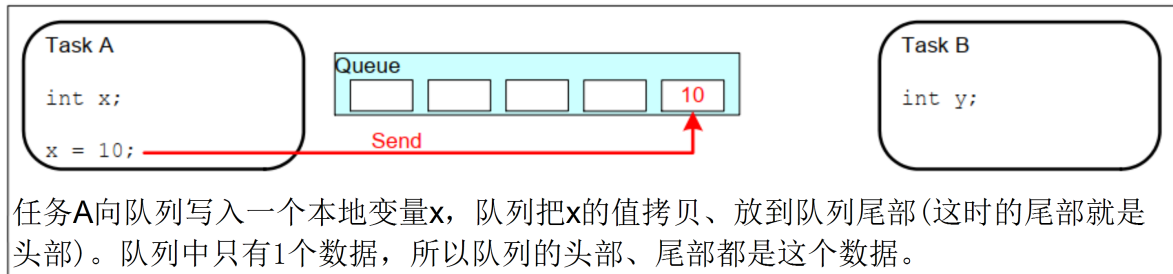
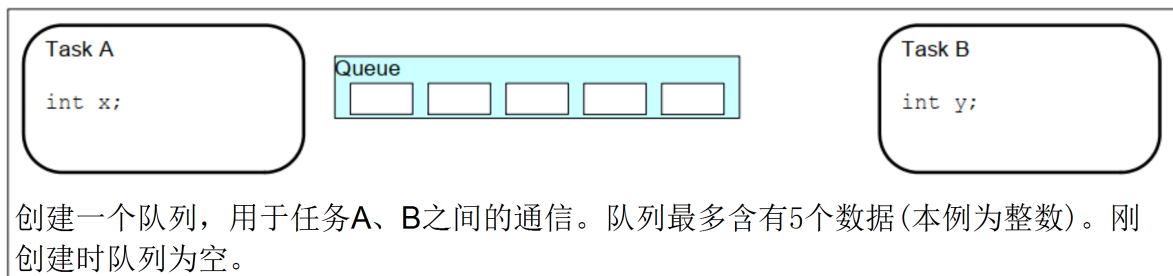
5.1.1 常规操作

队列的简化操作如下图所示，从此图可知：

- 队列可以包含若干个数据：队列中有若干项，这被称为"长度"(length)
- 每个数据大小固定
- 创建队列时就要指定长度、数据大小
- 数据的操作采用先进先出的方法(FIFO, First In First Out)：写数据时放到尾部，读数据时从头部读
- 也可以强制写队列头部：覆盖头部数据



更详细的操作如下图所示：



5.1.2 传输数据的两种方法

使用队列传输数据时有两种方法：

- 拷贝：把数据、把变量的值复制进队列里
- 引用：把数据、把变量的地址复制进队列里

FreeRTOS使用拷贝值的方法，这更简单：

- 局部变量的值可以发送到队列中，后续即使函数退出、局部变量被回收，也不会影响队列中的数据
- 无需分配buffer来保存数据，队列中有buffer
- 局部变量可以马上再次使用
- 发送任务、接收任务解耦：接收任务不需要知道这数据是谁的、也不需要发送任务来释放数据
- 如果数据实在太太，你还是可以使用队列传输它的地址
- 队列的空间有FreeRTOS内核分配，无需任务操心

- 对于有内存保护功能的系统，如果队列使用引用方法，也就是使用地址，必须确保双方任务对这个地址都有访问权限。使用拷贝方法时，则无此限制：内核有足够的权限，把数据复制进队列、再把数据复制出队列。

5.1.3 队列的阻塞访问

只要知道队列的句柄，谁都可以读、写该队列。任务、ISR都可读、写队列。可以多个任务读写队列。

任务读写队列时，简单地说：如果读写不成功，则阻塞；可以指定超时时间。口语化地说，就是可以定个闹钟：如果能读写了就马上进入就绪态，否则就阻塞直到超时。

某个任务读队列时，如果队列没有数据，则该任务可以进入阻塞状态：还可以指定阻塞的时间。如果队列有数据了，则该阻塞的任务会变为就绪态。如果一直都没有数据，则时间到之后它也会进入就绪态。

既然读取队列的任务个数没有限制，那么当多个任务读取空队列时，这些任务都会进入阻塞状态：有多个任务在等待同一个队列的数据。当队列中有数据时，哪个任务会进入就绪态？

- 优先级最高的任务
- 如果大家的优先级相同，那等待时间最久的任务会进入就绪态

跟读队列类似，一个任务要写队列时，如果队列满了，该任务也可以进入阻塞状态：还可以指定阻塞的时间。如果队列有空间了，则该阻塞的任务会变为就绪态。如果一直都没有空间，则时间到之后它也会进入就绪态。

既然写队列的任务个数没有限制，那么当多个任务写“满队列”时，这些任务都会进入阻塞状态：有多个任务在等待同一个队列的空间。当队列中有空间时，哪个任务会进入就绪态？

- 优先级最高的任务
- 如果大家的优先级相同，那等待时间最久的任务会进入就绪态

5.2 队列函数

使用队列的流程：创建队列、写队列、读队列、删除队列。

5.2.1 创建

队列的创建有两种方法：动态分配内存、静态分配内存，

- 动态分配内存：xQueueCreate，队列的内存存在函数内部动态分配

函数原型如下：

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength, UBaseType_t uxItemSize );
```

参数	说明
uxQueueLength	队列长度，最多能存放多少个数据(item)
uxItemSize	每个数据(item)的大小：以字节为单位
返回值	非0：成功，返回句柄，以后使用句柄来操作队列 NULL：失败，因为内存不足

- 静态分配内存：xQueueCreateStatic，队列的内存要事先分配好

函数原型如下：

```
QueueHandle_t xQueueCreateStatic(
    UBaseType_t uxQueueLength,
    UBaseType_t uxItemSize,
    uint8_t *pucQueueStorageBuffer,
    StaticQueue_t *pxQueueBuffer
);
```

参数	说明
uxQueueLength	队列长度，最多能存放多少个数据(item)
uxItemSize	每个数据(item)的大小：以字节为单位
pucQueueStorageBuffer	如果uxItemSize非0，pucQueueStorageBuffer必须指向一个uint8_t数组，此数组大小至少为"uxQueueLength * uxItemSize"
pxQueueBuffer	必须执行一个StaticQueue_t结构体，用来保存队列的数据结构
返回值	非0：成功，返回句柄，以后使用句柄来操作队列 NULL：失败，因为pxQueueBuffer为NULL

示例代码：


```

        TickType_t      xTicksToWait
    );

/*
 * 往队列尾部写入数据，如果没有空间，阻塞时间为xTicksToWait
 */
BaseType_t xQueueSendToBack(
    QueueHandle_t    xQueue,
    const void *pvItemToQueue,
    TickType_t      xTicksToWait
);

/*
 * 往队列尾部写入数据，此函数可以在中断函数中使用，不可阻塞
 */
BaseType_t xQueueSendToBackFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken
);

/*
 * 往队列头部写入数据，如果没有空间，阻塞时间为xTicksToWait
 */
BaseType_t xQueueSendToFront(
    QueueHandle_t    xQueue,
    const void *pvItemToQueue,
    TickType_t      xTicksToWait
);

/*
 * 往队列头部写入数据，此函数可以在中断函数中使用，不可阻塞
 */
BaseType_t xQueueSendToFrontFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken
);

```

这些函数用到的参数是类似的，统一说明如下：

参数	说明
xQueue	队列句柄，要写哪个队列
pvItemToQueue	数据指针，这个数据的值会被复制进队列， 复制多大的数据？在创建队列时已经指定了数据大小
xTicksToWait	如果队列满则无法写入新数据，可以让任务进入阻塞状态， xTicksToWait表示阻塞的最大时间(Tick Count)。 如果被设为0，无法写入数据时函数会立刻返回； 如果被设为portMAX_DELAY，则会一直阻塞直到有空间可写
返回值	pdPASS：数据成功写入了队列 errQUEUE_FULL：写入失败，因为队列满了。

5.2.5 读队列

使用 `xQueueReceive()` 函数读队列，读到一个数据后，队列中该数据会被移除。这个函数有两个版本：在任务中使用、在ISR中使用。函数原型如下：

```
 BaseType_t xQueueReceive( QueueHandle_t xQueue,
                          void * const pvBuffer,
                          TickType_t xTicksToWait );

 BaseType_t xQueueReceiveFromISR(
                                QueueHandle_t xQueue,
                                void * const pvBuffer,
                                BaseType_t *pxTaskWoken
                                );
```

参数说明如下：

参数	说明
xQueue	队列句柄，要读哪个队列
pvBuffer	buffer指针，队列的数据会被复制到这个buffer 复制多大的数据？在创建队列时已经指定了数据大小
xTicksToWait	果队列空则无法读出数据，可以让任务进入阻塞状态， xTicksToWait表示阻塞的最大时间(Tick Count)。 如果被设为0，无法读出数据时函数会立刻返回； 如果被设为portMAX_DELAY，则会一直阻塞直到有数据可写
返回值	pdPASS：从队列读出数据入 errQUEUE_EMPTY：读取失败，因为队列空了。

5.2.6 查询

可以查询队列中有多少个数据、有多少空余空间。函数原型如下：

```

/*
 * 返回队列中可用数据的个数
 */
UBaseType_t uxQueueMessagesWaiting( const QueueHandle_t xQueue );

/*
 * 返回队列中可用空间的个数
 */
UBaseType_t uxQueueSpacesAvailable( const QueueHandle_t xQueue );

```

5.2.7 覆盖/偷看

当队列长度为1时，可以使用 `xQueueOverwrite()` 或 `xQueueOverwriteFromISR()` 来覆盖数据。注意，队列长度必须为1。当队列满时，这些函数会覆盖里面的数据，这也意味着这些函数不会被阻塞。

函数原型如下：

```

/* 覆盖队列
 * xQueue: 写哪个队列
 * pvItemToQueue: 数据地址
 * 返回值: pdTRUE表示成功，pdFALSE表示失败
 */
BaseType_t xQueueOverwrite(
    QueueHandle_t xQueue,
    const void * pvItemToQueue
);

BaseType_t xQueueOverwriteFromISR(
    QueueHandle_t xQueue,
    const void * pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken
);

```

如果想让队列中的数据供多方读取，也就是说读取时不要移除数据，要留给后来人。那么可以使用"窥视"，也就是 `xQueuePeek()` 或 `xQueuePeekFromISR()`。这些函数会从队列中复制出数据，但是不移除数据。这也意味着，如果队列中没有数据，那么"偷看"时会导致阻塞；一旦队列中有数据，以后每次"偷看"都会成功。

函数原型如下：

```

/* 偷看队列
 * xQueue: 偷看哪个队列
 * pvItemToQueue: 数据地址，用来保存复制出来的数据
 * xTicksToWait: 没有数据的话阻塞一会
 * 返回值: pdTRUE表示成功，pdFALSE表示失败
 */
BaseType_t xQueuePeek(
    QueueHandle_t xQueue,
    void * const pvBuffer,
    TickType_t xTicksToWait
);

```

```
BaseType_t xQueuePeekFromISR(  
    QueueHandle_t xQueue,  
    void *pvBuffer,  
);
```

5.3 示例8: 队列的基本使用

本节代码为：FreeRTOS_08_queue。

本程序会创建一个队列，然后创建2个发送任务、1个接收任务：

- 发送任务优先级为1，分别往队列中写入100、200
- 接收任务优先级为2，读队列、打印数值

main函数中创建的队列、创建了发送任务、接收任务，代码如下：

```
/* 队列句柄，创建队列时会设置这个变量 */
QueueHandle_t xQueue;

int main( void )
{
    prvSetupHardware();

    /* 创建队列：长度为5，数据大小为4字节(存放一个整数) */
    xQueue = xQueueCreate( 5, sizeof( int32_t ) );

    if( xQueue != NULL )
    {
        /* 创建2个任务用于写队列，传入的参数分别是100、200
         * 任务函数会连续执行，向队列发送数值100、200
         * 优先级为1
         */
        xTaskCreate( vSenderTask, "Sender1", 1000, ( void * ) 100, 1, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, ( void * ) 200, 1, NULL );

        /* 创建1个任务用于读队列
         * 优先级为2，高于上面的两个任务
         * 这意味着队列一有数据就会被读走
         */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建队列 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务的函数中，不断往队列中写入数值，代码如下：

```
static void vSenderTask( void *pvParameters )
{
    int32_t lValueToSend;
```

```

BaseType_t xStatus;

/* 我们会使用这个函数创建2个任务
 * 这些任务的pvParameters不一样
 */
lValueToSend = ( int32_t ) pvParameters;

/* 无限循环 */
for( ;; )
{
    /* 写队列
     * xQueue: 写哪个队列
     * &lValueToSend: 写什么数据? 传入数据的地址, 会从这个地址把数据复制进队列
     * 0: 不阻塞, 如果队列满的话, 写入失败, 立刻返回
     */
    xStatus = xQueueSendToBack( xQueue, &lValueToSend, 0 );

    if( xStatus != pdPASS )
    {
        printf( "Could not send to the queue.\r\n" );
    }
}
}

```

接收任务的函数中, 读取队列、判断返回值、打印, 代码如下:

```

static void vReceiverTask( void *pvParameters )
{
    /* 读取队列时, 用这个变量来存放数据 */
    int32_t lReceivedValue;
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100UL );

    /* 无限循环 */
    for( ;; )
    {
        /* 读队列
         * xQueue: 读哪个队列
         * &lReceivedValue: 读到的数据复制到这个地址
         * xTicksToWait: 如果队列为空, 阻塞一会
         */
        xStatus = xQueueReceive( xQueue, &lReceivedValue, xTicksToWait );

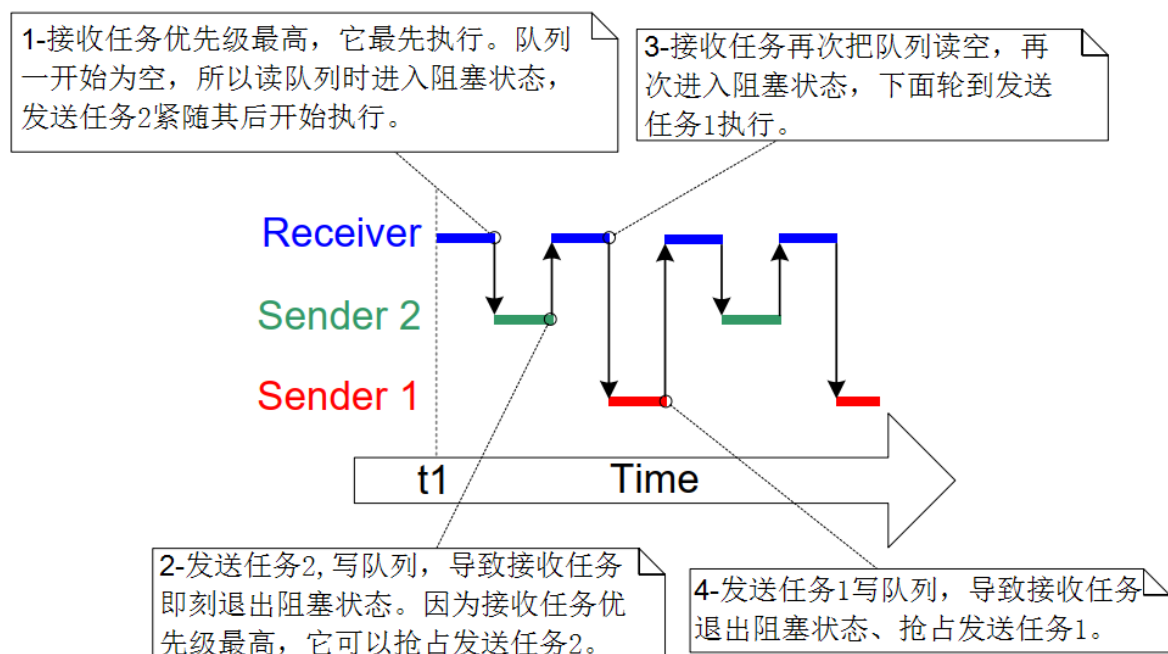
        if( xStatus == pdPASS )
        {
            /* 读到了数据 */
            printf( "Received = %d\r\n", lReceivedValue );
        }
        else
        {
            /* 没读到数据 */
            printf( "Could not receive from the queue.\r\n" );
        }
    }
}

```

程序运行结果如下：

UART #1
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200

任务调度情况如下图所示：



5.4 示例9: 分辨数据源

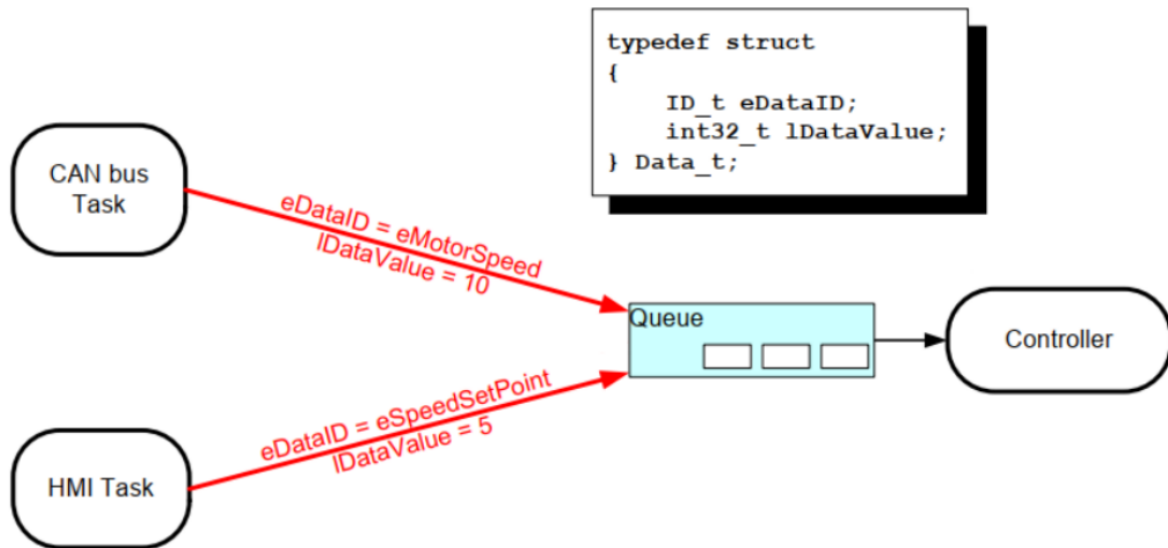
本节代码为: `FreeRTOS_09_queue_datasource`。

当有多个发送任务, 通过同一个队列发出数据, 接收任务如何分辨数据来源? 数据本身带有"来源"信息, 比如写入队列的数据是一个结构体, 结构体中的IDataSouceID用来表示数据来源:

```
typedef struct {
    ID_t eDataID;
    int32_t lDataValue;
}Data_t;
```

不同的发送任务, 先构造好结构体, 填入自己的 `eDataID`, 再写队列; 接收任务读出数据后, 根据 `eDataID` 就可以知道数据来源了, 如下图所示:

- CAN任务发送的数据: `eDataID=eMotorSpeed`
- HMI任务发送的数据: `eDataID=eSpeedSetPoint`



`FreeRTOS_09_queue_datasource` 程序会创建一个队列, 然后创建2个发送任务、1个接收任务:

- 创建的队列, 用来发送结构体: 数据大小是结构体的大小
- 发送任务优先级为2, 分别往队列中写入自己的结构体, 结构体中会标明数据来源
- 接收任务优先级为1, 读队列、根据数据来源打印信息

main函数中创建了队列、创建了发送任务、接收任务, 代码如下:

```
/* 定义2种数据来源(ID) */
typedef enum
{
    eMotorSpeed,
    eSpeedSetPoint
} ID_t;

/* 定义在队列中传输的数据的格式 */
typedef struct {
    ID_t eDataID;
    int32_t lDataValue;
}Data_t;
```

```

/* 定义2个结构体 */
static const Data_t xStructsToSend[ 2 ] =
{
    { eMotorSpeed,    10 }, /* CAN任务发送的数据 */
    { eSpeedSetPoint, 5 }   /* HMI任务发送的数据 */
};

/* vSenderTask被用来创建2个任务，用于写队列
 * vReceiverTask被用来创建1个任务，用于读队列
 */
static void vSenderTask( void *pvParameters );
static void vReceiverTask( void *pvParameters );

/*-----*/

/* 队列句柄，创建队列时会设置这个变量 */
QueueHandle_t xQueue;

int main( void )
{
    prvSetupHardware();

    /* 创建队列：长度为5，数据大小为4字节(存放一个整数) */
    xQueue = xQueueCreate( 5, sizeof( Data_t ) );

    if( xQueue != NULL )
    {
        /* 创建2个任务用于写队列，传入的参数是不同的结构体地址
         * 任务函数会连续执行，向队列发送结构体
         * 优先级为2
         */
        xTaskCreate(vSenderTask, "CAN Task", 1000, (void *) &
(xStructsToSend[0]), 2, NULL);
        xTaskCreate(vSenderTask, "HMI Task", 1000, (void *) &
(xStructsToSend[1]), 2, NULL);

        /* 创建1个任务用于读队列
         * 优先级为1，低于上面的两个任务
         * 这意味着发送任务优先写队列，队列常常是满的状态
         */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建队列 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

发送任务的函数中，不断往队列中写入数值，代码如下：

```

static void vSenderTask( void *pvParameters )
{
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100UL );

    /* 无限循环 */
    for( ;; )
    {
        /* 写队列
        * xQueue: 写哪个队列
        * pvParameters: 写什么数据? 传入数据的地址, 会从这个地址把数据复制进队列
        * xTicksToWait: 如果队列满的话, 阻塞一会
        */
        xStatus = xQueueSendToBack( xQueue, pvParameters, xTicksToWait );

        if( xStatus != pdPASS )
        {
            printf( "Could not send to the queue.\r\n" );
        }
    }
}

```

接收任务的函数中，读取队列、判断返回值、打印，代码如下：

```

static void vReceiverTask( void *pvParameters )
{
    /* 读取队列时，用这个变量来存放数据 */
    Data_t xReceivedStructure;
    BaseType_t xStatus;

    /* 无限循环 */
    for( ;; )
    {
        /* 读队列
        * xQueue: 读哪个队列
        * &xReceivedStructure: 读到的数据复制到这个地址
        * 0: 没有数据就即刻返回，不阻塞
        */
        xStatus = xQueueReceive( xQueue, &xReceivedStructure, 0 );

        if( xStatus == pdPASS )
        {
            /* 读到了数据 */
            if( xReceivedStructure.eDataID == eMotorSpeed )
            {
                printf( "From CAN, MotorSpeed = %d\r\n",
xReceivedStructure.lDataValue );
            }
            else if( xReceivedStructure.eDataID == eSpeedSetPoint )
            {
                printf( "From HMI, SpeedSetPoint = %d\r\n",
xReceivedStructure.lDataValue );
            }
        }
        else

```

```

    {
        /* 没读到数据 */
        printf( "could not receive from the queue.\r\n" );
    }
}
}

```

运行结果如下：

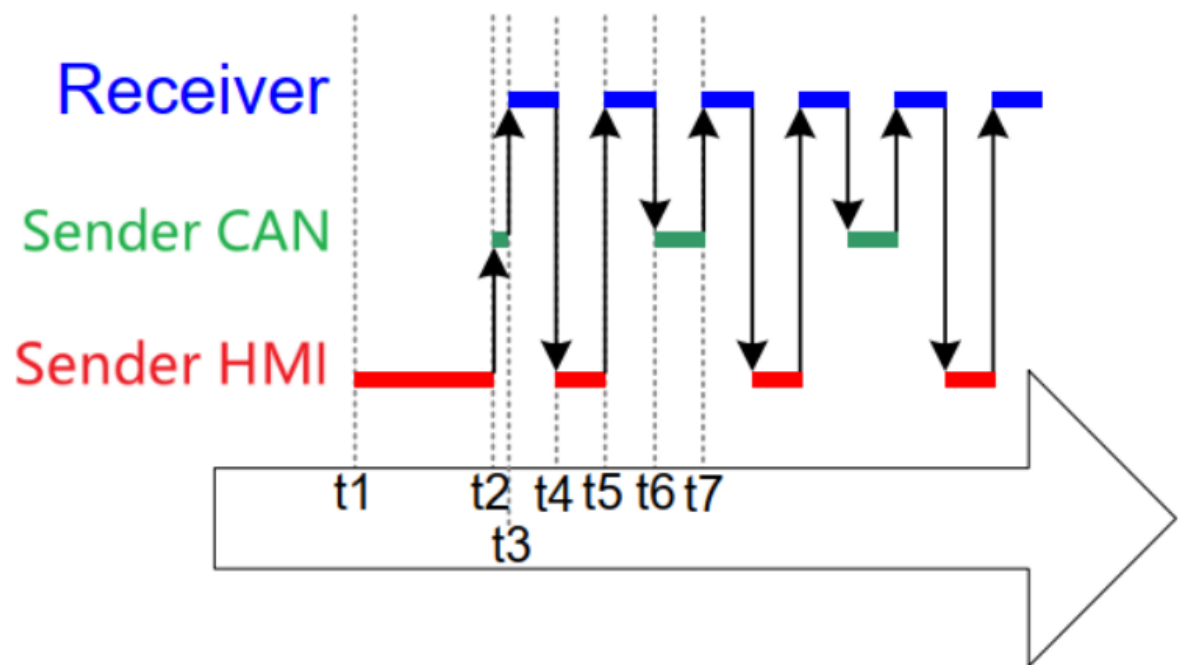
```

UART #1
From HMI, SpeedSetPoint = 5
From HMI, SpeedSetPoint = 5
From HMI, SpeedSetPoint = 5
From HMI, SpeedSetPoint = 5
From HMI, SpeedSetPoint = 5
From HMI, SpeedSetPoint = 5
From CAN, MotorSpeed = 10
From HMI, SpeedSetPoint = 5
From CAN, MotorSpeed = 10
From HMI, SpeedSetPoint = 5
From CAN, MotorSpeed = 10
From HMI, SpeedSetPoint = 5
From CAN, MotorSpeed = 10
From HMI, SpeedSetPoint = 5
From CAN, MotorSpeed = 10
From HMI, SpeedSetPoint = 5

```

任务调度情况如下图所示：

- t1: HMI是最后创建的最高优先级任务，它先执行，一下子向队列写入5个数据，把队列都写满了
- t2: 队列已经满了，HMI任务再发起第6次写操作时，进入阻塞状态。这时CAN任务是最高优先级的就绪态任务，它开始执行
- t3: CAN任务发现队列已经满了，进入阻塞状态；接收任务变为最高优先级的就绪态任务，它开始运行
- t4: 现在，HMI任务、CAN任务的优先级都比接收任务高，它们都在等待队列有空闲的空间；一旦接收任务读出1个数据，会马上被抢占。被谁抢占？谁等待最久？HMI任务！所以在t4时刻，切换到HMI任务。
- t5: HMI任务向队列写入第6个数据，然后再次阻塞，这是CAN任务已经阻塞很久了。接收任务变为最高优先级的就绪态任务，开始执行。
- t6: 现在，HMI任务、CAN任务的优先级都比接收任务高，它们都在等待队列有空闲的空间；一旦接收任务读出1个数据，会马上被抢占。被谁抢占？谁等待最久？CAN任务！所以在t6时刻，切换到CAN任务。
- t7: CAN任务向队列写入数据，因为仅仅有一个空间供写入，所以它马上再次进入阻塞状态。这时HMI任务、CAN任务都在等待空闲空间，只有接收任务可以继续执行。



5.5 示例10: 传输大块数据

本节代码为: `FreeRTOS_10_queue_bigtransfer`。

FreeRTOS的队列使用拷贝传输,也就是要传输uint32_t时,把4字节的数据拷贝进队列;要传输一个8字节的结构体时,把8字节的数据拷贝进队列。

如果要传输1000字节的结构体呢?写队列时拷贝1000字节,读队列时再拷贝1000字节?不建议这么做,影响效率!

这时候,我们要传输的是这个巨大结构体的地址:把它的地址写入队列,对方从队列得到这个地址,使用地址去访问那1000字节的数据。

使用地址来间接传输数据时,这些数据放在RAM里,对于这块RAM,要保证这几点:

- RAM的所有者、操作者,必须清晰明了
这块内存,就被称为"共享内存"。要确保不能同时修改RAM。比如,在写队列之前只有由发送者修改这块RAM,在读队列之后只能由接收者访问这块RAM。
- RAM要保持可用
这块RAM应该是全局变量,或者是动态分配的内存。对于动态分配的内存,要确保它不能提前释放:要等到接收者用完后释放。另外,不能是局部变量。

`FreeRTOS_10_queue_bigtransfer` 程序会创建一个队列,然后创建1个发送任务、1个接收任务:

- 创建的队列:长度为1,用来传输"char *"指针
- 发送任务优先级为1,在字符数组中写好数据后,把它的地址写入队列
- 接收任务优先级为2,读队列得到"char *"值,把它打印出来

这个程序故意设置接收任务的优先级更高,在它访问数组的过程中,接收任务无法执行、无法写这个数组。

main函数中创建了队列、创建了发送任务、接收任务,代码如下:

```
/* 定义一个字符数组 */
static char pcBuffer[100];

/* vSenderTask被用来创建2个任务,用于写队列
 * vReceiverTask被用来创建1个任务,用于读队列
 */
static void vSenderTask( void *pvParameters );
static void vReceiverTask( void *pvParameters );

/*-----*/

/* 队列句柄,创建队列时会设置这个变量 */
QueueHandle_t xQueue;

int main( void )
{
    prvSetupHardware();

    /* 创建队列:长度为1,数据大小为4字节(存放一个char指针) */
    xQueue = xQueueCreate( 1, sizeof(char *) );
```

```

if( xQueue != NULL )
{
    /* 创建1个任务用于写队列
     * 任务函数会连续执行，构造buffer数据，把buffer地址写入队列
     * 优先级为1
     */
    xTaskCreate( vSenderTask, "Sender", 1000, NULL, 1, NULL );

    /* 创建1个任务用于读队列
     * 优先级为2，高于上面的两个任务
     * 这意味着读队列得到buffer地址后，本任务使用buffer时不会被打断
     */
    xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

    /* 启动调度器 */
    vTaskStartScheduler();
}
else
{
    /* 无法创建队列 */
}

/* 如果程序运行到了这里就表示出错了，一般是内存不足 */
return 0;
}

```

发送任务的函数中，现在全局大数组pcBuffer中构造数据，然后把它的地址写入队列，代码如下：

```

static void vSenderTask( void *pvParameters )
{
    BaseType_t xStatus;
    static int cnt = 0;

    char *buffer;

    /* 无限循环 */
    for( ;; )
    {
        sprintf(pcBuffer, "www.100ask.net Msg %d\r\n", cnt++);
        buffer = pcBuffer; // buffer变量等于数组的地址，下面要把这个地址写入队列

        /* 写队列
         * xQueue: 写哪个队列
         * pvParameters: 写什么数据？传入数据的地址，会从这个地址把数据复制进队列
         * 0: 如果队列满的话，即刻返回
         */
        xStatus = xQueueSendToBack( xQueue, &buffer, 0 ); /* 只需要写入4字节，无需
        写入整个buffer */

        if( xStatus != pdPASS )
        {
            printf( "Could not send to the queue.\r\n" );
        }
    }
}

```

```
}
```

接收任务的函数中，读取队列、得到buffer的地址、打印，代码如下：

```
static void vReceiverTask( void *pvParameters )
{
    /* 读取队列时，用这个变量来存放数据 */
    char *buffer;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100UL );
    BaseType_t xStatus;

    /* 无限循环 */
    for( ;; )
    {
        /* 读队列
         * xQueue: 读哪个队列
         * &xReceivedStructure: 读到的数据复制到这个地址
         * xTicksToWait: 没有数据就阻塞一会
         */
        xStatus = xQueueReceive( xQueue, &buffer, xTicksToWait); /* 得到buffer地
        址，只是4字节 */

        if( xStatus == pdPASS )
        {
            /* 读到了数据 */
            printf("Get: %s", buffer);
        }
        else
        {
            /* 没读到数据 */
            printf( "Could not receive from the queue.\r\n" );
        }
    }
}
```

运行结果如下图所示：

UART #1
Get: www.100ask.net Msg 0
Get: www.100ask.net Msg 1
Get: www.100ask.net Msg 2
Get: www.100ask.net Msg 3
Get: www.100ask.net Msg 4
Get: www.100ask.net Msg 5
Get: www.100ask.net Msg 6
Get: www.100ask.net Msg 7
Get: www.100ask.net Msg 8
Get: www.100ask.net Msg 9
Get: www.100ask.net Msg 10
Get: www.100ask.net Msg 11
Get: www.100ask.net Msg 12

5.6 示例11: 邮箱(Mailbox)

本节代码为: `FreeRTOS_11_queue_mailbox`。

FreeRTOS的邮箱概念跟别的RTOS不一样, 这里的邮箱称为"橱窗"也许更恰当:

- 它是一个队列, 队列长度只有1
- 写邮箱: 新数据覆盖旧数据, 在任务中使用 `xQueueOverwrite()`, 在中断中使用 `xQueueOverwriteFromISR()`。
既然是覆盖, 那么无论邮箱中是否有数据, 这些函数总能成功写入数据。
- 读邮箱: 读数据时, 数据不会被移除; 在任务中使用 `xQueuePeek()`, 在中断中使用 `xQueuePeekFromISR()`。
这意味着, 第一次调用时会因为无数据而阻塞, 一旦曾经写入数据, 以后读邮箱时总能成功。

main函数中创建了队列(队列长度为1)、创建了发送任务、接收任务:

- 发送任务的优先级为2, 它先执行
- 接收任务的优先级为1

代码如下:

```
/* 队列句柄, 创建队列时会设置这个变量 */
QueueHandle_t xQueue;

int main( void )
{
    prvSetupHardware();

    /* 创建队列: 长度为1, 数据大小为4字节(存放一个char指针) */
    xQueue = xQueueCreate( 1, sizeof(uint32_t) );

    if( xQueue != NULL )
    {
        /* 创建1个任务用于写队列
         * 任务函数会连续执行, 构造buffer数据, 把buffer地址写入队列
         * 优先级为2
         */
        xTaskCreate( vSenderTask, "Sender", 1000, NULL, 2, NULL );

        /* 创建1个任务用于读队列
         * 优先级为1
         */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建队列 */
    }

    /* 如果程序运行到了这里就表示出错了, 一般是内存不足 */
    return 0;
}
```

发送任务、接收任务的代码和执行流程如下：

- A: 发送任务先执行，马上阻塞
- BC: 接收任务执行，这是邮箱无数据，打印"Could not ...". 在发送任务阻塞过程中，接收任务多次执行、多次打印。
- D: 发送任务从阻塞状态退出，立刻执行、写队列
- E: 发送任务再次阻塞
- FG、HI、.....: 接收任务不断"偷看"邮箱，得到同一个数据，打印出多个"Get: 0"
- J: 发送任务从阻塞状态退出，立刻执行、覆盖队列，写入1
- K: 发送任务再次阻塞
- LM、.....: 接收任务不断"偷看"邮箱，得到同一个数据，打印出多个"Get: 1"

```
127 static void vSenderTask( void *pvParameters )
128 {
129     BaseType_t xStatus;
130     static int cnt = 0;
131     const TickType_t xTicksToWait = pdMS_TO_TICKS( 5UL );
132
133     /* 无限循环 */
134     for( ;; )
135     {
136         /* 一开始就阻塞，让接收任务先运行
137          * 从下一个循环开始，在阻塞期间接收任务会打印很多值
138          */
139         vTaskDelay(xTicksToWait); A E K
140
141         /* 写队列
142          * xQueue: 写哪个队列
143          * pvParameters: 写什么数据?
144          * 传入数据的地址，会从这个地址把数据复制进队列
145          * 0: 如果队列满的话，即刻返回
146          */
147         xStatus = xQueueOverwrite( xQueue, &cnt ); D J
148         cnt++;
149
150         if( xStatus != pdPASS )
151         {
152             printf( "Could not send to the queue.\r\n" );
153         }
154     }
155 }
```

```
158 static void vReceiverTask( void *pvParameters )
159 {
160     /* 读取队列时，用这个变量来存放数据 */
161     int cnt = 0;
162     BaseType_t xStatus;
163
164     /* 无限循环 */
165     for( ;; )
166     {
167         /* 读队列
168          * xQueue: 读哪个队列
169          * &xReceivedStructure: 读到的数据复制到这个地址
170          * 0: 没有数据也即刻返回
171          */
172         xStatus = xQueuePeek( xQueue, &cnt, 0 ); B F H L
173
174         if( xStatus == pdPASS )
175         {
176             /* 读到了数据 */
177             printf( "Get: %d\r\n", cnt ); G I M
178         }
179         else
180         {
181             /* 没读到数据 */
182             printf( "Could not receive from the queue.\r\n" );
183         }
184     }
185 }
```

运行结果如下图所示：

```
UART #1
Could not receive from the queue.
Could not receive from the queue.
Get: 0
Get: 0
Get: 0
Get: 0
Get: 0
Get: 0
Get: 1
Get: 1
Get: 1
Get: 1
Get: 1
Get: 1
Get: 1
Get: 2
Get: 2
Get: 2
```

第六章 信号量(semaphore)

前面介绍的队列(queue)可以用于传输数据：在任务之间、任务和中断之间。

有时候我们只需要传递状态，并不需要传递具体的信息，比如：

- 我的事做完了，通知一下你
- 卖包子了、卖包子了，做好了1个包子！做好了2个包子！做好了3个包子！
- 这个停车位我占了，你们只能等着

在这种情况下我们可以使用信号量(semaphore)，它更节省内存。

本章涉及如下内容：

- 怎么创建、删除信号量
- 怎么发送、获得信号量
- 什么是计数型信号量？什么是二进制信号量？

6.1 信号量的特性

6.1.1 信号量的常规操作

信号量这个名字很恰当：

- 信号：起通知作用
- 量：还可以用来表示资源的数量
 - 当"量"没有限制时，它就是"计数型信号量"(Counting Semaphores)
 - 当"量"只有0、1两个取值时，它就是"二进制信号量"(Binary Semaphores)
- 支持的动作："give"给出资源，计数值加1；"take"获得资源，计数值减1

计数型信号量的典型场景是：

- 计数：事件产生时"give"信号量，让计数值加1；处理事件时要先"take"信号量，就是获得信号量，让计数值减1。
- 资源管理：要想访问资源需要先"take"信号量，让计数值减1；用完资源后"give"信号量，让计数值加1。

信号量的"give"、"take"双方并不需要相同，可以用于生产者-消费者场合：

- 生产者任务A、B，消费者任务C、D
- 一开始信号量的计数值为0，如果任务C、D想获得信号量，会有两种结果：
 - 阻塞：买不到东西咱就等等吧，可以定个闹钟(超时时间)
 - 即刻返回失败：不等
- 任务A、B可以生产资源，就是让信号量的计数值增加1，并且把等待这个资源的顾客唤醒
- 唤醒谁？谁优先级高就唤醒谁，如果大家优先级一样就唤醒等待时间最长的人

二进制信号量跟计数型的唯一差别，就是计数值的最大值被限定为1。



6.1.2 信号量跟队列的对比

差异列表如下：

队列	信号量
可以容纳多个数据， 创建队列时有2部分内存: 队列结构体、存储数据的空间	只有计数值，无法容纳其他数据。 创建信号量时，只需要分配信号量结构体
生产者：没有空间存入数据时可以阻塞	生产者：用于不阻塞，计数值已经达到最大时返回失败
消费者：没有数据时可以阻塞	消费者：没有资源时可以阻塞

6.1.3 两种信号量的对比

信号量的计数值都有限制：限定了最大值。如果最大值被限定为1，那么它就是二进制信号量；如果最大值不是1，它就是计数型信号量。

差别列表如下：

二进制信号量	技术型信号量
被创建时初始值为0	被创建时初始值可以设定
其他操作是一样的	其他操作是一样的

6.2 信号量函数

使用信号量时，先创建、然后去添加资源、获得资源。使用句柄来表示一个信号量。

6.2.1 创建

使用信号量之前，要先创建，得到一个句柄；使用信号量时，要使用句柄来表明使用哪个信号量。

对于二进制信号量、计数型信号量，它们的创建函数不一样：

	二进制信号量	计数型信号量
动态创建	<code>xSemaphoreCreateBinary</code> 计数值初始值为0	<code>xSemaphoreCreateCounting</code>
	<code>vSemaphoreCreateBinary</code> (过时了) 计数值初始值为1	
静态创建	<code>xSemaphoreCreateBinaryStatic</code>	<code>xSemaphoreCreateCountingStatic</code>

创建二进制信号量的函数原型如下：

```
/* 创建一个二进制信号量，返回它的句柄。
 * 此函数内部会分配信号量结构体
 * 返回值：返回句柄，非NULL表示成功
 */
SemaphoreHandle_t xSemaphoreCreateBinary( void );

/* 创建一个二进制信号量，返回它的句柄。
 * 此函数无需动态分配内存，所以需要先有一个StaticSemaphore_t结构体，并传入它的指针
 * 返回值：返回句柄，非NULL表示成功
 */
SemaphoreHandle_t xSemaphoreCreateBinaryStatic( StaticSemaphore_t
*pxSemaphoreBuffer );
```

创建计数型信号量的函数原型如下：

```
/* 创建一个计数型信号量，返回它的句柄。
 * 此函数内部会分配信号量结构体
 * uxMaxCount：最大计数值
 * uxInitialCount：初始计数值
 * 返回值：返回句柄，非NULL表示成功
 */
SemaphoreHandle_t xSemaphoreCreateCounting(UBaseType_t uxMaxCount, UBaseType_t
uxInitialCount);

/* 创建一个计数型信号量，返回它的句柄。
 * 此函数无需动态分配内存，所以需要先有一个StaticSemaphore_t结构体，并传入它的指针
 * uxMaxCount：最大计数值
 * uxInitialCount：初始计数值
```

```

    * pxSemaphoreBuffer: StaticSemaphore_t结构体指针
    * 返回值：返回句柄，非NULL表示成功
    */
SemaphoreHandle_t xSemaphoreCreateCountingStatic( UBaseType_t uxMaxCount,
                                                    UBaseType_t uxInitialCount,
                                                    StaticSemaphore_t
                                                    *pxSemaphoreBuffer );

```

6.2.2 删除

对于动态创建的信号量，不再需要它们时，可以删除它们以回收内存。

vSemaphoreDelete可以用来删除二进制信号量、计数型信号量，函数原型如下：

```

/*
 * xSemaphore: 信号量句柄，你要删除哪个信号量
 */
void vSemaphoreDelete( SemaphoreHandle_t xSemaphore );

```

6.2.3 give/take

二进制信号量、计数型信号量的give、take操作函数是一样的。这些函数也分为2个版本：给任务使用，给ISR使用。列表如下：

	在任务中使用	在ISR中使用
give	xSemaphoreGive	xSemaphoreGiveFromISR
take	xSemaphoreTake	xSemaphoreTakeFromISR

xSemaphoreGive的函数原型如下：

```

BaseType_t xSemaphoreGive( SemaphoreHandle_t xSemaphore );

```

xSemaphoreGive函数的参数与返回值列表如下：

参数	说明
xSemaphore	信号量句柄，释放哪个信号量
返回值	pdTRUE表示成功， 如果二进制信号量的计数值已经是1，再次调用此函数则返回失败； 如果计数型信号量的计数值已经是最大值，再次调用此函数则返回失败

pxHigherPriorityTaskWoken的函数原型如下：

```

BaseType_t xSemaphoreGiveFromISR(
    SemaphoreHandle_t xSemaphore,
    BaseType_t *pxHigherPriorityTaskWoken
);

```

xSemaphoreGiveFromISR函数的参数与返回值列表如下：

参数	说明
xSemaphore	信号量句柄，释放哪个信号量
pxHigherPriorityTaskWoken	如果释放信号量导致更高优先级的任务变为了就绪态，则*pxHigherPriorityTaskWoken = pdTRUE
返回值	pdTRUE表示成功， 如果二进制信号量的计数值已经是1，再次调用此函数则返回失败； 如果计数型信号量的计数值已经是最大值，再次调用此函数则返回失败

xSemaphoreTake的函数原型如下：

```

BaseType_t xSemaphoreTake(
    SemaphoreHandle_t xSemaphore,
    TickType_t xTicksToWait
);

```

xSemaphoreTake函数的参数与返回值列表如下：

参数	说明
xSemaphore	信号量句柄，获取哪个信号量
xTicksToWait	如果无法马上获得信号量，阻塞一会： 0: 不阻塞，马上返回 portMAX_DELAY: 一直阻塞直到成功 其他值: 阻塞的Tick个数，可以使用 pdMS_TO_TICKS() 来指定阻塞时间为若干ms
返回值	pdTRUE表示成功

xSemaphoreTakeFromISR的函数原型如下：

```

BaseType_t xSemaphoreTakeFromISR(
    SemaphoreHandle_t xSemaphore,
    BaseType_t *pxHigherPriorityTaskWoken
);

```

xSemaphoreTakeFromISR函数的参数与返回值列表如下：

参数	说明
xSemaphore	信号量句柄，获取哪个信号量
pxHigherPriorityTaskWoken	如果获取信号量导致更高优先级的任务变为了就绪态，则*pxHigherPriorityTaskWoken = pdTRUE
返回值	pdTRUE表示成功

6.3 示例12: 使用二进制信号量来同步

本节代码为： `FreeRTOS_12_semaphore_binary`。

main函数中创建了一个二进制信号量，然后创建2个任务：一个用于释放信号量，另一个用于获取信号量，代码如下：

```
/* 二进制信号量句柄 */
SemaphoreHandle_t xBinarySemaphore;

int main( void )
{
    prvSetupHardware();

    /* 创建二进制信号量 */
    xBinarySemaphore = xSemaphoreCreateBinary();

    if( xBinarySemaphore != NULL )
    {
        /* 创建1个任务用于释放信号量
         * 优先级为2
         */
        xTaskCreate( vSenderTask, "Sender", 1000, NULL, 2, NULL );

        /* 创建1个任务用于获取信号量
         * 优先级为1
         */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建二进制信号量 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务、接收任务的代码和执行流程如下：

- A：发送任务优先级高，先执行。连续3次释放二进制信号量，只有第1次成功
- B：发送任务进入阻塞态
- C：接收任务得以执行，得到信号量，打印OK；再次去获得信号量时，进入阻塞状态
- 在发送任务的vTaskDelay退出之前，运行的是空闲任务：现在发送任务、接收任务都阻塞了
- D：发送任务再次运行，连续3次释放二进制信号量，只有第1次成功
- E：发送任务进入阻塞态
- F：接收任务被唤醒，得到信号量，打印OK；再次去获得信号量时，进入阻塞状态

```

123 static void vSenderTask( void *pvParameters )
124 {
125     int i;
126     int cnt_ok = 0;
127     int cnt_err = 0;
128     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
129     /* 无限循环 */
130     for( ;; )
131     {
132         A D
133         {
134             for( i = 0; i < 3; i++)
135             {
136                 if( xSemaphoreGive(xBinarySemaphore) == pdTRUE)
137                     printf("Give BinarySemaphore %d time: OK\r\n", cnt_ok++);
138                 else
139                     printf("Give BinarySemaphore %d time: ERR\r\n", cnt_err++);
140             }
141             vTaskDelay(xTicksToWait); B E
142         }
143     }

```

```

146 static void vReceiverTask( void *pvParameters )
147 {
148     int cnt_ok = 0;
149     int cnt_err = 0;
150     /* 无限循环 */
151     for( ;; )
152     {
153         C F
154         {
155             if( xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE )
156             {
157                 /* 得到了二进制信号量 */
158                 printf("Get BinarySemaphore OK: %d\r\n", cnt_ok++);
159             }
160             else
161             {
162                 /* 没有得到了二进制信号量 */
163                 printf("Get BinarySemaphore ERR: %d\r\n", cnt_err++);
164             }
165         }
166     }

```

运行结果如下图所示，即使发送任务连续释放多个信号量，也只能成功1次。释放、获得信号量是一一对应的。

```

UART #1
Give BinarySemaphore 0 time: OK
Give BinarySemaphore 0 time: ERR
Give BinarySemaphore 1 time: ERR
Get BinarySemaphore OK: 0
Give BinarySemaphore 1 time: OK
Give BinarySemaphore 2 time: ERR
Give BinarySemaphore 3 time: ERR
Get BinarySemaphore OK: 1
Give BinarySemaphore 2 time: OK
Give BinarySemaphore 4 time: ERR
Give BinarySemaphore 5 time: ERR
Get BinarySemaphore OK: 2
Give BinarySemaphore 3 time: OK
Give BinarySemaphore 6 time: ERR
Give BinarySemaphore 7 time: ERR
Get BinarySemaphore OK: 3
Give BinarySemaphore 4 time: OK
Give BinarySemaphore 8 time: ERR

```

6.4 示例13: 防止数据丢失

本节代码为： `FreeRTOS_13_semaphore_circle_buffer`。

在示例12中，发送任务发出3次"提醒"，但是接收任务只接收到1次"提醒"，其中2次"提醒"丢失了。

这种情况很常见，比如每接收到一个串口字符，串口中断程序就给任务发一次"提醒"，假设收到多个字符、发出了多次"提醒"。当任务来处理时，它只能得到1次"提醒"。

你需要使用其他方法来防止数据丢失，比如：

- 在串口中断中，把数据放入缓冲区
- 在任务中，一次性把缓冲区中的数据都读出
- 简单地说，就是：你提醒了我多次，我太忙只响应你一次，但是我一次性拿走所有数据

main函数中创建了一个二进制信号量，然后创建2个任务：一个用于释放信号量，另一个用于获取信号量，代码如下：

```
/* 二进制信号量句柄 */
SemaphoreHandle_t xBinarySemaphore;

int main( void )
{
    prvSetupHardware();

    /* 创建二进制信号量 */
    xBinarySemaphore = xSemaphoreCreateBinary();

    if( xBinarySemaphore != NULL )
    {
        /* 创建1个任务用于释放信号量
         * 优先级为2
         */
        xTaskCreate( vSenderTask, "Sender", 1000, NULL, 2, NULL );

        /* 创建1个任务用于获取信号量
         * 优先级为1
         */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建二进制信号量 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务、接收任务的代码和执行流程如下：

- A: 发送任务优先级高，先执行。连续写入3个数据、释放3个信号量：只有1个信号量起作用
- B: 发送任务进入阻塞态
- C: 接收任务得以执行，得到信号量
- D: 接收任务一次性把所有数据取出
- E: 接收任务再次尝试获取信号量，进入阻塞状态
- 在发送任务的vTaskDelay退出之前，运行的是空闲任务：现在发送任务、接收任务都阻塞了
- F: 发送任务再次运行，连续写入3个数据、释放3个信号量：只有1个信号量起作用
- G: 发送任务进入阻塞态
- H: 接收任务被唤醒，得到信号量，一次性把所有数据取出

```

158 static void vSenderTask( void *pvParameters )
159 {
160     int i;
161     int cnt_tx = 0;
162     int cnt_ok = 0;
163     int cnt_err = 0;
164     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
165
166     /* 无限循环 */
167     for( ;; )
168     {
169         /* A F
170          * 放入数据 */
171         txbuf_put('a'+cnt_tx);
172         cnt_tx++;
173
174         /* 提醒对方 */
175         if (xSemaphoreGive(xBinarySemaphore) == pdTRUE)
176             printf("Give BinarySemaphore %d time: OK\r\n", cnt_ok++);
177         else
178             printf("Give BinarySemaphore %d time: ERR\r\n", cnt_err++);
179     }
180
181     vTaskDelay(xTicksToWait); /* B G
182
183
184 }

```

```

187 static void vReceiverTask( void *pvParameters )
188 {
189     int cnt_ok = 0;
190     int cnt_err = 0;
191     uint8_t c;
192
193     /* 无限循环 */
194     for( ;; )
195     {
196         /* C E
197          * 得到了二进制信号量 */
198         if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE )
199             printf("Get BinarySemaphore OK: %d, data: ", cnt_ok++);
200
201         /* 一次性把所有数据取出来 */
202         while (txbuf_get(&c) == 0)
203             printf("%c", c);
204         printf("\r\n");
205     }
206     else
207     {
208         /* 没有得到二进制信号量 */
209         printf("Get BinarySemaphore ERR: %d\r\n", cnt_err++);
210     }
211 }
212
213
214 }

```

程序运行结果如下，数据未丢失：

```

UART #1
Give BinarySemaphore 0 time: OK
Give BinarySemaphore 0 time: ERR
Give BinarySemaphore 1 time: ERR
Get BinarySemaphore OK: 0, data: abc
Give BinarySemaphore 1 time: OK
Give BinarySemaphore 2 time: ERR
Give BinarySemaphore 3 time: ERR
Get BinarySemaphore OK: 1, data: def
Give BinarySemaphore 2 time: OK
Give BinarySemaphore 4 time: ERR
Give BinarySemaphore 5 time: ERR
Get BinarySemaphore OK: 2, data: ghi
Give BinarySemaphore 3 time: OK
Give BinarySemaphore 6 time: ERR
Give BinarySemaphore 7 time: ERR
Get BinarySemaphore OK: 3, data: jkl
Give BinarySemaphore 4 time: OK

```

6.5 示例14: 使用计数型信号量

本节代码为： `FreeRTOS_14_semaphore_counting` 。

使用计数型信号量时，可以多次释放信号量；当信号量的技术值达到最大时，再次释放信号量就会出错。

如果信号量计数值为 n ，就可以连续 n 次获取信号量，第 $(n+1)$ 次获取信号量就会阻塞或失败。

main函数中创建了一个计数型信号量，最大计数值为3，初始值计数值为0；然后创建2个任务：一个用于释放信号量，另一个用于获取信号量，代码如下：

```
/* 计数型信号量句柄 */
SemaphoreHandle_t xCountingSemaphore;

int main( void )
{
    prvSetupHardware();

    /* 创建计数型信号量 */
    xCountingSemaphore = xSemaphoreCreateCounting(3, 0);

    if( xCountingSemaphore != NULL )
    {
        /* 创建1个任务用于释放信号量
         * 优先级为2
         */
        xTaskCreate( vSenderTask, "Sender", 1000, NULL, 2, NULL );

        /* 创建1个任务用于获取信号量
         * 优先级为1
         */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建信号量 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务、接收任务的代码和执行流程如下：

- A：发送任务优先级高，先执行。连续释放4个信号量：只有前面3次成功，第4次失败
- B：发送任务进入阻塞态
- CDE：接收任务得以执行，得到3个信号量
- F：接收任务试图获得第4个信号量时进入阻塞状态
- 在发送任务的`vTaskDelay`退出之前，运行的是空闲任务：现在发送任务、接收任务都阻塞了

- G: 发送任务再次运行, 连续释放4个信号量: 只有前面3次成功, 第4次失败
- H: 发送任务进入阻塞态
- IJK: 接收任务得以执行, 得到3个信号量
- L: 接收任务再次获取信号量时进入阻塞状态

```

122 static void vSenderTask( void *pvParameters )
123 {
124     int i;
125     int cnt_ok = 0;
126     int cnt_err = 0;
127     const TickType_t xTicksToWait = pdMS_TO_TICKS( 20UL );
128     /* 无限循环 */
129     for( ;; )
130     {
131         /* AG
132         for (i = 0; i < 4; i++)
133         {
134             /* 提醒对方 */
135             if (xSemaphoreGive(xCountingSemaphore) == pdTRUE)
136                 printf("Give CountingSemaphore %d time: OK\r\n", cnt_ok++);
137             else
138                 printf("Give CountingSemaphore %d time: ERR\r\n", cnt_err++);
139         }
140         vTaskDelay(xTicksToWait); BH
141     }
142 }
143
144 static void vReceiverTask( void *pvParameters )
145 {
146     int cnt_ok = 0;
147     int cnt_err = 0;
148     /* 无限循环 */
149     for( ;; )
150     {
151         /* CDEFIJKL
152         if (xSemaphoreTake(xCountingSemaphore, portMAX_DELAY) == pdTRUE )
153         {
154             /* 得到了信号量 */
155             printf("Get CountingSemaphore OK: %d\r\n", cnt_ok++);
156         }
157         else
158         {
159             /* 没有得到信号量 */
160             printf("Get BinarySemaphore ERR: %d\r\n", cnt_err++);
161         }
162     }
163 }
164
165 }

```

运行结果如下图所示:

```

UART #1
Give CountingSemaphore 0 time: OK
Give CountingSemaphore 1 time: OK
Give CountingSemaphore 2 time: OK
Give CountingSemaphore 0 time: ERR
Get CountingSemaphore OK: 0
Get CountingSemaphore OK: 1
Get CountingSemaphore OK: 2
Give CountingSemaphore 3 time: OK
Give CountingSemaphore 4 time: OK
Give CountingSemaphore 5 time: OK
Give CountingSemaphore 1 time: ERR
Get CountingSemaphore OK: 3
Get CountingSemaphore OK: 4
Get CountingSemaphore OK: 5
Give CountingSemaphore 6 time: OK
Give CountingSemaphore 7 time: OK
Give CountingSemaphore 8 time: OK
Give CountingSemaphore 2 time: ERR

```

第七章 互斥量(mutex)

怎么独享厕所？自己开门上锁，完事了自己开锁。

你当然可以进去后，让别人帮你把门：但是，命运就掌握在别人手上了。

使用队列、信号量，都可以实现互斥访问，以信号量为例：

- 信号量初始值为1
- 任务A想上厕所，"take"信号量成功，它进入厕所
- 任务B也想上厕所，"take"信号量不成功，等待
- 任务A用完厕所，"give"信号量；轮到任务B使用

这需要有2个前提：

- 任务B很老实，不撬门(一开始不"give"信号量)
- 没有坏人：别的任务不会"give"信号量

可以看到，使用信号量确实也可以实现互斥访问，但是不完美。

使用互斥量可以解决这个问题，互斥量的名字取得很好：

- 量：值为0、1
- 互斥：用来实现互斥访问

它的核心在于：谁上锁，就只能由谁开锁。

很奇怪的是，FreeRTOS的互斥锁，并没有在代码上实现这点：

- 即使任务A获得了互斥锁，任务B竟然也可以释放互斥锁。
- 谁上锁、谁释放：只是约定。

本章涉及如下内容：

- 为什么要实现互斥操作
- 怎么使用互斥量
- 互斥量导致的优先级反转、优先级继承

7.1 互斥量的使用场合

在多任务系统中，任务A正在使用某个资源，还没用完的情况下任务B也来使用的话，就可能导致问题。

比如对于串口，任务A正使用它来打印，在打印过程中任务B也来打印，客户看到的结果就是A、B的信息混杂在一起。

这种现象很常见：

- 访问外设：刚举的串口例子
- 读、修改、写操作导致的问题

对于同一个变量，比如 `int a`，如果有两个任务同时写它就有可能导致问题。

对于变量的修改，C代码只有一条语句，比如：`a=a+8;`，它的内部实现分为3步：读出原值、修改、写入。

```
int a = 1;
```

```
void add_a(void)
{
    a = a + 8;
}
```

①: R0 = [a地址]
②: R0 = R0+8
③: [a地址] = R0

我们想让任务A、B都执行add_a函数，a的最终结果是 `1+8+8=17`。

假设任务A运行完代码①，在执行代码②之前被任务B抢占了：现在任务A的R0等于1。

任务B执行完add_a函数，a等于9。

任务A继续运行，在代码②处R0仍然是被抢占前的数值1，执行完②③的代码，a等于9，这跟预期的17不符合。

- 对变量的非原子化访问

修改变量、设置结构体、在16位的机器上写32位的变量，这些操作都是非原子的。也就是它们的操作过程都可能被打断，如果被打断的过程有其他任务来操作这些变量，就可能导致冲突。

- 函数重入

"可重入的函数"是指：多个任务同时调用它、任务和中断同时调用它，函数的运行也是安全的。可重入的函数也被称为"线程安全"(thread safe)。

每个任务都维持自己的栈、自己的CPU寄存器，如果一个函数只使用局部变量，那么它就是线程安全的。

函数中一旦使用了全局变量、静态变量、其他外设，它就不是"可重入的"，如果改函数正在被调用，就必须阻止其他任务、中断再次调用它。

上述问题的解决方法是：任务A访问这些全局变量、函数代码时，独占它，就是上个锁。这些全局变量、函数代码必须被独占地使用，它们被称为临界资源。

互斥量也被称为互斥锁，使用过程如下：

- 互斥量初始值为1
- 任务A想访问临界资源，先获得并占有互斥量，然后开始访问
- 任务B也想访问临界资源，也要先获得互斥量：被别人占有了，于是阻塞
- 任务A使用完毕，释放互斥量；任务B被唤醒、得到并占有互斥量，然后开始访问临界资源
- 任务B使用完毕，释放互斥量

正常来说：在任务A占有互斥量的过程中，任务B、任务C等等，都无法释放互斥量。

但是FreeRTOS未实现这点：任务A占有互斥量的情况下，任务B也可释放互斥量。

7.2 互斥量函数

7.2.1 创建

互斥量是一种特殊的二进制信号量。

使用互斥量时，先创建、然后去获得、释放它。使用句柄来表示一个互斥量。

创建互斥量的函数有2种：动态分配内存，静态分配内存，函数原型如下：

```
/* 创建一个互斥量，返回它的句柄。
 * 此函数内部会分配互斥量结构体
 * 返回值：返回句柄，非NULL表示成功
 */
SemaphoreHandle_t xSemaphoreCreateMutex( void );

/* 创建一个互斥量，返回它的句柄。
 * 此函数无需动态分配内存，所以需要先有一个StaticSemaphore_t结构体，并传入它的指针
 * 返回值：返回句柄，非NULL表示成功
 */
SemaphoreHandle_t xSemaphoreCreateMutexStatic( StaticSemaphore_t *pxMutexBuffer
);
```

要想使用互斥量，需要在配置文件FreeRTOSConfig.h中定义：

```
##define configUSE_MUTEXES 1
```

7.2.2 其他函数

要注意的是，互斥量不能在ISR中使用。

各类操作函数，比如删除、give/take，跟一般是信号量是一样的。

```
/*
 * xSemaphore：信号量句柄，你要删除哪个信号量，互斥量也是一种信号量
 */
void vSemaphoreDelete( SemaphoreHandle_t xSemaphore );

/* 释放 */
BaseType_t xSemaphoreGive( SemaphoreHandle_t xSemaphore );

/* 释放(ISR版本) */
BaseType_t xSemaphoreGiveFromISR(
    SemaphoreHandle_t xSemaphore,
    BaseType_t *pxHigherPriorityTaskWoken
);

/* 获得 */
BaseType_t xSemaphoreTake(
    SemaphoreHandle_t xSemaphore,
```

```
        TickType_t xTicksToWait  
    );  
/* 获得(ISR版本) */  
xSemaphoreGiveFromISR(  
    SemaphoreHandle_t xSemaphore,  
    BaseType_t *pxHigherPriorityTaskWoken  
);
```

7.3 示例15: 互斥量基本使用

本节代码为： `FreeRTOS_15_mutex`。

使用互斥量时有如下特点：

- 刚创建的互斥量可以被成功"take"
- "take"互斥量成功的任务，被称为"holder"，只能由它"give"互斥量；别的任务"give"不成功
- 在ISR中不能使用互斥量

本程序创建2个发送任务：故意发送大量的字符。可以做2个实验：

- 使用互斥量：可以看到任务1、任务2打印的字符串没有混杂在一起
- 不使用互斥量：任务1、任务2打印的字符串混杂在一起

main函数代码如下：

```
/* 互斥量句柄 */
SemaphoreHandle_t xMutex;

int main( void )
{
    prvSetupHardware();

    /* 创建互斥量 */
    xMutex = xSemaphoreCreateMutex( );

    if( xMutex != NULL )
    {
        /* 创建2个任务：都是打印
         * 优先级相同
         */
        xTaskCreate( vSenderTask, "Sender1", 1000, (void *)1, 1, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, (void *)2, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建互斥量 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务的函数如下：

```
static void vSenderTask( void *pvParameters )
{
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
```

```

int cnt = 0;
int task = (int)pvParameters;
int i;
char c;

/* 无限循环 */
for( ;; )
{
    /* 获得互斥量：上锁 */
    xSemaphoreTake(xMutex, portMAX_DELAY);

    printf("Task %d use UART count: %d, ", task, cnt++);
    c = (task == 1) ? 'a' : 'A';
    for (i = 0; i < 26; i++)
        printf("%c", c + i);
    printf("\r\n");

    /* 释放互斥量：开锁 */
    xSemaphoreGive(xMutex);

    vTaskDelay(xTicksToWait);
}
}

```

可以做两个实验：vSenderTask函数的for循环中xSemaphoreTake和xSemaphoreGive这2句代码保留、不保留

- 保留：实验现象如下图左边，任务1、任务2的打印信息没有混在一起
- 不保留：实验现象如下图右边，打印信息混杂在一起

程序运行结果如下图所示：

使用互斥量

```

UART #1
Task 2 use UART count: 0, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 0, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 1, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 1, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 2, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 2, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 3, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 3, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 4, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 4, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 5, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 5, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 6, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 6, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 7, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 7, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 8, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 8, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 9, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 9, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 10, ABCDEFGHIJKLMNOPQRSTUVWXYZ

```

没使用互斥量

```

UART #1
Task 2 use UART count: 0, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 0, abcdefghijklmnopqrstuvwxyz
Task 2 Task 1 use UART count: 1, abcdefghijklmnopqrstuvwxyz
use UART count: 1, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 2, ABCDEFGHIJKLMNOPQRSTUVWXYZ
RT count: 2, abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 3, abcdefghijklmnopqrstuvwxyz
t: 3, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 4, Task 2 use UART count: 4, ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
Task 2 use UART count: 5, ABCDEFGTask 1 use UART count: 5, abcdefghijklmnopqrstuvwxyz
HIJKLMNOPQRSTUVWXYZ
Task 1 use UART count: 6, abcdefghijklmTask 2 use UART count: 6, ABCDEFGHIJKLMNOPQRSTUVWXYZ
nopqrstuvwxyz
Task 2 use UART count: 7, ABCDEFGHIJKLMNOPQRSTTask 1 use UART count: 7, abcdefghijklmnopqrstuvwxyz
UVWXYZ
Task 1 use UART count: 8, abcdefghijklmnopqrstuvwxyzTask 2 use UART count: 8, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task 2 use UART count: 9, ABCDEFGHIJKLMNOPQRSTUVWXYZ
Task Task 2 use UART count: 10, ABCDEFGHIJKLMNOPQRSTUVWXYZ
l use UART count: 9, abcdefghijklmnopqrstuvwxyz

```

7.4 示例16: 谁上锁就由谁解锁?

互斥量、互斥锁，本来的概念确实是：谁上锁就得由谁解锁。

但是FreeRTOS并没有实现这点，只是要求程序员按照这样的惯例写代码。

本节代码为： `FreeRTOS_16_mutex_who_give` 。

main函数创建了2个任务：

- 任务1：高优先级，一开始就获得互斥锁，永远不释放。
- 任务2：任务1阻塞时它开始执行，它先尝试获得互斥量，失败的话就监守自盗(释放互斥量、开锁)，然后再上锁

代码如下：

```
int main( void )
{
    prvSetupHardware();

    /* 创建互斥量 */
    xMutex = xSemaphoreCreateMutex( );

    if( xMutex != NULL )
    {
        /* 创建2个任务：一个上锁，另一个自己监守自盗(开别人的锁自己用)
        */
        xTaskCreate( vTakeTask, "Task1", 1000, NULL, 2, NULL );
        xTaskCreate( vGiveAndTakeTask, "Task2", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建互斥量 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

两个任务的代码和执行流程如下图所示：

- A：任务1的优先级高，先运行，立刻上锁
- B：任务1阻塞
- C：任务2开始执行，尝试获得互斥量(上锁)，超时时间设为0。根据返回值打印出：上锁失败
- D：任务2监守自盗，开锁，成功！
- E：任务2成功获得互斥量
- F：任务2阻塞

可见，任务1上的锁，被任务2解开了。所以，FreeRTOS并没有实现"谁上锁就得由谁开锁"的功能。

```

118 static void vTakeTask( void *pvParameters )
119 {
120     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
121     BaseType_t xStatus;
122
123     /* 获得互斥量: 上锁 */
124     xStatus = xSemaphoreTake(xMutex, portMAX_DELAY);
125     printf("Task1 take the Mutex %s\r\n", \
126           (xStatus == pdTRUE)? "Success" : "Failed"); A
127
128     /* 无限循环 */
129     for( ;; )
130     {
131         /* 什么都不做 */
132         vTaskDelay(xTicksToWait); B
133     }
134 }
135

```

```

136 static void vGiveAndTakeTask( void *pvParameters )
137 {
138     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
139     BaseType_t xStatus;
140
141     /* 尝试获得互斥量: 上锁 */
142     xStatus = xSemaphoreTake(xMutex, 0);
143     printf("Task2: at first, take the Mutex %s\r\n", \
144           (xStatus == pdTRUE)? "Success" : "Failed"); C
145
146     /* 如果失败则遵守自盗: 开锁 */
147     if (xStatus != pdTRUE)
148     {
149         xStatus = xSemaphoreGive(xMutex);
150         printf("Task2: give Mutex %s\r\n", \
151               (xStatus == pdTRUE)? "Success" : "Failed"); D
152     }
153
154     /* 最后成功获得互斥量 */
155     xStatus = xSemaphoreTake(xMutex, portMAX_DELAY);
156     printf("Task2: and then, take the Mutex %s\r\n", \
157           (xStatus == pdTRUE)? "Success" : "Failed"); E
158
159     /* 无限循环 */
160     for( ;; )
161     {
162         /* 什么都不做 */
163         vTaskDelay(xTicksToWait); F
164     }
165 }

```

程序运行结果如下图所示:

UART #1

```

Task1 take the Mutex Success
Task2: at first, take the Mutex Failed
Task2: give Mutex Success
Task2: and then, take the Mutex Success

```

7.5 示例17: 优先级反转

假设任务A、B都想使用串口，A优先级比较低：

- 任务A获得了串口的互斥量
- 任务B也想使用串口，它将会阻塞、等待A释放互斥量
- 高优先级的任务，被低优先级的任务延迟，这被称为"优先级反转"(priority inversion)

如果涉及3个任务，可以让"优先级反转"的后果更加恶劣。

本节代码为：FreeRTOS_17_mutex_inversion。

互斥量可以通过"优先级继承"，可以很大程度解决"优先级反转"的问题，这也是FreeRTOS中互斥量和二级制信号量的差别。

本节程序使用二级制信号量来演示"优先级反转"的恶劣后果。

main函数创建了3个任务：LPTask/MPTask/HPTask(低/中/高优先级任务)，代码如下：

```
/* 互斥量/二进制信号量句柄 */
SemaphoreHandle_t xLock;

int main( void )
{
    prvSetupHardware();

    /* 创建互斥量/二进制信号量 */
    xLock = xSemaphoreCreateBinary();

    if( xLock != NULL )
    {
        /* 创建3个任务：LP,MP,HP(低/中/高优先级任务) */
        /*
        xTaskCreate( vLPTask, "LP", 1000, NULL, 1, NULL );
        xTaskCreate( vMPTask, "MP", 1000, NULL, 2, NULL );
        xTaskCreate( vHPTask, "HP", 1000, NULL, 3, NULL );
        */

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建互斥量/二进制信号量 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

LPTask/MPTask/HPTask三个任务的代码和运行过程如下图所示：

- A：HPTask优先级最高，它最先运行。在这里故意打印，这样才可以观察到flagHPTaskRun的脉冲。
- HP Delay：HPTask阻塞

- B: MPTask开始运行。在这里故意打印，这样就可以观察到flagMPTaskRun的脉冲。
- MP Delay: MPTask阻塞
- C: LPTask开始运行，获得二进制信号量，然后故意打印很多字符
- D: HP Delay时间到，HPTask恢复运行，它无法获得二进制信号量，一直阻塞等待
- E: MP Delay时间到，MPTask恢复运行，它比LPTask优先级高，一直运行。导致LPTask无法运行，自然无法释放二进制信号量，于是HPTask无法运行。

总结:

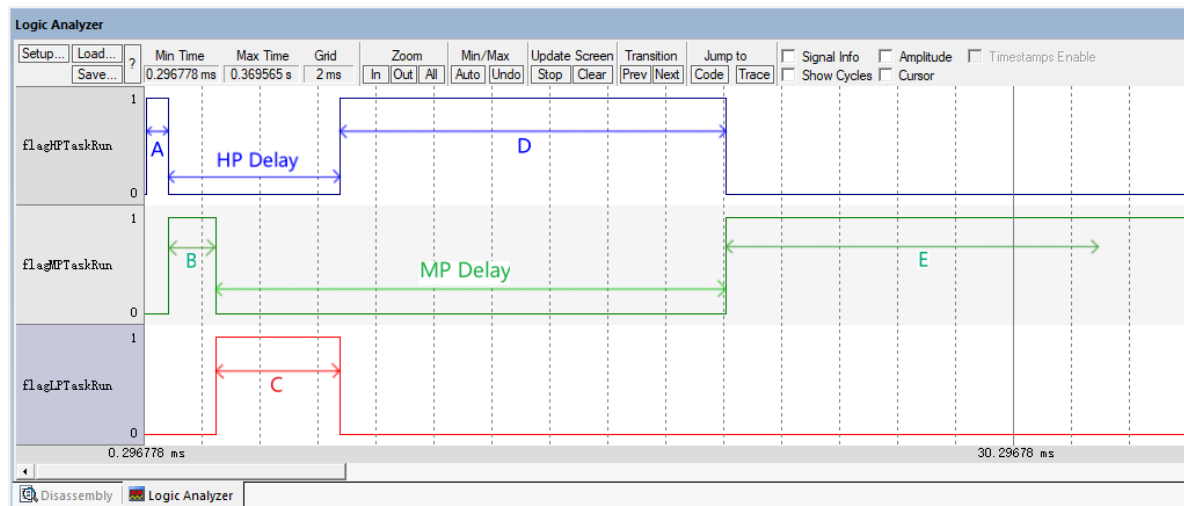
- LPTask先持有二进制信号量，
- 但是MPTask抢占LPTask，是的LPTask一直无法运行也就无法释放信号量，
- 导致HPTask任务无法运行
- 优先级最高的HPTask竟然一直无法运行！

```

123 static void vLPTask( void *pvParameters )
124 {
125     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
126     uint32_t i;
127     char c = 'A';
128
129     printf("LPTask start\r\n");
130
131     /* 无限循环 */
132     for ( ;; )
133     {
134         flagLPTaskRun = 1;
135         flagMPTaskRun = 0;
136         flagHPTaskRun = 0;
137
138         /* 获得互斥量/二进制信号量 */
139         xSemaphoreTake(xLock, portMAX_DELAY);
140
141         /* 耗时很久 */
142
143         printf("LPTask take the Lock for long time");
144         for ( i = 0; i < 26; i++)
145         {
146             flagLPTaskRun = 1;
147             flagMPTaskRun = 0;
148             flagHPTaskRun = 0;
149             printf("%c", c + i);
150         }
151         printf("\r\n");
152
153         /* 释放互斥量/二进制信号量 */
154         xSemaphoreGive(xLock);
155
156         vTaskDelay(xTicksToWait);
157     }
158 }
159
160 static void vMPTask( void *pvParameters )
161 {
162     const TickType_t xTicksToWait = pdMS_TO_TICKS( 30UL );
163
164     flagLPTaskRun = 0;
165     flagMPTaskRun = 1;
166     flagHPTaskRun = 0;
167
168     printf("MPTask start\r\n");
169
170     /* 让LPTask、HPTask先运行 */
171     vTaskDelay(xTicksToWait);
172
173     /* 无限循环 */
174     for ( ;; )
175     {
176         flagLPTaskRun = 0;
177         flagMPTaskRun = 1;
178         flagHPTaskRun = 0;
179     }
180 }
181
182 static void vHPTask( void *pvParameters )
183 {
184     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
185
186     flagLPTaskRun = 0;
187     flagMPTaskRun = 0;
188     flagHPTaskRun = 1;
189
190     printf("HPTask start\r\n");
191
192     /* 让LPTask先运行 */
193     vTaskDelay(xTicksToWait);
194
195     /* 无限循环 */
196     for ( ;; )
197     {
198         flagLPTaskRun = 0;
199         flagMPTaskRun = 0;
200         flagHPTaskRun = 1;
201         printf("HPTask wait for Lock\r\n");
202
203         /* 获得互斥量/二进制信号量 */
204         xSemaphoreTake(xLock, portMAX_DELAY);
205
206         flagLPTaskRun = 0;
207         flagMPTaskRun = 0;
208         flagHPTaskRun = 1;
209
210         /* 释放互斥量/二进制信号量 */
211         xSemaphoreGive(xLock);
212     }

```

程序运行的时序图如下:



7.6 示例18: 优先级继承

本节代码为： `FreeRTOS_18_mutex_inheritance`。

示例17的问题在于，LPTask低优先级任务获得了锁，但是它优先级太低而无法运行。

如果能提升LPTask任务的优先级，让它能尽快运行、释放锁，"优先级反转"的问题不就解决了吗？

把LPTask任务的优先级提升到什么水平？

优先级继承：

- 假设持有互斥锁的是任务A，如果更高优先级的任务B也尝试获得这个锁
- 任务B说：你既然持有宝剑，又不给我，那就继承我的愿望吧
- 于是任务A就继承了任务B的优先级
- 这就叫：优先级继承
- 等任务A释放互斥锁时，它就恢复为原来的优先级
- 互斥锁内部就实现了优先级的提升、恢复

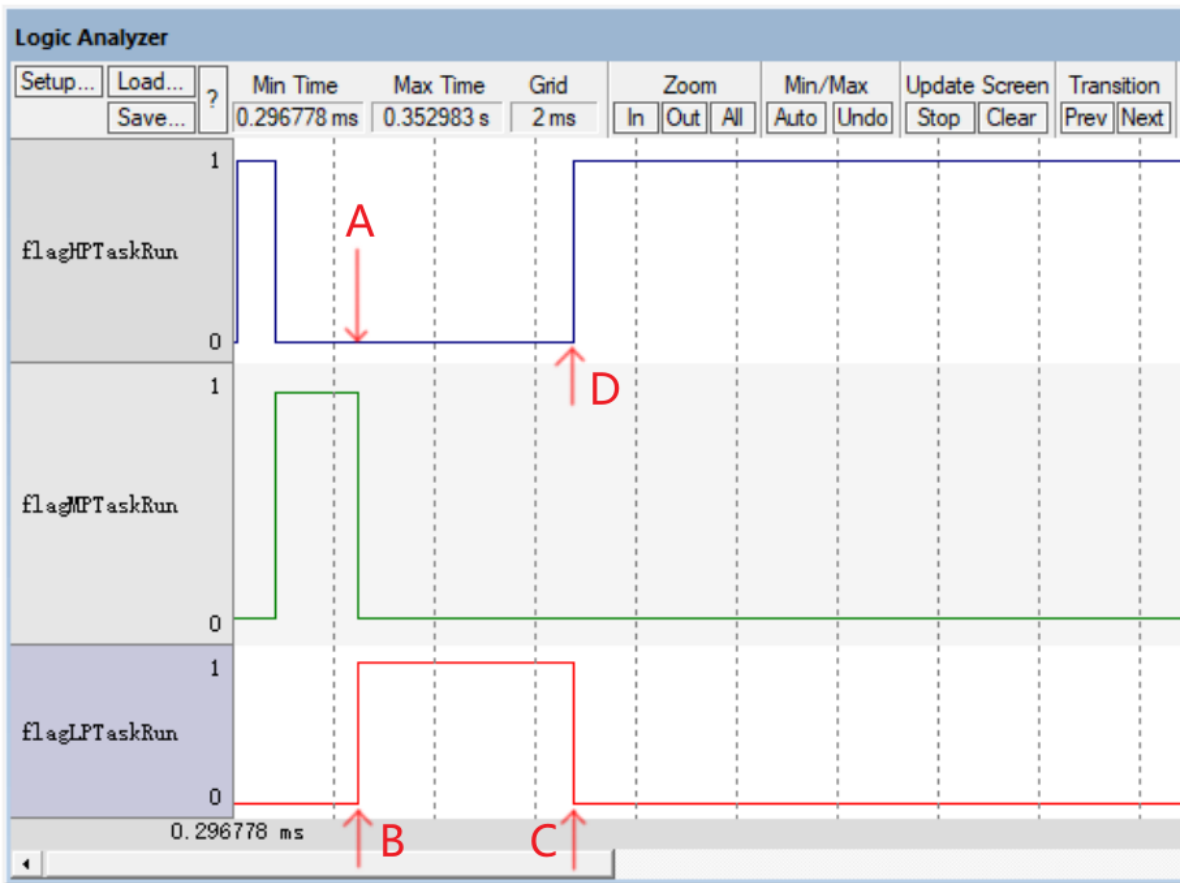
本节源码是在 `FreeRTOS_17_mutex_inversion` 的代码上做了一些简单修改：

```
int main( void )
{
    prvSetupHardware();

    /* 创建互斥量/二进制信号量 */
    //xLock = xSemaphoreCreateBinary( );
    xLock = xSemaphoreCreateMutex( );
```

运行时序图如下图所示：

- A：HPTask执行 `xSemaphoreTake(xLock, portMAX_DELAY);`，它的优先级被LPTask继承
- B：LPTask抢占MPTask，运行
- C：LPTask执行 `xSemaphoreGive(xLock);`，它的优先级恢复为原来值
- D：HPTask得到互斥锁，开始运行
- 互斥锁的"优先级继承"，可以减小"优先级反转"的影响



7.7 递归锁

7.7.1 死锁的概念

日常生活的死锁：我们只招有工作经验的人！我没有工作经验怎么办？那你就去找工作啊！

假设有2个互斥量M1、M2，2个任务A、B：

- A获得了互斥量M1
- B获得了互斥量M2
- A还要获得互斥量M2才能运行，结果A阻塞
- B还要获得互斥量M1才能运行，结果B阻塞
- A、B都阻塞，再无法释放它们持有的互斥量
- 死锁发生！

7.7.2 自我死锁

假设这样的场景：

- 任务A获得了互斥锁M
- 它调用一个库函数
- 库函数要去获取同一个互斥锁M，于是它阻塞：任务A休眠，等待任务A来释放互斥锁！
- 死锁发生！

7.7.3 函数

怎么解决这类问题？可以使用递归锁(Recursive Mutexes)，它的特性如下：

- 任务A获得递归锁M后，它还可以多次去获得这个锁
- "take"了N次，要"give"N次，这个锁才会被释放

递归锁的函数跟一般互斥量的函数名不一样，参数类型一样，列表如下：

	递归锁	一般互斥量
创建	xSemaphoreCreateRecursiveMutex	xSemaphoreCreateMutex
获得	xSemaphoreTakeRecursive	xSemaphoreTake
释放	xSemaphoreGiveRecursive	xSemaphoreGive

函数原型如下：

```
/* 创建一个递归锁，返回它的句柄。  
 * 此函数内部会分配互斥量结构体  
 * 返回值：返回句柄，非NULL表示成功  
 */  
SemaphoreHandle_t xSemaphoreCreateRecursiveMutex( void );
```

```

/* 释放 */
BaseType_t xSemaphoreGiveRecursive( SemaphoreHandle_t xSemaphore );

/* 获得 */
BaseType_t xSemaphoreTakeRecursive(
    SemaphoreHandle_t xSemaphore,
    TickType_t xTicksToWait
);

```

7.7.4 示例19: 递归锁

本节代码为： `FreeRTOS_19_mutex_recursive`。

递归锁实现了：谁上锁就由谁解锁。

本程序从 `FreeRTOS_16_mutex_who_give` 修改得来，它的main函数里创建了2个任务

- 任务1：高优先级，一开始就获得递归锁，然后故意等待很长时间，让任务2运行
- 任务2：低优先级，看看能否操作别人持有的锁

main函数代码如下：

```

/* 递归锁句柄 */
SemaphoreHandle_t xMutex;

int main( void )
{
    prvSetupHardware();

    /* 创建递归锁 */
    xMutex = xSemaphoreCreateRecursiveMutex( );

    if( xMutex != NULL )
    {
        /* 创建2个任务：一个上锁，另一个自己监守自盗(看看能否开别人的锁自己用) */
        xTaskCreate( vTakeTask, "Task1", 1000, NULL, 2, NULL );
        xTaskCreate( vGiveAndTakeTask, "Task2", 1000, NULL, 1, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建递归锁 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

两个任务经过精心设计，代码和运行流程如下图所示：

- A：任务1优先级最高，先运行，获得递归锁
- B：任务1阻塞，让任务2得以运行
- C：任务2运行，看看能否获得别人持有的递归锁：不能
- D：任务2故意执行"give"操作，看看能否释放别人持有的递归锁：不能
- E：任务2等待递归锁
- F：任务1阻塞时间到后继续运行，使用循环多次获得、释放递归锁
- 递归锁在代码上实现了：谁持有递归锁，必须由谁释放。

```
116 static void vTakeTask( void *pvParameters )
117 {
118     const TickType_t xTicksToWait = pdMS_TO_TICKS( 100UL );
119     BaseType_t xStatus;
120     int i;
121
122     /* 无限循环 */
123     for( ;; )
124     {
125         /* 获得递归锁：上锁 */
126         xStatus = xSemaphoreTakeRecursive(xMutex, portMAX_DELAY);
127         printf("Task1 take the Mutex in main loop %s\r\n", \
128             (xStatus == pdTRUE)? "Success" : "Failed");
129         /* 阻塞很长时间，让另一个任务执行，
130          * 看看它有无办法再次获得递归锁
131          */
132         vTaskDelay(xTicksToWait);
133
134         for( i = 0; i < 10; i++)
135         {
136             /* 获得递归锁：上锁 */
137             xStatus = xSemaphoreTakeRecursive(xMutex, portMAX_DELAY);
138             printf("Task1 take the Mutex in sub loop %s, for time %d\r\n", \
139                 (xStatus == pdTRUE)? "Success" : "Failed", i);
140             /* 释放递归锁 */
141             xSemaphoreGiveRecursive(xMutex);
142         }
143         /* 释放递归锁 */
144         xSemaphoreGiveRecursive(xMutex);
145     }
146 }
147
148
149
150
151
152
153 static void vGiveAndTakeTask( void *pvParameters )
154 {
155     const TickType_t xTicksToWait = pdMS_TO_TICKS( 10UL );
156     BaseType_t xStatus;
157
158     /* 尝试获得递归锁：上锁 */
159     xStatus = xSemaphoreTakeRecursive(xMutex, 0);
160     printf("Task2: at first, take the Mutex %s\r\n", \
161         (xStatus == pdTRUE)? "Success" : "Failed");
162
163     /* 如果失败则监听自旋：开锁 */
164     if (xStatus != pdTRUE)
165     {
166         /* 无法释放别人持有的锁 */
167         xStatus = xSemaphoreGiveRecursive(xMutex);
168         printf("Task2: give Mutex %s\r\n", \
169             (xStatus == pdTRUE)? "Success" : "Failed");
170     }
171
172     /* 如果无法获得，一直等待 */
173     xStatus = xSemaphoreTakeRecursive(xMutex, portMAX_DELAY);
174     printf("Task2: and then, take the Mutex %s\r\n", \
175         (xStatus == pdTRUE)? "Success" : "Failed");
176
177     /* 无限循环 */
178     for( ;; )
179     {
180         /* 什么都不做 */
181         vTaskDelay(xTicksToWait);
182     }
183 }
```

程序运行结果如下图所示：

```
UART #1
Task1 take the Mutex in main loop Success
Task2: at first, take the Mutex Failed
Task2: give Mutex Failed
Task1 take the Mutex in sub loop Success, for time 0
Task1 take the Mutex in sub loop Success, for time 1
Task1 take the Mutex in sub loop Success, for time 2
Task1 take the Mutex in sub loop Success, for time 3
Task1 take the Mutex in sub loop Success, for time 4
Task1 take the Mutex in sub loop Success, for time 5
Task1 take the Mutex in sub loop Success, for time 6
Task1 take the Mutex in sub loop Success, for time 7
Task1 take the Mutex in sub loop Success, for time 8
Task1 take the Mutex in sub loop Success, for time 9
Task1 take the Mutex in main loop Success
Task1 take the Mutex in sub loop Success, for time 0
Task1 take the Mutex in sub loop Success, for time 1
Task1 take the Mutex in sub loop Success, for time 2
Task1 take the Mutex in sub loop Success, for time 3
Task1 take the Mutex in sub loop Success, for time 4
Task1 take the Mutex in sub loop Success, for time 5
Task1 take the Mutex in sub loop Success, for time 6
```

7.8 常见问题

使用互斥量的两个任务是相同优先级时的注意事项。

第八章 事件组(event group)

学校组织秋游，组长在等待：

- 张三：我到了
- 李四：我到了
- 王五：我到了
- 组长说：好，大家都到齐了，出发！

秋游回来第二天就要提交一篇心得报告，组长在焦急等待：张三、李四、王五谁先写好就交谁的。

在这个日常生活场景中：

- 出发：要等待这3个人都到齐，他们是"与"的关系
- 交报告：只需等待这3人中的任何一个，他们是"或"的关系

在FreeRTOS中，可以使用事件组(event group)来解决这些问题。

本章涉及如下内容：

- 事件组的概念与操作函数
- 事件组的优缺点
- 怎么设置、等待、清除事件组中的位
- 使用事件组来同步多个任务

8.1 事件组概念与操作

8.1.1 事件组的概念

事件组可以简单地认为就是一个整数：

- 的每一位表示一个事件
- 每一位事件的含义由程序员决定，比如：Bit0表示用来串口是否就绪，Bit1表示按键是否被按下
- 这些位，值为1表示事件发生了，值为0表示事件没发生
- 一个或多个任务、ISR都可以去写这些位；一个或多个任务、ISR都可以去读这些位
- 可以等待某一位、某些位中的任意一个，也可以等待多位



事件组用一个整数来表示，其中的高8位留给内核使用，只能用其他的位来表示事件。那么这个整数是多少位的？

- 如果configUSE_16_BIT_TICKS是1，那么这个整数就是16位的，低8位用来表示事件
- 如果configUSE_16_BIT_TICKS是0，那么这个整数就是32位的，低24位用来表示事件
- configUSE_16_BIT_TICKS是用来表示Tick Count的，怎么会影响事件组？这只是基于效率来考虑
 - 如果configUSE_16_BIT_TICKS是1，就表示该处理器使用16位更高效，所以事件组也使用16位
 - 如果configUSE_16_BIT_TICKS是0，就表示该处理器使用32位更高效，所以事件组也使用32位

8.1.2 事件组的操作

事件组和队列、信号量等不太一样，主要集中在2个地方：

- 唤醒谁？
 - 队列、信号量：事件发生时，只会唤醒一个任务
 - 事件组：事件发生时，会唤醒所有符号条件的任务，简单地说它有"广播"的作用
- 是否清除事件？
 - 队列、信号量：是消耗型的资源，队列的数据被读走就没了；信号量被获取后就减少了
 - 事件组：被唤醒的任务有两个选择，可以让事件保留不动，也可以清除事件

以上图为例，事件组的常规操作如下：

- 先创建事件组
- 任务C、D等待事件：
 - 等待什么事件？可以等待某一位、某些位中的任意一个，也可以等待多位。简单地说就是"或"、"与"的关系。
 - 得到事件时，要不要清除？可选择清除、不清除。
- 任务A、B产生事件：设置事件组里的某一位、某些位

8.2 事件组函数

8.2.1 创建

使用事件组之前，要先创建，得到一个句柄；使用事件组时，要使用句柄来表明使用哪个事件组。

有两种创建方法：动态分配内存、静态分配内存。函数原型如下：

```
/* 创建一个事件组，返回它的句柄。
 * 此函数内部会分配事件组结构体
 * 返回值：返回句柄，非NULL表示成功
 */
EventGroupHandle_t xEventGroupCreate( void );

/* 创建一个事件组，返回它的句柄。
 * 此函数无需动态分配内存，所以需要先有一个StaticEventGroup_t结构体，并传入它的指针
 * 返回值：返回句柄，非NULL表示成功
 */
EventGroupHandle_t xEventGroupCreateStatic( StaticEventGroup_t *
pxEventGroupBuffer );
```

8.2.2 删除

对于动态创建的事件组，不再需要它们时，可以删除它们以回收内存。

vEventGroupDelete可以用来删除事件组，函数原型如下：

```
/*
 * xEventGroup：事件组句柄，你要删除哪个事件组
 */
void vEventGroupDelete( EventGroupHandle_t xEventGroup )
```

8.2.3 设置事件

可以设置事件组的某个位、某些位，使用的函数有2个：

- 在任务中使用 xEventGroupSetBits()
- 在ISR中使用 xEventGroupSetBitsFromISR()

有一个或多个任务在等待事件，如果这些事件符合这些任务的期望，那么任务还会被唤醒。

函数原型如下：

```
/* 设置事件组中的位
 * xEventGroup：哪个事件组
 * uxBitsToSet：设置哪些位？
 *
 * 如果uxBitsToSet的bitX, bitY为1，那么事件组中的bitX, bitY被设置为1
 *
 * 可以用来设置多个位，比如 0x15 就表示设置bit4, bit2, bit0
 * 返回值：返回原来的事件值(没什么意义，因为很可能已经被其他任务修改了)
 */
EventBits_t xEventGroupSetBits( EventGroupHandle_t xEventGroup,
```

```

const EventBits_t uxBitsToSet );

/* 设置事件组中的位
 * xEventGroup: 哪个事件组
 * uxBitsToSet: 设置哪些位?
 *
 * 如果uxBitsToSet的bitX, bitY为1, 那么事件组中的bitX, bitY被设置为1
 *
 * 可以用来设置多个位, 比如 0x15 就表示设置bit4, bit2, bit0
 * pxHigherPriorityTaskWoken: 有没有导致更高优先级的任务进入就绪态? pdTRUE-有,
pdFALSE-没有
 * 返回值: pdPASS-成功, pdFALSE-失败
 */
BaseType_t xEventGroupSetBitsFromISR( EventGroupHandle_t xEventGroup,
const EventBits_t uxBitsToSet,
BaseType_t * pxHigherPriorityTaskWoken );

```

值得注意的是, ISR中的函数, 比如队列函数 `xQueueSendToBackFromISR`、信号量函数 `xSemaphoreGiveFromISR`, 它们会唤醒某个任务, 最多只会唤醒1个任务。

但是设置事件组时, 有可能导致多个任务被唤醒, 这会带来很大的不确定性。所以 `xEventGroupSetBitsFromISR` 函数不是直接去设置事件组, 而是给一个FreeRTOS后台任务(daemon task)发送队列数据, 由这个任务来设置事件组。

如果后台任务的优先级比当前被中断的任务优先级高, `xEventGroupSetBitsFromISR` 会设置 `*pxHigherPriorityTaskWoken` 为pdTRUE。

如果daemon task成功地把队列数据发送给了后台任务, 那么 `xEventGroupSetBitsFromISR` 的返回值就是pdPASS。

8.2.4 等待事件

使用 `xEventGroupWaitBits` 来等待事件, 可以等待某一位、某些位中的任意一个, 也可以等待多位; 等到期望的事件后, 还可以清除某些位。

函数原型如下:

```

EventBits_t xEventGroupWaitBits( EventGroupHandle_t xEventGroup,
const EventBits_t uxBitsToWaitFor,
const BaseType_t xClearOnExit,
const BaseType_t xWaitForAllBits,
TickType_t xTicksToWait );

```

先引入一个概念: unblock condition。一个任务在等待事件发生时, 它处于阻塞状态; 当期望的时间发生时, 这个状态就叫"unblock condition", 非阻塞条件, 或称为"非阻塞条件成立"; 当"非阻塞条件成立"后, 该任务就可以变为就绪态。

函数参数说明列表如下:

参数	说明
xEventGroup	等待哪个事件组？
uxBitsToWaitFor	等待哪些位？ 哪些位要被测试？
xWaitForAllBits	怎么测试？ 是"AND"还是"OR"？ pdTRUE: 等待的位，全部为1； pdFALSE: 等待的位，某一个为1即可
xClearOnExit	函数提出前是否要清除事件？ pdTRUE: 清除uxBitsToWaitFor指定的位 pdFALSE: 不清除
xTicksToWait	如果期待的事件未发生，阻塞多久。 可以设置为0：判断后即刻返回； 可设置为portMAX_DELAY：一定等到成功才返回； 可以设置为期望的Tick Count，一般用 pdMS_TO_TICKS() 把ms转换为Tick Count
返回值	返回的是事件值， 如果期待的事件发生了，返回的是"非阻塞条件成立"时的事件值； 如果是超时退出，返回的是超时时刻的事件值。

举例如下：

事件组的值	uxBitsToWaitFor	xWaitForAllBits	说明
0100	0101	pdTRUE	任务期望bit0,bit2都为1， 当前值只有bit2满足，任务进入阻塞态； 当事件组中bit0,bit2都为1时退出阻塞态
0100	0110	pdFALSE	任务期望bit0,bit2某一个为1， 当前值满足，所以任务成功退出
0100	0110	pdTRUE	任务期望bit1,bit2都为1， 当前值不满足，任务进入阻塞态； 当事件组中bit1,bit2都为1时退出阻塞态

你可以使用 `xEventGroupWaitBits()` 等待期望的事件，它发生之后再使用 `xEventGroupClearBits()` 来清除。但是这两个函数之间，有可能被其他任务或中断抢占，它们可能会修改事件组。

可以使用设置 `xClearOnExit` 为 `pdTRUE`，使得对事件组的测试、清零都在 `xEventGroupWaitBits()` 函数内部完成，这是一个原子操作。

8.2.5 同步点

有一个事情需要多个任务协同，比如：

- 任务A：炒菜
- 任务B：买酒
- 任务C：摆台
- A、B、C做好自己的事后，还要等别人做完；大家一起做完，才可开饭

使用 `xEventGroupSync()` 函数可以同步多个任务：

- 可以设置某位、某些位，表示自己做了什么事
- 可以等待某位、某些位，表示要等等其他任务
- 期望的时间发生后，`xEventGroupSync()` 才会成功返回。
- `xEventGroupSync` 成功返回后，会清除事件

`xEventGroupSync` 函数原型如下：

```
EventBits_t xEventGroupSync(    EventGroupHandle_t xEventGroup,
                                const EventBits_t uxBitsToSet,
                                const EventBits_t uxBitsToWaitFor,
                                TickType_t xTicksToWait );
```

参数列表如下：

参数	说明
xEventGroup	哪个事件组？
uxBitsToSet	要设置哪些事件？我完成了哪些事件？ 比如0x05(二进制为0101)会导致事件组的bit0,bit2被设置为1
uxBitsToWaitFor	等待那个位、哪些位？ 比如0x15(二进制10101)，表示要等待bit0,bit2,bit4都为1
xTicksToWait	如果期待的事件未发生，阻塞多久。 可以设置为0：判断后即刻返回； 可设置为portMAX_DELAY：一定等到成功才返回； 可以设置为期望的Tick Count，一般用 <code>pdMS_TO_TICKS()</code> 把ms转换为Tick Count
返回值	返回的是事件值， 如果期待的事件发生了，返回的是"非阻塞条件成立"时的事件值； 如果是超时退出，返回的是超时时刻的事件值。

8.3 示例20: 等待多个事件

本节源码是 `FreeRTOS_20_event_group_wait_multi_events`。

要使用事件组，代码中要有如下操作：

```
/* 1. 工程中添加event_groups.c */

/* 2. 源码中包含头文件 */
#include "event_groups.h"
```

假设大厨要等手下做完这些事才可以炒菜：洗菜、生火。

本程序创建3个任务：

- 任务1：洗菜
- 任务2：生火
- 任务3：炒菜。

main函数代码如下，它创建了3个任务：

```
int main( void )
{
    prvSetupHardware();

    /* 创建递归锁 */
    xEventGroup = xEventGroupCreate();

    if( xEventGroup != NULL )
    {
        /* 创建3个任务：洗菜/生火/炒菜 */
        /*
        xTaskCreate( vWashingTask, "Task1", 1000, NULL, 1, NULL );
        xTaskCreate( vFiringTask, "Task2", 1000, NULL, 2, NULL );
        xTaskCreate( vCookingTask, "Task3", 1000, NULL, 3, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建事件组 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

这3个任务的代码和执行流程如下：

- A: "炒菜任务"优先级最高，先执行。它要等待的2个事件未发生：洗菜、生火，进入阻塞状态
- B: "生火任务"接着执行，它要等待的1个事件未发生：洗菜，进入阻塞状态

- C: "洗菜任务"接着执行，它洗好菜，发出事件：洗菜，然后调用F等待"炒菜"事件
- D: "生火任务"等待的事件满足了，从B处继续执行，开始生火、发出"生火"事件
- E: "炒菜任务"等待的事件满足了，从A处继续执行，开始炒菜、发出"炒菜"事件
- F: "洗菜任务"等待的事件满足了，退出F、继续执行C

要注意的是，代码B处等待到"洗菜任务"后并不清除该事件，如果清除的话会导致"炒菜任务"无法执行。

```

126 static void vWashingTask( void *pvParameters )
127 {
128     int i = 0;
129     /* 无限循环 */
130     for( ;; )
131     {
132         C
133         printf("I am washing %d time...\r\n", i++);
134         /* 发出事件：我洗完菜了 */
135         xEventGroupSetBits(xEventGroup, WASHING);
136         /* 等待大厨炒完菜，再继续洗菜 */
137         xEventGroupWaitBits(xEventGroup, COOKING, pdTRUE, pdTRUE, portMAX_DELAY);
138         F
139     }
140 }
141
142 static void vFiringTask( void *pvParameters )
143 {
144     int i = 0;
145     /* 无限循环 */
146     for( ;; )
147     {
148         B
149         /* 等待洗完菜，才生火 */
150         xEventGroupWaitBits(xEventGroup, WASHING, pdFALSE, pdTRUE, portMAX_DELAY);
151         printf("I am firing %d time...\r\n", i++);
152         /* 发出事件：我生好火了 */
153         xEventGroupSetBits(xEventGroup, FIRING);
154         D
155     }
156 }
157
158
159
160 static void vCookingTask( void *pvParameters )
161 {
162     int i = 0;
163     /* 无限循环 */
164     for( ;; )
165     {
166         A
167         /* 等待2件事：洗菜，生火 */
168         xEventGroupWaitBits(xEventGroup, WASHING|FIRING, pdTRUE, pdTRUE, portMAX_DELAY);
169         printf("I am cooking %d time...\r\n", i++);
170         /* 发出事件：我炒好菜了 */
171         xEventGroupSetBits(xEventGroup, COOKING);
172         E
173     }
174 }
175
176

```

运行结果如下图所示：

```

UART #1
I am washing 0 time....
I am firing 0 time....
I am cooking 0 time....
I am washing 1 time....
I am firing 1 time....
I am cooking 1 time....
I am washing 2 time....
I am firing 2 time....
I am cooking 2 time....
I am washing 3 time....
I am firing 3 time....
I am cooking 3 time....

```

8.3 示例21: 任务同步

本节代码是 `FreeRTOS_21_event_group_task_sync`。

假设ABC三人要吃饭，各司其职：

- A: 炒菜
- B: 买酒
- C: 摆台

三人都做完后，才可以开饭。

main函数代码如下，它创建了3个任务：

```
int main( void )
{
    prvSetupHardware();

    /* 创建递归锁 */
    xEventGroup = xEventGroupCreate( );

    if( xEventGroup != NULL )
    {
        /* 创建3个任务：洗菜/生火/炒菜
         */
        xTaskCreate( vCookingTask, "task1", 1000, "A", 1, NULL );
        xTaskCreate( vBuyingTask, "task2", 1000, "B", 2, NULL );
        xTaskCreate( vTableTask, "task3", 1000, "C", 3, NULL );

        /* 启动调度器 */
        vTaskStartScheduler();
    }
    else
    {
        /* 无法创建事件组 */
    }

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

被创建的3个任务，代码都很类似，以任务1为例：

```
static void vCookingTask( void *pvParameters )
{
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100UL );
    int i = 0;

    /* 无限循环 */
    for( ;; )
    {
        /* 做自己的事 */
    }
}
```

```

    printf("%s is cooking %d time....\r\n", (char *)pvParameters, i);

    /* 表示我做好了，还要等别人都做好 */
    xEventGroupSync(xEventGroup, COOKING, ALL, portMAX_DELAY);

    /* 别人也做好了，开饭 */
    printf("%s is eating %d time....\r\n", (char *)pvParameters, i++);
    vTaskDelay(xTicksToWait);
}
}

```

要点在于 `xEventGroupSync` 函数，它有3个功能：

- 设置事件：表示自己完成了某个、某些事件
- 等待事件：跟别的任务同步
- 成功返回后，清除"等待的事件"

运行结果如下图所示：

```

UART #1
C is do the table 0 time....
B is buying 0 time....
A is cooking 0 time....
C is eating 0 time....
B is eating 0 time....
A is eating 0 time....
C is do the table 1 time....
B is buying 1 time....
A is cooking 1 time....
C is eating 1 time....
B is eating 1 time....
A is eating 1 time....

```

第九章 任务通知(Task Notifications)

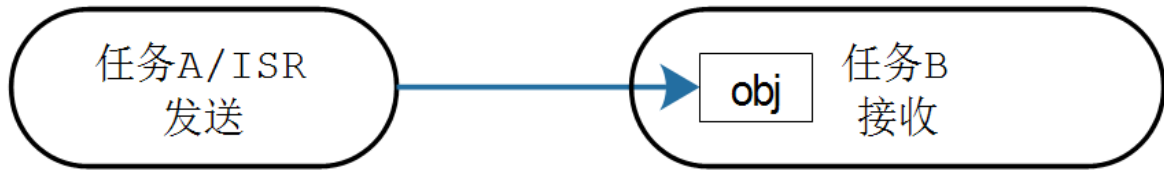
所谓"任务通知", 你可以反过来读"通知任务".

我们使用队列、信号量、事件组等等方法时, 并不知道对方是谁。使用任务通知时, 可以明确指定: 通知哪个任务。

使用队列、信号量、事件组时, 我们都要事先创建对应的结构体, 双方通过中间的结构体通信:



使用任务通知时, 任务结构体TCB中就包含了内部对象, 可以直接接收别人发过来的"通知":



本章涉及如下内容:

- 任务通知: 通知状态、通知值
- 任务通知的使用场合
- 任务通知的优势

9.1 任务通知的特性

9.1.1 优势及限制

任务通知的优势：

- 效率更高：使用任务通知来发送事件、数据给某个任务时，效率更高。比队列、信号量、事件组都有大的优势。
- 更节省内存：使用其他方法时都要先创建对应的结构体，使用任务通知时无需额外创建结构体。

任务通知的限制：

- 不能发送数据给ISR：
ISR并没有任务结构体，所以无法使用任务通知的功能给ISR发送数据。但是ISR可以使用任务通知的功能，发数据给任务。
- 数据只能给该任务独享
使用队列、信号量、事件组时，数据保存在这些结构体中，其他任务、ISR都可以访问这些数据。
使用任务通知时，数据存放入目标任务中，只有它可以访问这些数据。
在日常工作中，这个限制影响不大。因为很多场合是从多个数据源把数据发给某个任务，而不是把一个数据源的数据发给多个任务。
- 无法缓冲数据
使用队列时，假设队列深度为N，那么它可以保持N个数据。
使用任务通知时，任务结构体中只有一个任务通知值，只能保持一个数据。
- 无法广播给多个任务
使用事件组可以同时给多个任务发送事件。
使用任务通知，只能发一个任务。
- 如果发送受阻，发送方无法进入阻塞状态等待
假设队列已经满了，使用 `xQueueSendToBack()` 给队列发送数据时，任务可以进入阻塞状态等待发送完成。
使用任务通知时，即使对方无法接收数据，发送方也无法阻塞等待，只能即刻返回错误。

9.1.2 通知状态和通知值

每个任务都有一个结构体：TCB(Task Control Block)，里面有2个成员：

- 一个是uint8_t类型，用来表示通知状态
- 一个是uint32_t类型，用来表示通知值

```
typedef struct tskTaskControlBlock
{
    .....
    /* configTASK_NOTIFICATION_ARRAY_ENTRIES = 1 */
    volatile uint32_t ulNotifiedValue[ configTASK_NOTIFICATION_ARRAY_ENTRIES ];
    volatile uint8_t ucNotifyState[ configTASK_NOTIFICATION_ARRAY_ENTRIES ];
    .....
} tskTCB;
```

通知状态有3种取值：

- taskNOT_WAITING_NOTIFICATION: 任务没有在等待通知
- taskWAITING_NOTIFICATION: 任务在等待通知
- taskNOTIFICATION_RECEIVED: 任务接收到了通知, 也被称为pending(有数据了, 待处理)

```
##define taskNOT_WAITING_NOTIFICATION      ( ( uint8_t ) 0 )  /* 也是初始  
状态 */  
##define taskWAITING_NOTIFICATION          ( ( uint8_t ) 1 )  
##define taskNOTIFICATION_RECEIVED         ( ( uint8_t ) 2 )
```

通知值可以有很多种类型:

- 计数值
- 位(类似事件组)
- 任意数值

9.2 任务通知的使用

使用任务通知，可以实现轻量级的队列(长度为1)、邮箱(覆盖的队列)、计数型信号量、二进制信号量、事件组。

9.2.1 两类函数

任务通知有2套函数，简化版、专业版，列表如下：

- 简化版函数的使用比较简单，它实际上也是使用专业版函数实现的
- 专业版函数支持很多参数，可以实现很多功能

	简化版	专业版
发出通知	xTaskNotifyGive vTaskNotifyGiveFromISR	xTaskNotify xTaskNotifyFromISR
取出通知	ulTaskNotifyTake	xTaskNotifyWait

9.2.2 xTaskNotifyGive/ulTaskNotifyTake

在任务中使用xTaskNotifyGive函数，在ISR中使用vTaskNotifyGiveFromISR函数，都是直接给其他任务发送通知：

- 使得通知值加一
- 并使得通知状态变为"pending"，也就是 taskNOTIFICATION_RECEIVED，表示有数据了、待处理

可以使用ulTaskNotifyTake函数来取出通知值：

- 如果通知值等于0，则阻塞(可以指定超时时间)
- 当通知值大于0时，任务从阻塞态进入就绪态
- 在ulTaskNotifyTake返回之前，还可以做些清理工作：把通知值减一，或者把通知值清零

使用ulTaskNotifyTake函数可以实现轻量级的、高效的二进制信号量、计数型信号量。

这几个函数的原型如下：

```
BaseType_t xTaskNotifyGive( TaskHandle_t xTaskToNotify );

void vTaskNotifyGiveFromISR( TaskHandle_t xTaskHandle, BaseType_t
*pxHigherPriorityTaskWoken );

uint32_t ulTaskNotifyTake( BaseType_t xClearCountOnExit, TickType_t xTicksToWait
);
```

xTaskNotifyGive函数的参数说明如下：

参数	说明
xTaskToNotify	任务句柄(创建任务时得到), 给哪个任务发通知
返回值	必定返回pdPASS

vTaskNotifyGiveFromISR函数的参数说明如下:

参数	说明
xTaskHandle	任务句柄(创建任务时得到), 给哪个任务发通知
pxHigherPriorityTaskWoken	被通知的任务, 可能正处于阻塞状态。 此函数发出通知后, 会把它从阻塞状态切换为就绪态。 如果被唤醒的任务的优先级, 高于当前任务的优先级, 则"*pxHigherPriorityTaskWoken"被设置为pdTRUE, 这表示在中断返回之前要进行任务切换。

ulTaskNotifyTake函数的参数说明如下:

参数	说明
xClearCountOnExit	函数返回前是否清零: pdTRUE: 把通知值清零 pdFALSE: 如果通知值大于0, 则把通知值减一
xTicksToWait	任务进入阻塞态的超时时间, 它在等待通知值大于0。 0: 不等待, 即刻返回; portMAX_DELAY: 一直等待, 直到通知值大于0; 其他值: Tick Count, 可以用 pdMS_TO_TICKS() 把ms转换为Tick Count
返回值	函数返回之前, 在清零或减一之前的通知值。 如果xTicksToWait非0, 则返回值有2种情况: 1. 大于0: 在超时前, 通知值被增加了 2. 等于0: 一直没有其他任务增加通知值, 最后超时返回0

9.2.3 xTaskNotify/xTaskNotifyWait

xTaskNotify 函数功能更强大, 可以使用不同参数实现各类功能, 比如:

- 让接收任务的通知值加一: 这时 xTaskNotify() 等同于 xTaskNotifyGive()
- 设置接收任务的通知值的某一位、某些位, 这就是一个轻量级的、更高效的事件组
- 把一个新值写入接收任务的通知值: 上一次的通知值被读走后, 写入才成功。这就是轻量级的、长度为1的队列
- 用一个新值覆盖接收任务的通知值: 无论上一次的通知值是否被读走, 覆盖都成功。类似 xQueueOverwrite() 函数, 这就是轻量级的邮箱。

xTaskNotify() 比 xTaskNotifyGive() 更灵活、强大, 使用上也就更复杂。

xTaskNotifyFromISR() 是它对应的ISR版本。

这两个函数用来发出任务通知，使用哪个函数来取出任务通知呢？

使用 `xTaskNotifyWait()` 函数！它比 `ulTaskNotifyTake()` 更复杂：

- 可以让任务等待(可以加上超时时间)，等到任务状态为"pending"(也就是有数据)
- 还可以在函数进入、退出时，清除通知值的指定位

这几个函数的原型如下：

```
BaseType_t xTaskNotify( TaskHandle_t xTaskToNotify, uint32_t ulValue,
eNotifyAction eAction );

BaseType_t xTaskNotifyFromISR( TaskHandle_t xTaskToNotify,
                                uint32_t ulValue,
                                eNotifyAction eAction,
                                BaseType_t *pxHigherPriorityTaskWoken );

BaseType_t xTaskNotifyWait( uint32_t ulBitsToClearOnEntry,
                             uint32_t ulBitsToClearOnExit,
                             uint32_t *pulNotificationValue,
                             TickType_t xTicksToWait );
```

`xTaskNotify`函数的参数说明如下：

参数	说明
xTaskToNotify	任务句柄(创建任务时得到)，给哪个任务发通知
ulValue	怎么使用ulValue，由eAction参数决定
eAction	见下表
返回值	pdPASS：成功，大部分调用都会成功 pdFAIL：只有一种情况会失败，当eAction为eSetValueWithoutOverwrite，并且通知状态为"pending"(表示有新数据未读)，这时就会失败。

`eNotifyAction`参数说明：

eNotifyAction取值	说明
eNoAction	仅仅是更新通知状态为"pending"，未使用ulValue。 这个选项相当于轻量级的、更高效的二进制信号量。
eSetBits	通知值 = 原来的通知值 ulValue，按位或。 相当于轻量级的、更高效的事件组。
eIncrement	通知值 = 原来的通知值 + 1，未使用ulValue。 相当于轻量级的、更高效的二进制信号量、计数型信号量。 相当于 <code>xTaskNotifyGive()</code> 函数。
eSetValueWithoutOverwrite	不覆盖。 如果通知状态为"pending"(表示有数据未读)， 则此次调用 <code>xTaskNotify</code> 不做任何事，返回 <code>pdFAIL</code> 。

eSetValueWithoutOverwrite eNotifyAction取值	则此次调用xTaskNotify不做任何事，返回pdFAIL。 说明 如果通知状态不是"pending"(表示没有新数据)， 则：通知值 = ulValue。
eSetValueWithOverwrite	覆盖。 无论如何，不管通知状态是否为"pending"， 通知值 = ulValue。

xTaskNotifyFromISR函数跟xTaskNotify很类似，就多了最后一个参数

`pxHigherPriorityTaskWoken`。在很多ISR函数中，这个参数的作用都是类似的，使用场景如下：

- 被通知的任务，可能正处于阻塞状态
- `xTaskNotifyFromISR` 函数发出通知后，会把接收任务从阻塞状态切换为就绪状态
- 如果被唤醒的任务的优先级，高于当前任务的优先级，则"`*pxHigherPriorityTaskWoken`"被设置为pdTRUE，这表示在中断返回之前要进行任务切换。

xTaskNotifyWait函数列表如下：

参数	说明
ulBitsToClearOnEntry	在xTaskNotifyWait入口处，要清除通知值的哪些位？ 通知状态不是"pending"的情况下，才会清除。 它的本意是：我想等待某些事件发生，所以先把"旧数据"的某些位清零。 能清零的话：通知值 = 通知值 & ~(ulBitsToClearOnEntry)。 比如传入0x01，表示清除通知值的bit0； 传入0xffffffff即ULONG_MAX，表示清除所有位，即把值设置为0
ulBitsToClearOnExit	在xTaskNotifyWait出口处，如果不是因为超时推出，而是因为得到了数据而退出时： 通知值 = 通知值 & ~(ulBitsToClearOnExit)。 在清除某些位之前，通知值先被赋给" <code>*pulNotificationValue</code> "。 比如入0x03，表示清除通知值的bit0、bit1； 传入0xffffffff即ULONG_MAX，表示清除所有位，即把值设置为0
pulNotificationValue	用来取出通知值。 在函数退出时，使用ulBitsToClearOnExit清除之前，把通知值赋给" <code>*pulNotificationValue</code> "。 如果不需要取出通知值，可以设为NULL。
xTicksToWait	任务进入阻塞态的超时时间，它在等待通知状态变为"pending"。 0：不等待，即刻返回； portMAX_DELAY：一直等待，直到通知状态变为"pending"； 其他值：Tick Count，可以用 <code>pdMS_TO_TICKS()</code> 把ms转换为Tick Count
返回值	1. pdPASS：成功 这表示xTaskNotifyWait成功获得了通知： 可能是调用函数之前，通知状态就是"pending"； 也可能是在阻塞期间，通知状态变为了"pending"。 2. pdFAIL：没有得到通知。

9.3 示例22: 传输计数值

本节源码是 `FreeRTOS_22_tasknotify_tansfer_count`，基于 `FreeRTOS_13_semaphore_circle_buffer` 修改。

本程序创建2个任务：

- 发送任务：把数据写入唤醒缓冲区，使用 `xTaskNotifyGive()` 让通知值加一
- 接收任务：使用 `ulTaskNotifyTake()` 取出通知值，这表示字符数，打印字符

main函数代码如下：

```
int main( void )
{
    prvSetupHardware();

    /* 创建1个任务用于发送任务通知
     * 优先级为2
     */
    xTaskCreate( vSenderTask, "Sender", 1000, NULL, 2, NULL );

    /* 创建1个任务用于接收任务通知
     * 优先级为1
     */
    xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, &xRecvTask );

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务、接收任务的代码和执行流程如下：

- A：发送任务优先级最高，先执行。连续存入3个字符、发出3次任务通知：通知值累加为3
- B：发送任务阻塞，让接收任务能执行
- C：接收任务读到通知值为3，并把通知值清零
- D：把3个字符依次读出、打印
- E：再次读取任务通知，阻塞

```
147 static void vSenderTask( void *pvParameters )
148 {
149     int i;
150     int cnt_tx = 0;
151     int cnt_ok = 0;
152     int cnt_err = 0;
153     char c;
154     const TickType_t xTicksToWait = pdMS_TO_TICKS( 20UL );
155
156     /* 无限循环 */
157     for( ;; )
158     {
159         for ( i = 0; i < 3; i++)
160         {
161             /* 放入数据 */
162             c = 'a'+cnt_tx;
163             txbuf_put(c);
164             cnt_tx++;
165
166             /* 发出任务通知 */
167             if (xTaskNotifyGive(xRecvTask) == pdPASS)
168                 printf("xTaskNotifyGive %d time: OK, val :%c\r\n", cnt_ok++, c);
169             else
170                 printf("xTaskNotifyGive %d time: ERR\r\n", cnt_err++);
171         }
172
173         vTaskDelay(xTicksToWait);
174     }
175 }
```

```
178 static void vReceiverTask( void *pvParameters )
179 {
180     int cnt_ok = 0;
181     uint8_t c;
182     int notify_val;
183
184     /* 无限循环 */
185     for( ;; )
186     {
187         /* 得到了任务通知，让通知值清零 */
188         notify_val = ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
189
190         /* 打印这几个字符 */
191         printf("ulTaskNotifyTake OK: %d, data: ", cnt_ok++);
192
193         /* 一次性把所有数据取出来 */
194         while (notify_val-->0)
195         {
196             txbuf_get(&c);
197             printf("%c", c);
198         }
199         printf("\r\n");
200     }
201 }
202 }
```

运行结果如下图所示：

```
UART #1
xTaskNotifyGive 0 time: OK, val :a
xTaskNotifyGive 1 time: OK, val :b
xTaskNotifyGive 2 time: OK, val :c
ulTaskNotifyTake OK: 0, data: abc
xTaskNotifyGive 3 time: OK, val :d
xTaskNotifyGive 4 time: OK, val :e
xTaskNotifyGive 5 time: OK, val :f
ulTaskNotifyTake OK: 1, data: def
xTaskNotifyGive 6 time: OK, val :g
xTaskNotifyGive 7 time: OK, val :h
xTaskNotifyGive 8 time: OK, val :i
ulTaskNotifyTake OK: 2, data: ghi
```

本程序使用 xTaskNotifyGive/ulTaskNotifyTake 实现了轻量级的计数型信号量，代码更简单：

- 无需创建信号量
- 消耗内存更少
- 效率更高

信号量是个公开的资源，任何任务、ISR都可以使用它：可以释放、获取信号量。

而本节程序中，发送任务只能给指定的任务发送通知，目标明确；接收任务只能从自己的通知值中得到数据，来源明确。

9.4 示例23: 传输任意值

本节源码是 `FreeRTOS_23_tasknotify_transfer_value`。

在上述例子中使用任务通知来传输计数值、传输通知。

本节程序使用任务通知来传输任意数据，它创建2个任务：

- 发送任务：把数据通过 `xTaskNotify()` 发送给其他任务
- 接收任务：使用 `xTaskNotifyWait` 取出通知值，这表示字符，并打印出来

main函数代码如下：

```
int main( void )
{
    prvSetupHardware();

    /* 创建1个任务用于发送任务通知
     * 优先级为2
     */
    xTaskCreate( vSenderTask, "Sender", 1000, NULL, 2, NULL );

    /* 创建1个任务用于接收任务通知
     * 优先级为1
     */
    xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, &xRecvTask );

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}
```

发送任务、接收任务的代码和执行流程如下：

- A：发送任务优先级最高，先执行。连续给对方任务发送3个字符，只成功了1次
- B：发送任务阻塞，让接收任务能执行
- C：接收任务读取通知值
- D：把读到的通知值作为字符打印出来
- E：再次读取任务通知，阻塞

```
111 static void vSenderTask( void *pvParameters )
112 {
113     int i;
114     int cnt_tx = 0;
115     int cnt_ok = 0;
116     int cnt_err = 0;
117     char c;
118     const TickType_t xTicksToWait = pdMS_TO_TICKS( 200UL );
119
120     /* 无限循环 */
121     for( ;; )
122     {
123         /* A 放入数据 */
124         c = 'a'+cnt_tx;
125         cnt_tx++;
126
127         /* 发出任务通知
128          * 发给谁? xRecvTask
129          * 发什么? c
130          * 能否覆盖? 不能, eSetValueWithoutOverwrite
131          */
132         if (xTaskNotify(xRecvTask,
133             (uint32_t)c, /* 发送的数据 */
134             eSetValueWithoutOverwrite) == pdPASS)
135             printf("xTaskNotify %d time: OK, val :%c\r\n", cnt_ok++, c);
136         else
137             printf("xTaskNotify %d time: ERR, val :%c\r\n", cnt_err++, c);
138     }
139     vTaskDelay(xTicksToWait); B
140 }
141
142
143
144
145
146
147 static void vReceiverTask( void *pvParameters )
148 {
149     int cnt_ok = 0;
150     uint32_t ulValue;
151     BaseType_t xResult;
152
153     /* 无限循环 */
154     for( ;; )
155     {
156         /* C E 等待数据 */
157         xResult = xTaskNotifyWait(
158             0, /* 发送任务会存入新数据，无需接收方在进入函数时清零 */
159             0, /* 发送任务会存入新数据，无需接收方在退出函数时清零 */
160             &ulValue, /* 用来保存读出的数据 */
161             portMAX_DELAY /* 一直等待数据 */
162         );
163
164         /* D 打印这个字符 */
165         if (xResult == pdPASS)
166             printf("xTaskNotifyWait OK: %d, data: %c\r\n",
167                 cnt_ok++, (char)ulValue);
168     }
169 }
170
```

运行结果如下图所示：

```
UART #1
xTaskNotify 0 time: OK, val :a
xTaskNotify 0 time: ERR, val :b
xTaskNotify 1 time: ERR, val :c
xTaskNotifyWait OK: 0, data: a
xTaskNotify 1 time: OK, val :d
xTaskNotify 2 time: ERR, val :e
xTaskNotify 3 time: ERR, val :f
xTaskNotifyWait OK: 1, data: d
```

本程序使用 `xTaskNotify`/`xTaskNotifyWait` 实现了轻量级的队列(该队列长度只有1)，代码更简单：

- 无需创建队列
- 消耗内存更少
- 效率更高

队列是个公开的资源，任何任务、ISR都可以使用它：可以存入数据、取出数据。

而本节程序中，发送任务只能给指定的任务发送通知，目标明确；接收任务只能从自己的通知值中得到数据，来源明确。

注意：任务通知值只有一个，数据可能丢失，设计程序时要考虑这点。

第十章 软件定时器(software timer)

软件定时器就是"闹钟", 你可以设置闹钟,

- 在30分钟后让你起床工作
- 每隔1小时让你例行检查机器运行情况

软件定时器也可以完成两类事情:

- 在"未来"某个时间点, 运行函数
- 周期性地运行函数

日常生活中我们可以定无数个"闹钟", 这无数的"闹钟"要基于一个真实的闹钟。

在FreeRTOS里, 我们也可以设置无数个"软件定时器", 它们都是基于系统滴答中断(Tick Interrupt)。

本章涉及如下内容:

- 软件定时器的特性
- Daemon Task
- 定时器命令队列
- 一次性定时器、周期性定时器的差别
- 怎么操作定时器: 创建、启动、复位、修改周期

10.1 软件定时器的特性

我们在手机上添加闹钟时，需要指定时间、指定类型(一次性的，还是周期性的)、指定做什么事；还有一些过时的、不再使用的闹钟。如下图所示：



使用定时器跟使用手机闹钟是类似的:

- 指定时间: 启动定时器和运行回调函数, 两者的间隔被称为定时器的周期(period)。

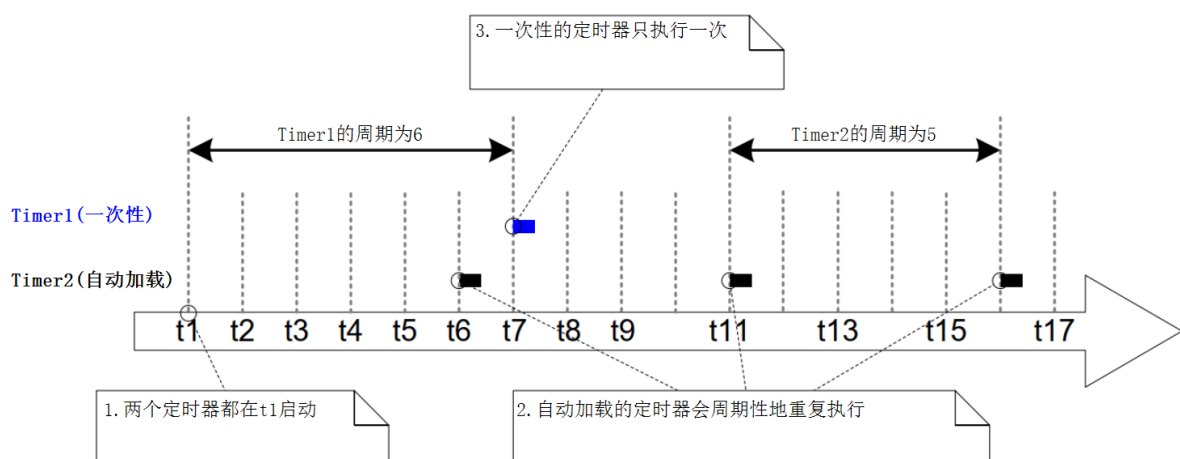
- 指定类型，定时器有两种类型：
 - 一次性(One-shot timers):
这类定时器启动后，它的回调函数只会被调用一次；
可以手工再次启动它，但是不会自动启动它。
 - 自动加载定时器(Auto-reload timers):
这类定时器启动后，时间到之后它会自动启动它；
这使得回调函数被周期性地调用。
- 指定要做什么事，就是指定回调函数

实际的闹钟分为：有效、无效两类。软件定时器也是类似的，它由两种状态：

- 运行(Running、Active)：运行态的定时器，当指定时间到达之后，它的回调函数会被调用
- 冬眠(Dormant)：冬眠态的定时器还可以通过句柄来访问它，但是它不再运行，它的回调函数不会被调用

定时器运行情况示例如下：

- Timer1：它是一次性的定时器，在t1启动，周期是6个Tick。经过6个tick后，在t7执行回调函数。它的回调函数只会被执行一次，然后该定时器进入冬眠状态。
- Timer2：它是自动加载的定时器，在t1启动，周期是5个Tick。每经过5个tick它的回调函数都被执行，比如在t6、t11、t16都会执行。



10.2 软件定时器的上下文

10.2.1 守护任务

要理解软件定时器API函数的参数，特别是里面的 `xTicksToWait`，需要知道定时器执行的过程。

FreeRTOS中有一个Tick中断，软件定时器基于Tick来运行。在哪里执行定时器函数？第一印象就是在Tick中断里执行：

- 在Tick中断中判断定时器是否超时
- 如果超时了，调用它的回调函数

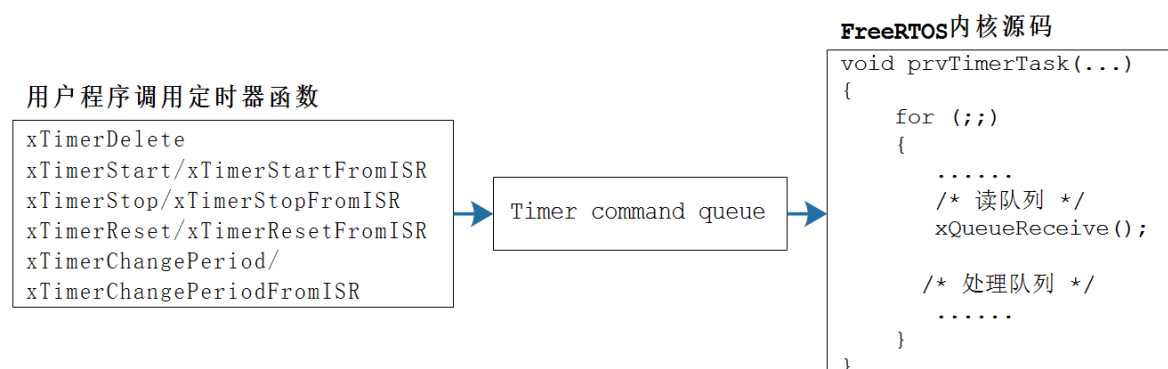
FreeRTOS是RTOS，它不允许在内核、在中断中执行不确定的代码：如果定时器函数很耗时，会影响整个系统。

所以，FreeRTOS中，不在Tick中断中执行定时器函数。

在哪里执行？在某个任务里执行，这个任务就是：RTOS Damemon Task，RTOS守护任务。以前被称为"Timer server"，但是这个任务要做并不仅仅是定时器相关，所以改名为：RTOS Damemon Task。

当FreeRTOS的配置项 `configUSE_TIMERS` 被设置为1时，在启动调度器时，会自动创建RTOS Damemon Task。

我们自己编写的任务函数要使用定时器时，是通过"定时器命令队列"(timer command queue)和守护任务交互，如下图所示：



守护任务的优先级为：`configTIMER_TASK_PRIORITY`；定时器命令队列的长度为 `configTIMER_QUEUE_LENGTH`。

10.2.2 守护任务的调度

守护任务的调度，跟普通的任务并无差别。当守护任务是当前优先级最高的就绪态任务时，它就可以运行。它的工作有两类：

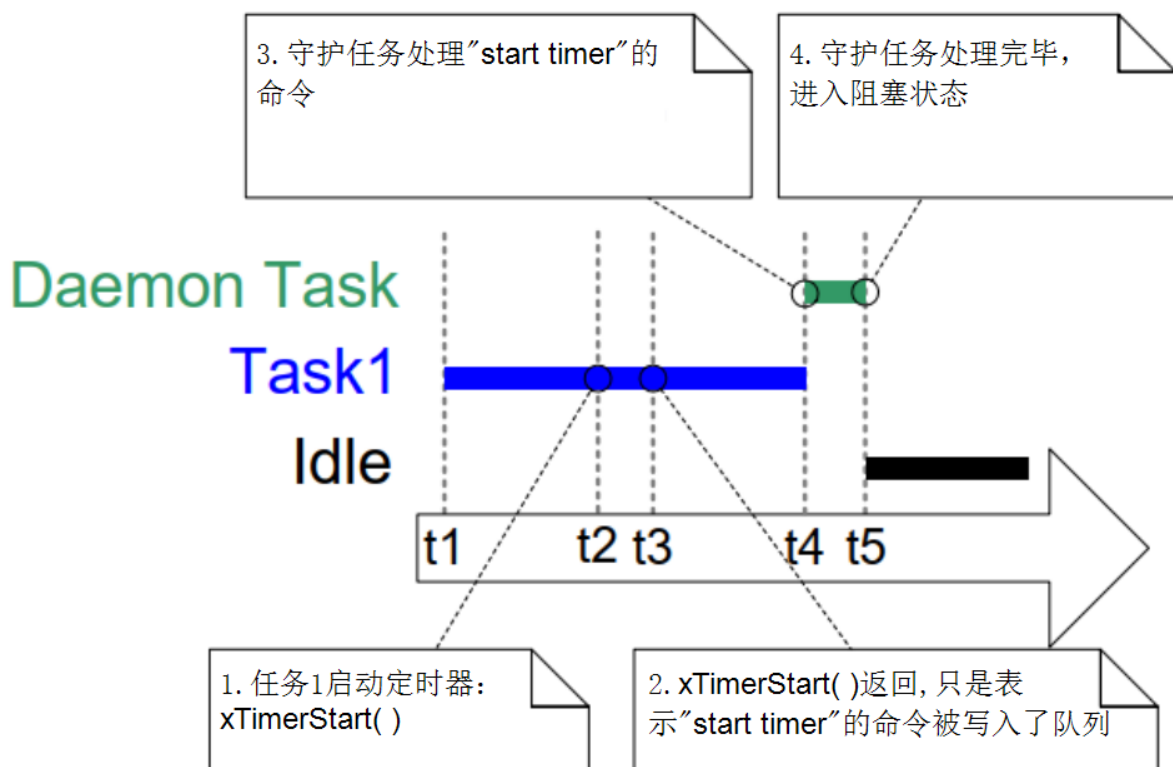
- 处理命令：从命令队列里取出命令、处理
- 执行定时器的回调函数

能否及时处理定时器的命令、能否及时执行定时器的回调函数，严重依赖于守护任务的优先级。下面使用2个例子来演示。

例子1：守护任务的优先性级较低

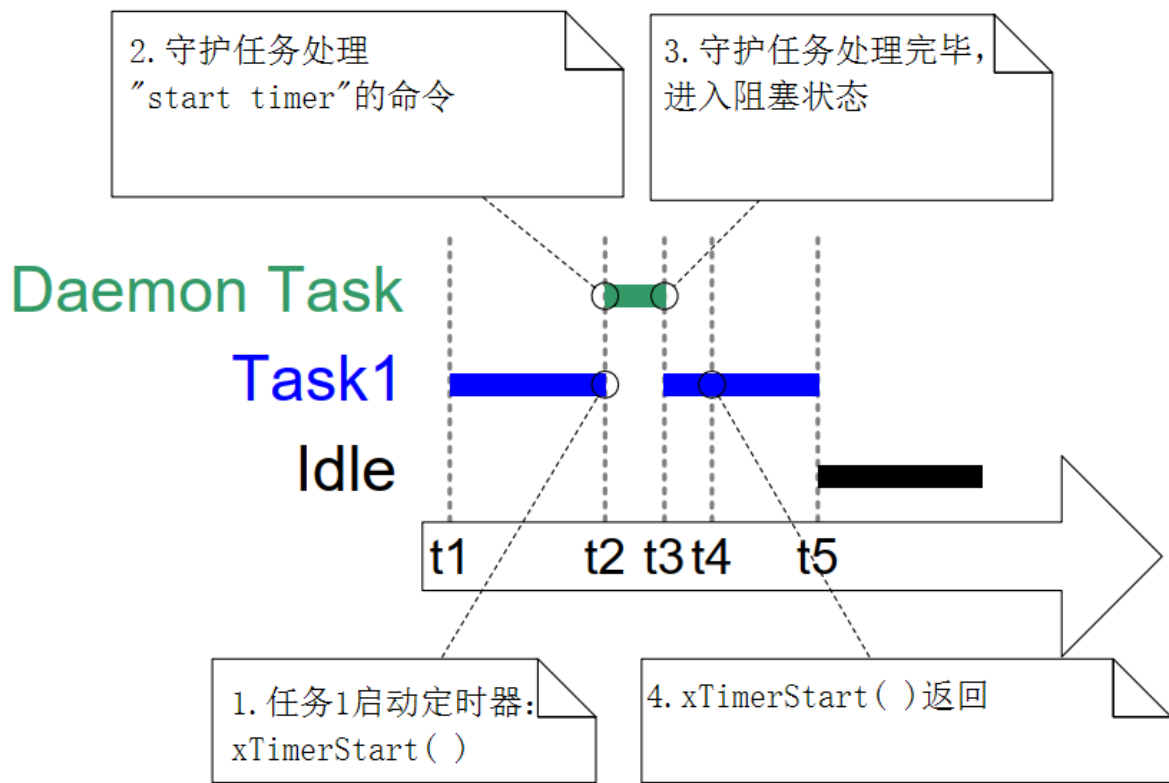
- t1：Task1处于运行态，守护任务处于阻塞态。
守护任务在这两种情况下会退出阻塞态切换为就绪态：命令队列中有数据、某个定时器超时了。
至于守护任务能否马上执行，取决于它的优先级。

- t2: Task1调用 `xTimerStart()`
要注意的是, `xTimerStart()` 只是把"start timer"的命令发给"定时器命令队列", 使得守护任务退出阻塞态。
在本例中, Task1的优先级高于守护任务, 所以守护任务无法抢占Task1。
- t3: Task1执行完 `xTimerStart()`
但是定时器的启动工作由守护任务来实现, 所以 `xTimerStart()` 返回并不表示定时器已经被启动了。
- t4: Task1由于某些原因进入阻塞态, 现在轮到守护任务运行。
守护任务从队列中取出"start timer"命令, 启动定时器。
- t5: 守护任务处理完队列中所有的命令, 再次进入阻塞态。Idle任务时优先级最高的就绪态任务, 它执行。
- 注意: 假设定时器在后续某个时刻tX超时了, 超时时间是"tX-t2", 而非"tX-t4", 从 `xTimerStart()` 函数被调用时算起。



例子2: 守护任务的优先级较高

- t1: Task1处于运行态, 守护任务处于阻塞态。
守护任务在这两种情况下会退出阻塞态切换为就绪态: 命令队列中有数据、某个定时器超时了。
至于守护任务能否马上执行, 取决于它的优先级。
- t2: Task1调用 `xTimerStart()`
要注意的是, `xTimerStart()` 只是把"start timer"的命令发给"定时器命令队列", 使得守护任务退出阻塞态。
在本例中, 守护任务的优先级高于Task1, 所以守护任务抢占Task1, 守护任务开始处理命令队列。
Task1在执行 `xTimerStart()` 的过程中被抢占, 这时它无法完成此函数。
- t3: 守护任务处理完命令队列中所有的命令, 再次进入阻塞态。
此时Task1是优先级最高的就绪态任务, 它开始执行。
- t4: Task1之前被守护任务抢占, 对 `xTimerStart()` 的调用尚未返回。现在开始继续运行次函数、返回。
- t5: Task1由于某些原因进入阻塞态, 进入阻塞态。Idle任务时优先级最高的就绪态任务, 它执行。



注意，定时器的超时时间是基于调用 `xTimerStart()` 的时刻 `tX`，而不是基于守护任务处理命令的时刻 `tY`。假设超时时间是10个Tick，超时时间是 "`tX+10`"，而非 "`tY+10`"。

10.2.3 回调函数

定时器的回调函数的原型如下：

```
void ATimerCallback( TimerHandle_t xTimer );
```

定时器的回调函数是在守护任务中被调用的，守护任务不是专为某个定时器服务的，它还要处理其他定时器。

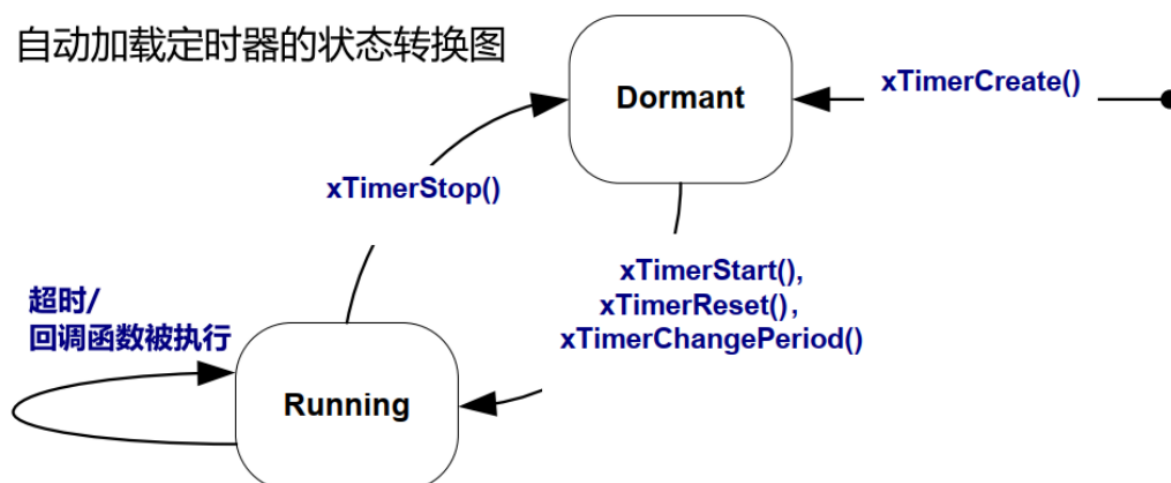
所以，定时器的回调函数不要影响其他人：

- 回调函数要尽快实行，不能进入阻塞状态
- 不要调用会导致阻塞的API函数，比如 `vTaskDelay()`
- 可以调用 `xQueueReceive()` 之类的函数，但是超时时间要设为0：即刻返回，不可阻塞

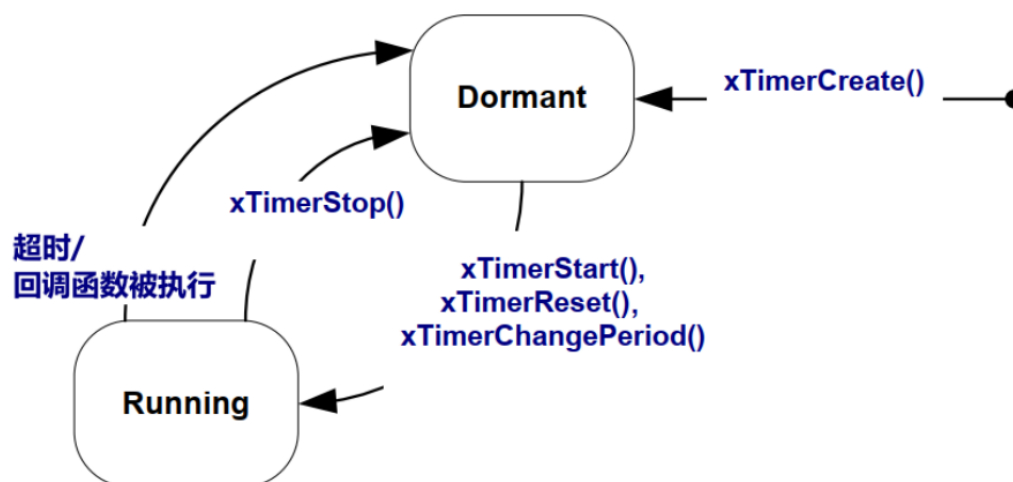
10.3 软件定时器的函数

根据定时器的状态转换图，就可以知道所涉及的函数：

自动加载定时器的状态转换图



一次性定时器的状态转换图



10.3.1 创建

要使用定时器，需要先创建它，得到它的句柄。

有两种方法创建定时器：动态分配内存、静态分配内存。函数原型如下：

```
/* 使用动态分配内存的方法创建定时器
 * pcTimerName: 定时器名字，用处不大，尽在调试时用到
 * xTimerPeriodInTicks: 周期，以Tick为单位
 * uxAutoReload: 类型，pdTRUE表示自动加载，pdFALSE表示一次性
 * pvTimerID: 回调函数可以使用此参数，比如分辨是哪个定时器
 * pxCallbackFunction: 回调函数
 * 返回值: 成功则返回TimerHandle_t，否则返回NULL
 */
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                           const TickType_t xTimerPeriodInTicks,
                           const UBaseType_t uxAutoReload,
                           void * const pvTimerID,
                           TimerCallbackFunction_t pxCallbackFunction );

/* 使用静态分配内存的方法创建定时器
```

```

    * pcTimerName: 定时器名字，用处不大，尽在调试时用到
    * xTimerPeriodInTicks: 周期，以Tick为单位
    * uxAutoReload: 类型，pdTRUE表示自动加载，pdFALSE表示一次性
    * pvTimerID: 回调函数可以使用此参数，比如分辨是哪个定时器
    * pxCallbackFunction: 回调函数
    * pxTimerBuffer: 传入一个StaticTimer_t结构体，将在上面构造定时器
    * 返回值: 成功则返回TimerHandle_t，否则返回NULL
    */
TimerHandle_t xTimerCreateStatic(const char * const pcTimerName,
                                TickType_t xTimerPeriodInTicks,
                                UBaseType_t uxAutoReload,
                                void * pvTimerID,
                                TimerCallbackFunction_t pxCallbackFunction,
                                StaticTimer_t *pxTimerBuffer );

```

回调函数的类型是：

```

void ATimerCallback( TimerHandle_t xTimer );

typedef void (* TimerCallbackFunction_t)( TimerHandle_t xTimer );

```

10.3.2 删除

动态分配的定时器，不再需要时可以删除掉以回收内存。删除函数原型如下：

```

/* 删除定时器
 * xTimer: 要删除哪个定时器
 * xTicksToWait: 超时时间
 * 返回值: pdFAIL表示"删除命令"在xTicksToWait个Tick内无法写入队列
 *         pdPASS表示成功
 */
BaseType_t xTimerDelete( TimerHandle_t xTimer, TickType_t xTicksToWait );

```

定时器的很多API函数，都是通过发送"命令"到命令队列，由守护任务来实现。

如果队列满了，"命令"就无法即刻写入队列。我们可以指定一个超时时间 `xTicksToWait`，等待一会。

10.3.3 启动/停止

启动定时器就是设置它的状态为运行态(Running、Active)。

停止定时器就是设置它的状态为冬眠(Dormant)，让它不能运行。

涉及的函数原型如下：

```

/* 启动定时器
 * xTimer: 哪个定时器
 * xTicksToWait: 超时时间
 * 返回值: pdFAIL表示"启动命令"在xTicksToWait个Tick内无法写入队列
 *         pdPASS表示成功
 */

```

```

 BaseType_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );

/* 启动定时器(ISR版本)
 * xTimer: 哪个定时器
 * pxHigherPriorityTaskWoken: 向队列发出命令使得守护任务被唤醒,
 *                            如果守护任务的优先级比当前任务的高,
 *                            则"*pxHigherPriorityTaskWoken = pdTRUE",
 *                            表示需要进行任务调度
 * 返回值: pdFAIL表示"启动命令"无法写入队列
 *         pdPASS表示成功
 */
 BaseType_t xTimerStartFromISR(   TimerHandle_t xTimer,
                                   BaseType_t *pxHigherPriorityTaskWoken );

/* 停止定时器
 * xTimer: 哪个定时器
 * xTicksToWait: 超时时间
 * 返回值: pdFAIL表示"停止命令"在xTicksToWait个Tick内无法写入队列
 *         pdPASS表示成功
 */
 BaseType_t xTimerStop( TimerHandle_t xTimer, TickType_t xTicksToWait );

/* 停止定时器(ISR版本)
 * xTimer: 哪个定时器
 * pxHigherPriorityTaskWoken: 向队列发出命令使得守护任务被唤醒,
 *                            如果守护任务的优先级比当前任务的高,
 *                            则"*pxHigherPriorityTaskWoken = pdTRUE",
 *                            表示需要进行任务调度
 * 返回值: pdFAIL表示"停止命令"无法写入队列
 *         pdPASS表示成功
 */
 BaseType_t xTimerStopFromISR(   TimerHandle_t xTimer,
                                   BaseType_t *pxHigherPriorityTaskWoken );

```

注意，这些函数的 `xTicksToWait` 表示的是，把命令写入命令队列的超时时间。命令队列可能已经满了，无法马上把命令写入队列里，可以等待一会。

`xTicksToWait` 不是定时器本身的超时时间，不是定时器本身的"周期"。

创建定时器时，设置了它的周期(period)。`xTimerStart()` 函数是用来启动定时器。假设调用 `xTimerStart()` 的时刻是 t_X ，定时器的周期是 n ，那么在 $t_X + n$ 时刻定时器的回调函数被调用。

如果定时器已经被启动，但是它的函数尚未被执行，再次执行 `xTimerStart()` 函数相当于执行 `xTimerReset()`，重新设定它的启动时间。

10.3.4 复位

从定时器的状态转换图可以知道，使用 `xTimerReset()` 函数可以让定时器的状态从冬眠态转换为运行态，相当于使用 `xTimerStart()` 函数。

如果定时器已经处于运行态，使用 `xTimerReset()` 函数就相当于重新确定超时时间。假设调用 `xTimerReset()` 的时刻是 t_X ，定时器的周期是 n ，那么 $t_X + n$ 就是重新确定的超时时间。

复位函数的原型如下：

```

/* 复位定时器

```

```

* xTimer: 哪个定时器
* xTicksToWait: 超时时间
* 返回值: pdFAIL表示"复位命令"在xTicksToWait个Tick内无法写入队列
*          pdPASS表示成功
*/
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );

/* 复位定时器(ISR版本)
* xTimer: 哪个定时器
* pxHigherPriorityTaskWoken: 向队列发出命令使得守护任务被唤醒,
*                            如果守护任务的优先级比当前任务的高,
*                            则"*pxHigherPriorityTaskWoken = pdTRUE",
*                            表示需要进行任务调度
* 返回值: pdFAIL表示"停止命令"无法写入队列
*          pdPASS表示成功
*/
BaseType_t xTimerResetFromISR( TimerHandle_t xTimer,
                               BaseType_t *pxHigherPriorityTaskWoken );

```

10.3.5 修改周期

从定时器的状态转换图可以知道, 使用 `xTimerChangePeriod()` 函数, 处理能修改它的周期外, 还可以让定时器的状态从冬眠态转换为运行态。

修改定时器的周期时, 会使用新的周期重新计算它的超时时间。假设调用 `xTimerChangePeriod()` 函数的时间 t_x , 新的周期是 n , 则 $t_x + n$ 就是新的超时时间。

相关函数的原型如下:

```

/* 修改定时器的周期
* xTimer: 哪个定时器
* xNewPeriod: 新周期
* xTicksToWait: 超时时间, 命令写入队列的超时时间
* 返回值: pdFAIL表示"修改周期命令"在xTicksToWait个Tick内无法写入队列
*          pdPASS表示成功
*/
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,
                               TickType_t xNewPeriod,
                               TickType_t xTicksToWait );

/* 修改定时器的周期
* xTimer: 哪个定时器
* xNewPeriod: 新周期
* pxHigherPriorityTaskWoken: 向队列发出命令使得守护任务被唤醒,
*                            如果守护任务的优先级比当前任务的高,
*                            则"*pxHigherPriorityTaskWoken = pdTRUE",
*                            表示需要进行任务调度
* 返回值: pdFAIL表示"修改周期命令"在xTicksToWait个Tick内无法写入队列
*          pdPASS表示成功
*/
BaseType_t xTimerChangePeriodFromISR( TimerHandle_t xTimer,
                                       TickType_t xNewPeriod,
                                       BaseType_t *pxHigherPriorityTaskWoken );

```

10.3.6 定时器ID

定时器的结构体如下，里面有一项 `pvTimerID`，它就是定时器ID：

```
typedef struct tmrTimerControl
{
    const char * pcTimerName;
    ListItem_t xTimerListItem;
    TickType_t xTimerPeriodInTicks;
    void * pvTimerID;
    TimerCallbackFunction_t pxCallbackFunction;
    #if ( configUSE_TRACE_FACILITY == 1 )
        UBaseType_t uxTimerNumber;
    #endif
    uint8_t ucStatus;
} xTIMER;
```

怎么使用定时器ID，完全由程序来决定：

- 可以用来标记定时器，表示自己是什么定时器
- 可以用来保存参数，给回调函数使用

它的初始值在创建定时器时由 `xTimerCreate()` 这类函数传入，后续可以使用这些函数来操作：

- 更新ID：使用 `vTimerSetTimerID()` 函数
- 查询ID：查询 `pvTimerGetTimerID()` 函数

这两个函数不涉及命令队列，它们是直接操作定时器结构体。

函数原型如下：

```
/* 获得定时器的ID
 * xTimer: 哪个定时器
 * 返回值: 定时器的ID
 */
void *pvTimerGetTimerID( TimerHandle_t xTimer );

/* 设置定时器的ID
 * xTimer: 哪个定时器
 * pvNewID: 新ID
 * 返回值: 无
 */
void vTimerSetTimerID( TimerHandle_t xTimer, void *pvNewID );
```

10.4 示例24: 一般使用

本节程序为 `FreeRTOS_24_software_timer`。

要使用定时器，需要做些准备工作：

```
/* 1. 工程中 */
添加 timer.c

/* 2. 配置文件FreeRTOSConfig.h中 */
#define configUSE_TIMERS 1 /* 使能定时器 */
#define configTIMER_TASK_PRIORITY 31 /* 守护任务的优先级，尽可能高一些 */
#define configTIMER_QUEUE_LENGTH 5 /* 命令队列长度 */
#define configTIMER_TASK_STACK_DEPTH 32 /* 守护任务的栈大小 */

/* 3. 源码中 */
#include "timers.h"
```

main函数中创建、启动了2个定时器：一次性的、周期

```
static volatile uint8_t flagONEShotTimerRun = 0;
static volatile uint8_t flagAutoLoadTimerRun = 0;

static void vONEShotTimerFunc( TimerHandle_t xTimer );
static void vAutoLoadTimerFunc( TimerHandle_t xTimer );

/*-----*/

#define mainONE_SHOT_TIMER_PERIOD pdMS_TO_TICKS( 10 )
#define mainAUTO_RELOAD_TIMER_PERIOD pdMS_TO_TICKS( 20 )

int main( void )
{
    TimerHandle_t xOneShotTimer;
    TimerHandle_t xAutoReloadTimer;

    prvSetupHardware();

    xOneShotTimer = xTimerCreate(
        "OneShot", /* 名字，不重要 */
        mainONE_SHOT_TIMER_PERIOD, /* 周期 */
        pdFALSE, /* 一次性 */
        0, /* ID */
        vONEShotTimerFunc /* 回调函数 */
    );

    xAutoReloadTimer = xTimerCreate(
        "AutoReload", /* 名字，不重要 */
        mainAUTO_RELOAD_TIMER_PERIOD, /* 周期 */
        pdTRUE, /* 自动加载 */
        0, /* ID */
        vAutoLoadTimerFunc /* 回调函数 */
    );
}
```

```

if (xOneShotTimer && xAutoReloadTimer)
{
    /* 启动定时器 */
    xTimerStart(xOneShotTimer, 0);
    xTimerStart(xAutoReloadTimer, 0);

    /* 启动调度器 */
    vTaskStartScheduler();
}

/* 如果程序运行到了这里就表示出错了，一般是内存不足 */
return 0;
}

```

这两个定时器的回调函数比较简单：

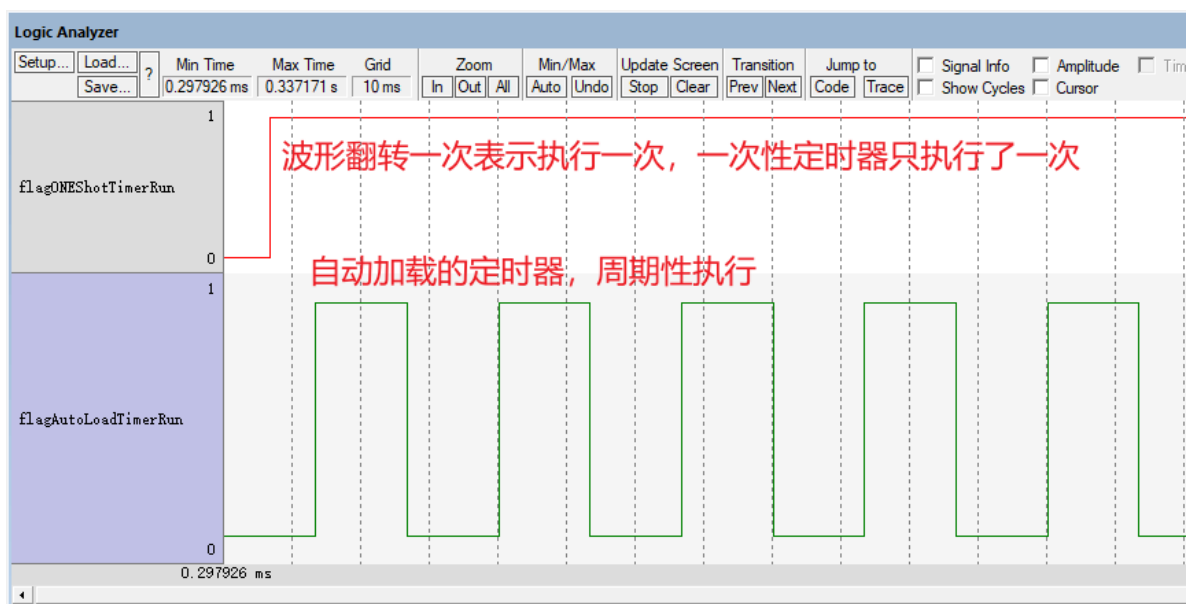
```

static void vONeshotTimerFunc( TimerHandle_t xTimer )
{
    static int cnt = 0;
    flagONeshotTimerRun = !flagONeshotTimerRun;
    printf("run vONeshotTimerFunc %d\r\n", cnt++);
}

static void vAutoLoadTimerFunc( TimerHandle_t xTimer )
{
    static int cnt = 0;
    flagAutoLoadTimerRun = !flagAutoLoadTimerRun;
    printf("run vAutoLoadTimerFunc %d\r\n", cnt++);
}

```

逻辑分析仪如下图所示：



运行结果如下图所示：

UART #1

```
run vONEShotTimerFunc 0
run vAutoLoadTimerFunc 0
run vAutoLoadTimerFunc 1
run vAutoLoadTimerFunc 2
run vAutoLoadTimerFunc 3
run vAutoLoadTimerFunc 4
run vAutoLoadTimerFunc 5
run vAutoLoadTimerFunc 6
run vAutoLoadTimerFunc 7
run vAutoLoadTimerFunc 8
```

10.5 示例25: 消除抖动

本节程序为 `FreeRTOS_25_software_timer_readkey`。

在嵌入式开发中，我们使用机械开关时经常碰到抖动问题：引脚电平在短时间内反复变化。

怎么读到确定的按键状态？

- 连续读很多次，知道数值稳定：浪费CPU资源
- 使用定时器：要结合中断来使用

对于第2种方法，处理方法如下图所示，按下按键后：

- 在 t_1 产生中断，这时不马上确定按键，而是复位定时器，假设周期为20ms，超时时间为" $t_1+20\text{ms}$ "
- 由于抖动，在 t_2 再次产生中断，再次复位定时器，超时时间变为" $t_2+20\text{ms}$ "
- 由于抖动，在 t_3 再次产生中断，再次复位定时器，超时时间变为" $t_3+20\text{ms}$ "
- 在" $t_3+20\text{ms}$ "处，按键已经稳定，读取按键值

按键引脚电平



- t3. 发生第3次中断，复位定时器，20ms后超时
- t2. 发生第2次中断，复位定时器，20ms后超时
- t1. 发生第1次中断，复位定时器，20ms后超时

main函数中创建了一个一次性的定时器，用来处理抖动；创建了一个任务，用来模拟产生抖动。代码如下：

```
/*-----*/  
  
static void vKeyFilteringTimerFunc( TimerHandle_t xTimer );  
void vEmulateKeyTask( void *pvParameters );  
  
static TimerHandle_t xKeyFilteringTimer;  
  
/*-----*/  
  
#define KEY_FILTERING_PERIOD pdMS_TO_TICKS( 20 )  
  
int main( void )
```

```

{

    prvSetupHardware();

    xKeyFilteringTimer = xTimerCreate(
        "KeyFiltering",           /* 名字，不重要 */
        KEY_FILTERING_PERIOD,    /* 周期 */
        pdFALSE,                 /* 一次性 */
        0,                       /* ID */
        vKeyFilteringTimerFunc    /* 回调函数 */
    );

    /* 在这个任务中多次调用xTimerReset来模拟按键抖动 */
    xTaskCreate( vEmulateKeyTask, "EmulateKey", 1000, NULL, 1, NULL );

    /* 启动调度器 */
    vTaskStartScheduler();

    /* 如果程序运行到了这里就表示出错了，一般是内存不足 */
    return 0;
}

```

模拟产生按键：每个循环里调用3次xTimerReset，代码如下：

```

void vEmulateKeyTask( void *pvParameters )
{
    int cnt = 0;
    const TickType_t xDelayTicks = pdMS_TO_TICKS( 200UL );

    for( ;; )
    {
        /* 模拟按键抖动，多次调用xTimerReset */
        xTimerReset(xKeyFilteringTimer, 0); cnt++;
        xTimerReset(xKeyFilteringTimer, 0); cnt++;
        xTimerReset(xKeyFilteringTimer, 0); cnt++;

        printf("Key jitters %d\r\n", cnt);

        vTaskDelay(xDelayTicks);
    }
}

```

定时器回调函数代码如下：

```

static void vKeyFilteringTimerFunc( TimerHandle_t xTimer )
{
    static int cnt = 0;
    printf("vKeyFilteringTimerFunc %d\r\n", cnt++);
}

```

在人户函数中多次调用xTimerReset，只触发1次定时器回调函数，运行结果如下图所示：

UART #1	
Key jitters 3	
vKeyFilteringTimerFunc 0	
Key jitters 6	
vKeyFilteringTimerFunc 1	
Key jitters 9	
vKeyFilteringTimerFunc 2	
Key jitters 12	
vKeyFilteringTimerFunc 3	
Key jitters 15	
vKeyFilteringTimerFunc 4	
Key jitters 18	
vKeyFilteringTimerFunc 5	
Key jitters 21	
vKeyFilteringTimerFunc 6	
Key jitters 24	
vKeyFilteringTimerFunc 7	
Key jitters 27	
vKeyFilteringTimerFunc 8	

第十一章 中断管理(Interrupt Management)

在RTOS中，需要应对各类事件。这些事件很多时候是通过硬件中断产生，怎么处理中断呢？

假设当前系统正在运行Task1时，用户按下了按键，触发了按键中断。这个中断的处理流程如下：

- CPU跳到固定地址去执行代码，这个固定地址通常被称为中断向量，这个跳转时硬件实现的
- 执行代码做什么？
 - 保存现场：Task1被打断，需要先保存Task1的运行环境，比如各类寄存器的值
 - 分辨中断、调用处理函数(这个函数就被称为ISR，interrupt service routine)
 - 恢复现场：继续运行Task1，或者运行其他优先级更高的任务

你要注意到，ISR是在内核中被调用的，ISR执行过程中，用户的任务无法执行。ISR要尽量快，否则：

- 其他低优先级的中断无法被处理：实时性无法保证
- 用户任务无法被执行：系统显得很卡顿

如果这个硬件中断的处理，就是非常耗费时间呢？对于这类中断的处理就要分为2部分：

- ISR：尽快做些清理、记录工作，然后触发某个任务
- 任务：更复杂的事情放在任务中处理
- 所以：需要ISR和任务之间进行通信

要在FreeRTOS中熟练使用中断，有几个原则要先说明：

- FreeRTOS把任务认为是硬件无关的，任务的优先级由程序员决定，任务何时运行由调度器决定
- ISR虽然也是使用软件实现的，但是它被认为是硬件特性的一部分，因为它跟硬件密切相关
 - 何时执行？由硬件决定
 - 哪个ISR被执行？由硬件决定
- ISR的优先级高于任务：即使是优先级最低的中断，它的优先级也高于任务。任务只有在没有中断的情况下，才能执行。

本章涉及如下内容：

- FreeRTOS的哪些API函数能在ISR中使用
- 怎么把中断的处理分为两部分：ISR、任务
- ISR和任务之间的通信

11.1 两套API函数

11.1.1 为什么需要两套API

在任务函数中，我们可以调用各类API函数，比如队列操作函数：xQueueSendToBack。但是在ISR中使用这个函数会导致问题，应该使用另一个函数：xQueueSendToBackFromISR，它的函数名含有后缀"FromISR"，表示"从ISR中给队列发送数据"。

FreeRTOS中很多API函数都有两套：一套在任务中使用，另一套在ISR中使用。后者的函数名含有"FromISR"后缀。

为什么要引入两套API函数？

- 很多API函数会导致任务计入阻塞状态：
 - 运行这个函数的**任务**进入阻塞状态
 - 比如写队列时，如果队列已满，可以进入阻塞状态等待一会
- ISR调用API函数时，ISR不是"任务"，ISR不能进入阻塞状态
- 所以，在任务中、在ISR中，这些函数的功能是有差别的

为什么不使用同一套函数，比如在函数里面分辨当前调用者是任务还是ISR呢？示例代码如下：

```
BaseType_t xQueueSend(...)
{
    if (is_in_isr())
    {
        /* 把数据放入队列 */

        /* 不管是否成功都直接返回 */
    }
    else /* 在任务中 */
    {
        /* 把数据放入队列 */
        /* 不成功就等待一会再重试 */
    }
}
```

FreeRTOS使用两套函数，而不是使用一套函数，是因为有如下好处：

- 使用同一套函数的话，需要增加额外的判断代码、增加额外的分支，是的函数更长、更复杂、难以测试
- 在任务、ISR中调用时，需要的参数不一样，比如：
 - 在任务中调用：需要指定超时时间，表示如果不成功就阻塞一会
 - 在ISR中调用：不需要指定超时时间，无论是否成功都要即刻返回
 - 如果强行把两套函数揉在一起，会导致参数臃肿、无效
- 移植FreeRTOS时，还需要提供监测上下文的函数，比如 `is_in_isr()`
- 有些处理器架构没有办法轻易分辨当前是处于任务中，还是处于ISR中，就需要额外添加更多、更复杂的代码

使用两套函数可以让程序更高效，但是也有一些缺点，比如你要使用第三方库函数时，即会在任务中调用它，也会在ISR总调用它。这个第三方库函数用到了FreeRTOS的API函数，你无法修改库函数。这个问题可以解决：

- 把中断的处理推迟到任务中进行(Defer interrupt processing)，在任务中调用库函数
- 尝试在库函数中使用"FromISR"函数：
 - 在任务中、在ISR中都可以调用"FromISR"函数
 - 反过来就不行，非FromISR函数无法在ISR中使用
- 第三方库函数也许会提供OS抽象层，自行判断当前环境是在任务还是在ISR中，分别调用不同的函数

11.1.2 两套API函数列表

类型	在任务中	在ISR中
队列(queue)	xQueueSendToBack	xQueueSendToBackFromISR
	xQueueSendToFront	xQueueSendToFrontFromISR
	xQueueReceive	xQueueReceiveFromISR
	xQueueOverwrite	xQueueOverwriteFromISR
	xQueuePeek	xQueuePeekFromISR
信号量(semaphore)	xSemaphoreGive	xSemaphoreGiveFromISR
	xSemaphoreTake	xSemaphoreTakeFromISR

类型	在任务中	在ISR中
事件组(event group)	xEventGroupSetBits	xEventGroupSetBitsFromISR
	xEventGroupGetBits	xEventGroupGetBitsFromISR
任务通知(task notification)	xTaskNotifyGive	vTaskNotifyGiveFromISR
	xTaskNotify	xTaskNotifyFromISR
软件定时器(software timer)	xTimerStart	xTimerStartFromISR
	xTimerStop	xTimerStopFromISR
	xTimerReset	xTimerResetFromISR
	xTimerChangePeriod	xTimerChangePeriodFromISR

11.1.3 xHigherPriorityTaskWoken参数

xHigherPriorityTaskWoken的含义是：是否有更高优先级的任务被唤醒了。如果为pdTRUE，则意味着后面要进行任务切换。

还是以写队列为例。

任务A调用 xQueueSendToBack() 写队列，有几种情况发生：

- 队列满了，任务A阻塞等待，另一个任务B运行
- 队列没满，任务A成功写入队列，但是它导致另一个任务B被唤醒，任务B的优先级更高：任务B先运行
- 队列没满，任务A成功写入队列，即刻返回

可以看到，在任务中调用API函数可能导致任务阻塞、任务切换，这叫做"context switch"，上下文切换。这个函数可能很长时间才返回，在函数的内部实现了任务切换。

xQueueSendToBackFromISR() 函数也可能导致任务切换，但是不会在函数内部进行切换，而是返回一个参数：表示是否需要切换，函数原型与用法如下：

```

/*
 * 往队列尾部写入数据，此函数可以在中断函数中使用，不可阻塞
 */
BaseType_t xQueueSendToBackFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskwoken
);

/* 用法示例 */

BaseType_t xHigherPriorityTaskwoken = pdFALSE;
xQueueSendToBackFromISR(xQueue, pvItemToQueue, &xHigherPriorityTaskwoken);

if (xHigherPriorityTaskwoken == pdTRUE)
{
    /* 任务切换 */
}

```

```
}
```

pxHigherPriorityTaskWoken参数，就是用来保存函数的结果：是否需要切换

- *pxHigherPriorityTaskWoken等于pdTRUE：函数的操作导致更高优先级的任务就绪了，ISR应该进行任务切换
- *pxHigherPriorityTaskWoken等于pdFALSE：没有进行任务切换的必要

为什么不在"FromISR"函数内部进行任务切换，而只是标记一下而已呢？为了效率！示例代码如下：

```
void XXX_ISR()
{
    int i;
    for (i = 0; i < N; i++)
    {
        xQueueSendToBackFromISR(...); /* 被多次调用 */
    }
}
```

ISR中有可能多次调用"FromISR"函数，如果在"FromISR"内部进行任务切换，会浪费时间。解决方法是：

- 在"FromISR"中标记是否需要切换
- 在ISR返回之前再进行任务切换
- 示例代码如下

```
void XXX_ISR()
{
    int i;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    for (i = 0; i < N; i++)
    {
        xQueueSendToBackFromISR(..., &xHigherPriorityTaskWoken); /* 被多次调用 */
    }

    /* 最后再决定是否进行任务切换 */
    if (xHigherPriorityTaskWoken == pdTRUE)
    {
        /* 任务切换 */
    }
}
```

上述的例子很常见，比如UART中断：在UART的ISR中读取多个字符，发现收到回车符时才进行任务切换。

在ISR中调用API时不进行任务切换，而只是在"xHigherPriorityTaskWoken"中标记一下，除了效率，还有多种好处：

- 效率高：避免不必要的任务切换
- 让ISR更可控：中断随机产生，在API中进行任务切换的话，可能导致问题更复杂
- 可移植性
- 在Tick中断中，调用 `vApplicationTickHook()`：它运行与ISR，只能使用"FromISR"的函数

使用"FromISR"函数时，如果不想使用xHigherPriorityTaskWoken参数，可以设置为NULL。

11.1.4 怎么切换任务

FreeRTOS的ISR函数中，使用两个宏进行任务切换：

```
portEND_SWITCHING_ISR( xHigherPriorityTaskWoken );  
或  
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
```

这两个宏做的事情是完全一样的，在老版本的FreeRTOS中，

- `portEND_SWITCHING_ISR` 使用汇编实现
- `portYIELD_FROM_ISR` 使用C语言实现

新版本都统一使用 `portYIELD_FROM_ISR`。

使用示例如下：

```
void XXX_ISR()  
{  
    int i;  
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;  
  
    for (i = 0; i < N; i++)  
    {  
        xQueueSendToBackFromISR(..., &xHigherPriorityTaskWoken); /* 被多次调用 */  
    }  
  
    /* 最后再决定是否进行任务切换  
     * xHigherPriorityTaskWoken为pdTRUE时才切换  
     */  
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);  
}
```

11.2 中断的延迟处理

前面讲过，ISR要尽量快，否则：

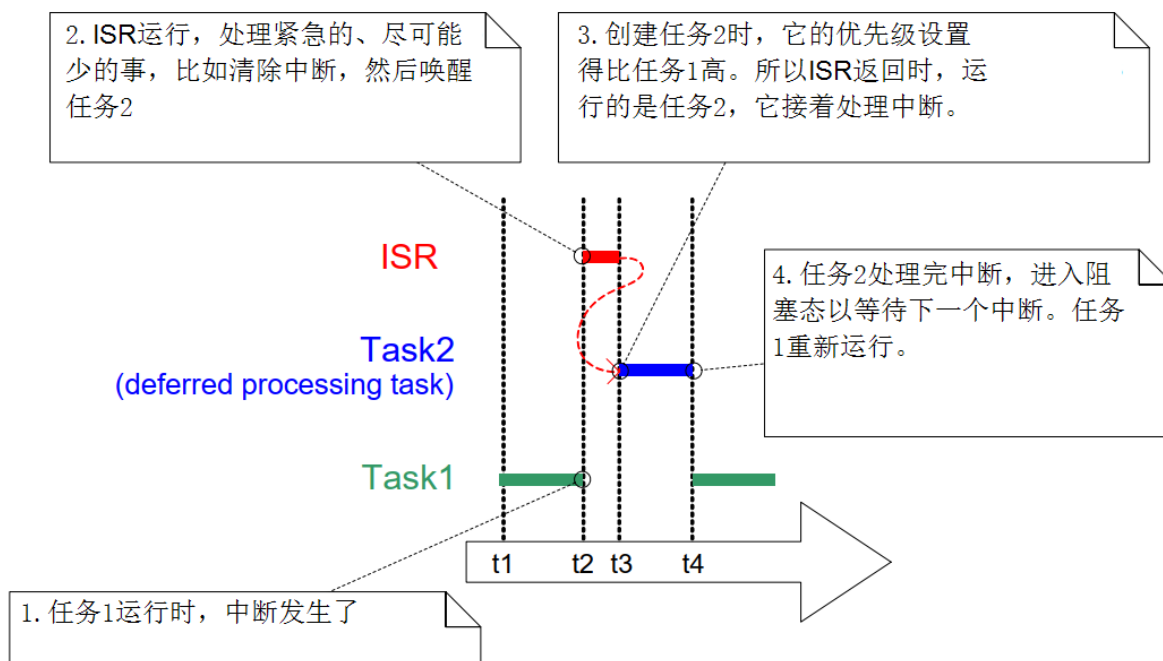
- 其他低优先级的中断无法被处理：实时性无法保证
- 用户任务无法被执行：系统显得很卡顿
- 如果运行中断嵌套，这会更复杂，ISR越快执行约有助于中断嵌套

如果这个硬件中断的处理，就是非常耗费时间呢？对于这类中断的处理就要分为2部分：

- ISR：尽快做些清理、记录工作，然后触发某个任务
- 任务：更复杂的事情放在任务中处理

这种处理方式叫"中断的延迟处理"(Deferring interrupt processing)，处理流程如下图所示：

- t1：任务1运行，任务2阻塞
- t2：发生中断，
 - 该中断的ISR函数被执行，任务1被打断
 - ISR函数要尽快能快速地运行，它做一些必要的操作(比如清除中断)，然后唤醒任务2
- t3：在创建任务时设置任务2的优先级比任务1高(这取决于设计者)，所以ISR返回后，运行的是任务2，它要完成中断的处理。任务2就被称为"deferred processing task"，中断的延迟处理任务。
- t4：任务2处理完中断后，进入阻塞态以等待下一个中断，任务1重新运行



11.3 中断与任务间的通信

前面讲解过的队列、信号量、互斥量、事件组、任务通知等等方法，都可使用。

要注意的是，在ISR中使用的函数要有"FromISR"后缀。

第十二章 资源管理(Resource Management)

在前面讲解互斥量时，引入过临界资源的概念。在前面课程里，已经实现了临界资源的互斥访问。

本章节的内容比较少，只是引入两个功能：屏蔽/使能中断、暂停/恢复调度器。

要独占式地访问临界资源，有3种方法：

- 公平竞争：比如使用互斥量，谁先获得互斥量谁就访问临界资源，这部分内容前面讲过。
- 谁要跟我抢，我就灭掉谁：
 - 中断要跟我抢？我屏蔽中断
 - 其他任务要跟我抢？我禁止调度器，不运行任务切换

12.1 屏蔽中断

屏蔽中断有两套宏：任务中使用、ISR中使用：

- 任务中使用： `taskENTER_CRITICAL()/taskEXIT_CRITICAL()`
- ISR中使用： `taskENTER_CRITICAL_FROM_ISR()/taskEXIT_CRITICAL_FROM_ISR()`

12.1.1 在任务中屏蔽中断

在任务中屏蔽中断的示例代码如下：

```
/* 在任务中，当前时刻中断是使能的
 * 执行这句代码后，屏蔽中断
 */
taskENTER_CRITICAL();

/* 访问临界资源 */

/* 重新使能中断 */
taskEXIT_CRITICAL();
```

在 `taskENTER_CRITICAL()/taskEXIT_CRITICAL()` 之间：

- 低优先级的中断被屏蔽了：优先级低于、等于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`
- 高优先级的中断可以产生：优先级高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`
 - 但是，这些中断ISR里，不允许使用FreeRTOS的API函数
- 任务调度依赖于中断、依赖于API函数，所以：这两段代码之间，不会有任务调度产生

这套 `taskENTER_CRITICAL()/taskEXIT_CRITICAL()` 宏，是可以递归使用的，它的内部会记录嵌套的深度，只有嵌套深度变为0时，调用 `taskEXIT_CRITICAL()` 才会重新使能中断。

使用 `taskENTER_CRITICAL()/taskEXIT_CRITICAL()` 来访问临界资源是很粗鲁的方法：

- 中断无法正常运行
- 任务调度无法进行
- 所以，之间的代码要尽可能快速地执行

12.1.2 在ISR中屏蔽中断

要使用含有"FROM_ISR"后缀的宏，示例代码如下：

```
void vAnInterruptServiceRoutine( void )
{
    /* 用来记录当前中断是否使能 */
    UBaseType_t uxSavedInterruptStatus;

    /* 在ISR中，当前时刻中断可能是使能的，也可能是禁止的
     * 所以要记录当前状态，后面要恢复为原先的状态
     */
}
```

```

    * 执行这句代码后，屏蔽中断
    */
    uxSavedInterruptStatus = taskENTER_CRITICAL_FROM_ISR();

    /* 访问临界资源 */

    /* 恢复中断状态 */
    taskEXIT_CRITICAL_FROM_ISR( uxSavedInterruptStatus );
    /* 现在，当前ISR可以被更高优先级的中断打断了 */
}

```

在 `taskENTER_CRITICAL_FROM_ISR()`/`taskEXIT_CRITICAL_FROM_ISR()` 之间：

- 低优先级的中断被屏蔽了：优先级低于、等于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`
- 高优先级的中断可以产生：优先级高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`
 - 但是，这些中断ISR里，不允许使用FreeRTOS的API函数
- 任务调度依赖于中断、依赖于API函数，所以：这两段代码之间，不会有任务调度产生

12.2 暂停调度器

如果有别的任务来跟你竞争临界资源，你可以把中断关掉：这当然可以禁止别的任务运行，但是这代价太大了。它会影响中断的处理。

如果只是禁止别的任务来跟你竞争，不需要关中断，暂停调度器就可以了：在这期间，中断还是会发生、处理。

使用这2个函数来暂停、恢复调度器：

```
/* 暂停调度器 */
void vTaskSuspendAll( void );

/* 恢复调度器
 * 返回值：pdTRUE表示在暂定期间有更高优先级的任务就绪了
 *         可以不理睬这个返回值
 */
 BaseType_t xTaskResumeAll( void );
```

示例代码如下：

```
vTaskSuspendScheduler();

/* 访问临界资源 */

xTaskResumeScheduler();
```

这套 `vTaskSuspendScheduler()/xTaskResumeScheduler()` 宏，是可以递归使用的，它的内部会记录嵌套的深度，只有嵌套深度变为0时，调用 `taskEXIT_CRITICAL()` 才会重新使能中断。

第十三章 调试方法

待续。